

SRv6 Deep Dive

One packet, one topology, five questions

Quintus Zhu

CCDE #20240069 · CCIE #15189 (R&S, SP, Wireless)

Cisco IOS XR 26.2.2 · uSID f3216 · Lab platform: XRd vRouter on CML

Why these experiments exist, and what sets them apart

Why these experiments exist

- SRv6 instructions hide inside a 128-bit address: the configuration does not show them
- A successful commit or ping does not prove the data plane works
- Goal: see one packet and say who runs which instruction
- Every statement needs evidence from the lab

What sets them apart

- Ordered by the five questions a packet must answer
- Every conclusion comes from a capture or router output; quoted documents are marked apart from measurements
- One lab from day one to graduation
- Every step is Predict → Observe → Explain

1 topology

5 questions

12 modules

24 episodes

The one idea: the destination address is a program



The router that owns the active instruction executes it, shifts the program left by 16 bits, and forwards on the new DA (destination address).

- Every other router is plain transit: IPv6 longest-match on the DA, without reading or changing the program.
- So learning SRv6 is asking one thing repeatedly: which cell is the active instruction, and who owns it?

Five questions every packet must answer

1	**Where am I going?**	**Locator reachability**	IS-IS puts each /48 locator into every RIB and CEF
2	**What do I do on arrival?**	**Local SID and behavior**	uN, uA, uDT, uDX2: the local SID table decides the action
3	**Who am I carrying?**	**Service**	BGP binds a tenant route to a service SID
4	**What if the path breaks?**	**Protection**	TI-LFA precomputes a repair list; microloop avoidance
5	**Who chose this path?**	**Intent**	SR Policy, ODN and Flex-Algo express path intent

The method: Predict → Observe → Explain

1 Predict

Before touching anything, write the prediction: which field changes first, and what evidence would refute it?

→

2 Observe

Change one thing. Read the show output and capture packets on the CML link.

→

3 Explain

When prediction and observation differ, explain the gap first. Then decide whether the config needs to change.

Every conclusion carries an evidence label

Standard	Lab	Cisco guide	Design choice	Not verified
defined by an RFC	output or capture from our routers	Cisco documentation, when we cannot test it	chosen so you can build the same lab	no document and no lab evidence

Experiment map: 5 parts, 12 modules

01

The packet

Where am I going? What do I do there?

- M1 SID anatomy
- M2 uSID shift
- M3 Encapsulation and MTU

02

Reachability

How the network learns a locator

- M4 Bring-up and acceptance levels
- M5 IS-IS: LSP → RIB → CEF

03

Services

Who am I carrying?

- M6 L3VPN and the resolution chain
- M7 EVPN: unicast and BUM

04

Resilience and intent

What if it breaks? Who chose the path?

- M8 TI-LFA
- M9 SR Policy and ODN
- M10 Flex-Algo

05

Operate

Running it in production

- M11 Five-layer troubleshooting
- M12 Migration, boundary, graduation

Experiment map: 24 episodes in 6 modules

1 Ep 1–3

Foundations

- 1 Where state lives
- 2 Build the lab
- 3 SIDs and two lookups

2 Ep 4–7

Data plane

- 4 Endpoint behaviors
- 5 uSID shift
- 6 SRH and size
- 7 MTU and ICMP

3 Ep 8–11

Underlay

- 8 IS-IS to CEF
- 9 TI-LFA
- 10 Microloop and BFD
- 11 Flex-Algo

4 Ep 12–16

Services

- 12 L3VPN
- 13 Resolution chain
- 14 EVPN
- 15 SR Policy and BSID
- 16 ODN needs a PCE

5 Ep 17–19

Operations

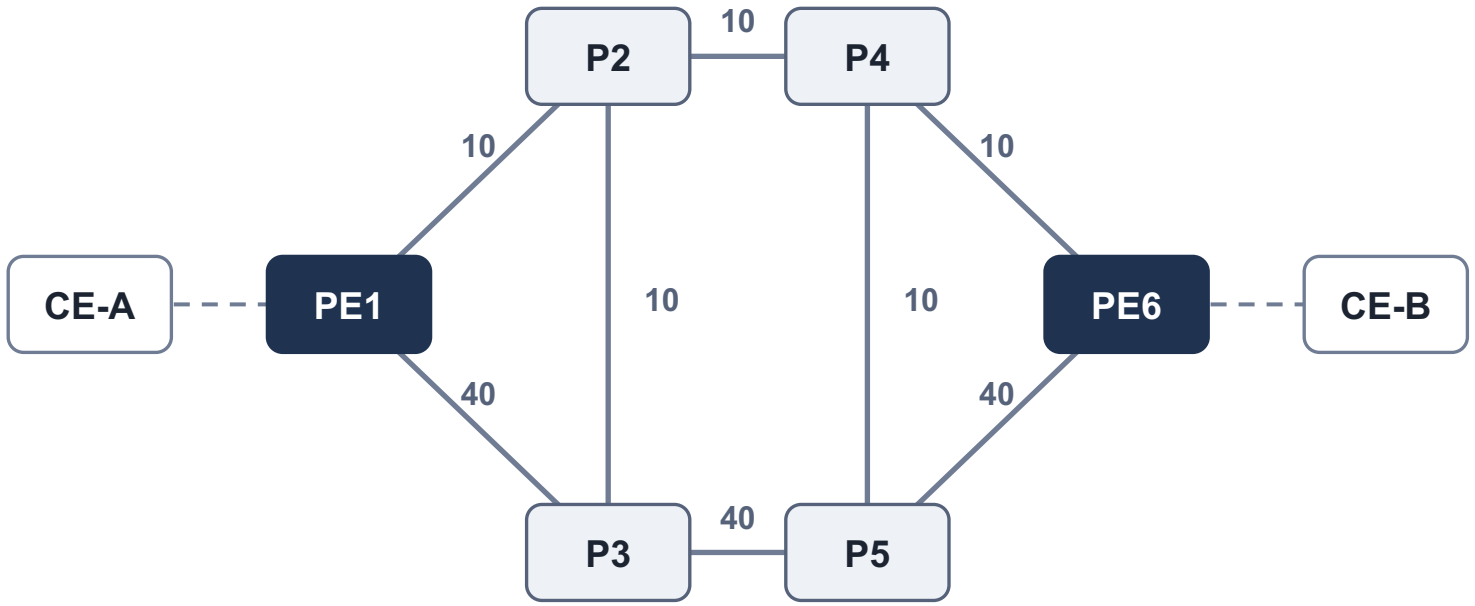
- 17 Method and OAM
- 18 Four faults
- 19 Security and scale

6 Ep 20–24

Migration and capstone

- 20 LDP to SRv6
- 21 SR-MPLS to SRv6
- 22 OSPF to IS-IS
- 23 What changes on hardware
- 24 Graduation lab

One topology, used throughout



Upper path PE1–P2–P4–PE6: cost 30, the only shortest path

Lower links 40 each; cross links P2–P3 and P4–P5 are 10

RFC 9602: IANA allocated 5f00::/16 for SRv6 SIDs. This course uses fcbb:bb00::/32 to match Cisco documents and community labs. A different block only replaces the prefix; the principle stays.

Item	Plan
Loopback0	`2001:db8:100::n/128`
MAIN locator	`fcbb:bb00:n::/48`
FA128 locator	`fcbb:bb01:n::/48`
Core links	`2001:db8:200:ab::/127`
IS-IS	Level 2, point-to-point
BLUE	RT 65000:100
RED	RT 65000:200
Software	XRd vRouter · IOS XR 26.2.2

n is the node number 1–6. BLUE and RED deliberately use the same address on PE6, 10.99.0.6, to test isolation. GREEN is a third customer.



Part 1: The packet

Where am I going, and what do I do on arrival? Both answers are written in 128 bits.

M1 · The state lives at the edge, not in the core

Router	LDP + MPLS VPN	RSVP-TE (design only, not built)	SRv6
Edge PE1	VPN routes and labels; one LDP label for each remote PE	Headend: the path and the signaling	Customer routes, service SIDs, the encapsulation
Core P2	One LDP binding for each prefix; one session for each neighbor	State for each tunnel through it, refreshed	IPv6 routes to locators and its own SIDs. Nothing about the customer

14 labels PE1 received from each of its two LDP neighbors

12 local SIDs on P2, for 3 customers or 3000

5 service SIDs on PE1: BLUE 2, RED 2, GREEN 1

Only one kind of state is gone: per-path and per-customer state on the middle routers. IGP, local SID and protection state stay.

M1 · One 100-byte ping, three encapsulations

LDP

- Two labels: outer 24009 is P2 label for PE6; inner 24000 is the VPN label from PE6
- 8 bytes

SR-MPLS

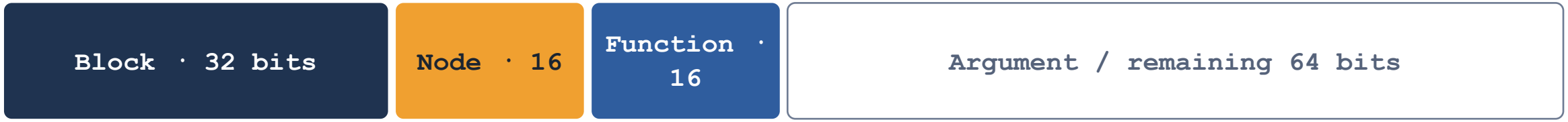
- Outer label 16006 is the prefix SID of PE6
- Same value on each hop: each router calculates it from the IGP

SRv6

- No labels. An outer IPv6 header has the service SID of PE6 as its DA
- 40 bytes; the frame is 32 bytes larger than with LDP

- The 40-byte versus 8-byte difference is always there. Ep 7 shows its effect on large packets.
- One outer header does two tasks: it sends the packet to PE6, and it tells PE6 the customer.

M1 · SID anatomy: f3216



The block is the common prefix of the SR domain. RFC 9602: IANA allocated 5f00::/16 for SRv6 SIDs. This course uses fcbb:bb00::/32 to match Cisco documents and community labs. A different block only replaces the prefix; the principle stays.

Locator = Block + Node

A routable /48 that IS-IS advertises. It answers "where am I going". The PE6 locator has metric 31: path cost 30 plus 1.

```
fcbb:bb00:6::/48
```

Function

An index into the local SID table. It answers "what do I do here". Local functions start at 0xe000. uA, uDT and uDX2 use them.

```
e000, e001: first two uA on each router
```

Argument

The rest of the program. A uN advertises function 0 and argument 80. The 80 bits hold the next uSIDs.

```
IS-IS: uN 32/16/0/80 · uA 32/16/16/64
```

M1 · uN and uA in the SID table and CEF

	SID (P3)	Owner	CEF	Next hop
uN	`fcbb:bb00:3::`	SID manager	/48 · Endpoint uN, shift	None fixed: shift, then look up again
uA	`fcbb:bb00:3:e005::`	IS-IS	/64 · block + node + function	Fixed: Gi0/0/0/1 to P5

Protected and unprotected

- One interface has two uA SIDs: one with the P mark (protected), one without
- The router can repair a protected adjacency. Use the unprotected one when you want only that link

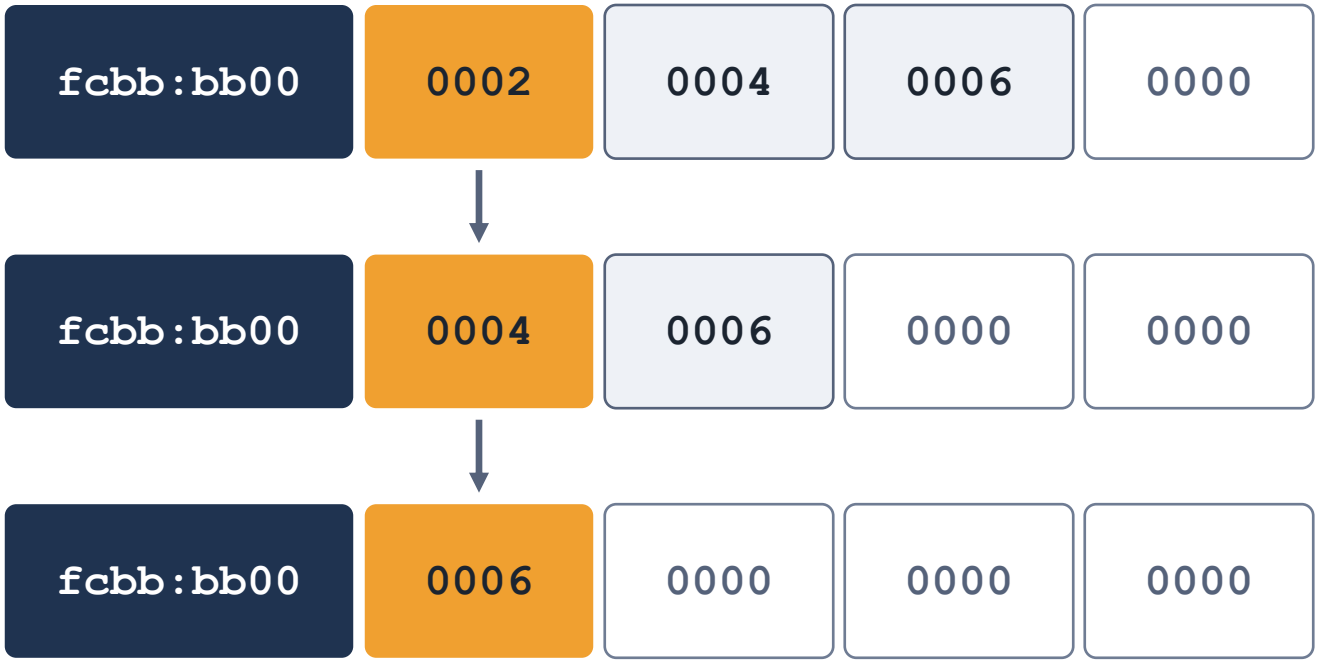
The count adds up

- P3 has 3 links × 2 algorithms × 2 (protected or not) = 12 uA SIDs
- Compare show segment-routing srv6 sid with show isis database verbose

Transit needs only a route

- P2 and P4 forward a SID like a normal IPv6 address
- On PE1: PE6 loopback metric 30, locator metric 31

M2 · uSID shift: three observations



Predict: ping fcbb:bb00:2:4:6:: from PE1. Which DA appears on each of the three links?
Lab: the DA and hop limit agree with the table, and no other link carries the packet. This ping gets no echo reply (see the capture rules).

M2 · Waypoints, container capacity, capture rules

Node SIDs are waypoints; the IGP picks the path

- The list P3, P5, PE6 takes PE1–P2–P3–P2–P4–P5–P4–PE6
- Hop limit 60 to 54; the three lower links carry nothing
- To force a link, use a uA

Container capacity

- Six 16-bit positions follow the 32-bit block
- A service SID uses one position
- One container holds one block: bb00 and bb01 cannot mix (see M3)

Read four fields in a capture

- DA: the active and the remaining instructions
- Hop limit: how many routers handled the packet
- Next header: 43 means a routing header follows
- Segments Left: present only with an SRH

A shifted ping gets no echo reply: the ICMPv6 checksum used the original DA (RFC 9800 6.5 says to use the final DA). Customer traffic is not affected. In tests, read captures, not ping replies.

M2 · Behavior reference

IOS XR	RFC	Action	Allocated by
`uN`	End + NEXT-CSID (PSP/USD)	Shift and forward to the next instruction	SID manager (from the locator)
`uA`	End.X + NEXT-CSID	Shift and send out one specific adjacency	IS-IS
`uDT4 / uDT6 / uDT46`	End.DT4 / DT6 / DT46	Decapsulate, look up in the VRF table	BGP
`uDX2`	End.DX2	Decapsulate, send the L2 frame to the AC	EVPN (VPWS)
`uDT2U / uDT2M`	End.DT2U / DT2M	Decapsulate, MAC lookup or flood (the platform rejects bridge domains)	EVPN (E-LAN)
`uB6`	Insert form (RFC 8986 defines only Encaps)	Steer traffic into an SR Policy (Ep 15)	SR-TE

- PSP removes the SRH and USD removes the outer header; both are flavors of uN. Decapsulation in uDT is the service behavior itself.
- Never guess a behavior from the configuration name. Read show segment-routing srv6 sid. A behavior in the list does not prove the platform accepts the feature.

M2 · The endpoint procedure and its flavors

1 **Is another micro-SID in the DA?**

Shift 16 bits, decrease the hop limit, route on the new DA.

2 **No, but does the SRH have Segments Left above 0?**

Copy the next SID from the SRH into the DA.

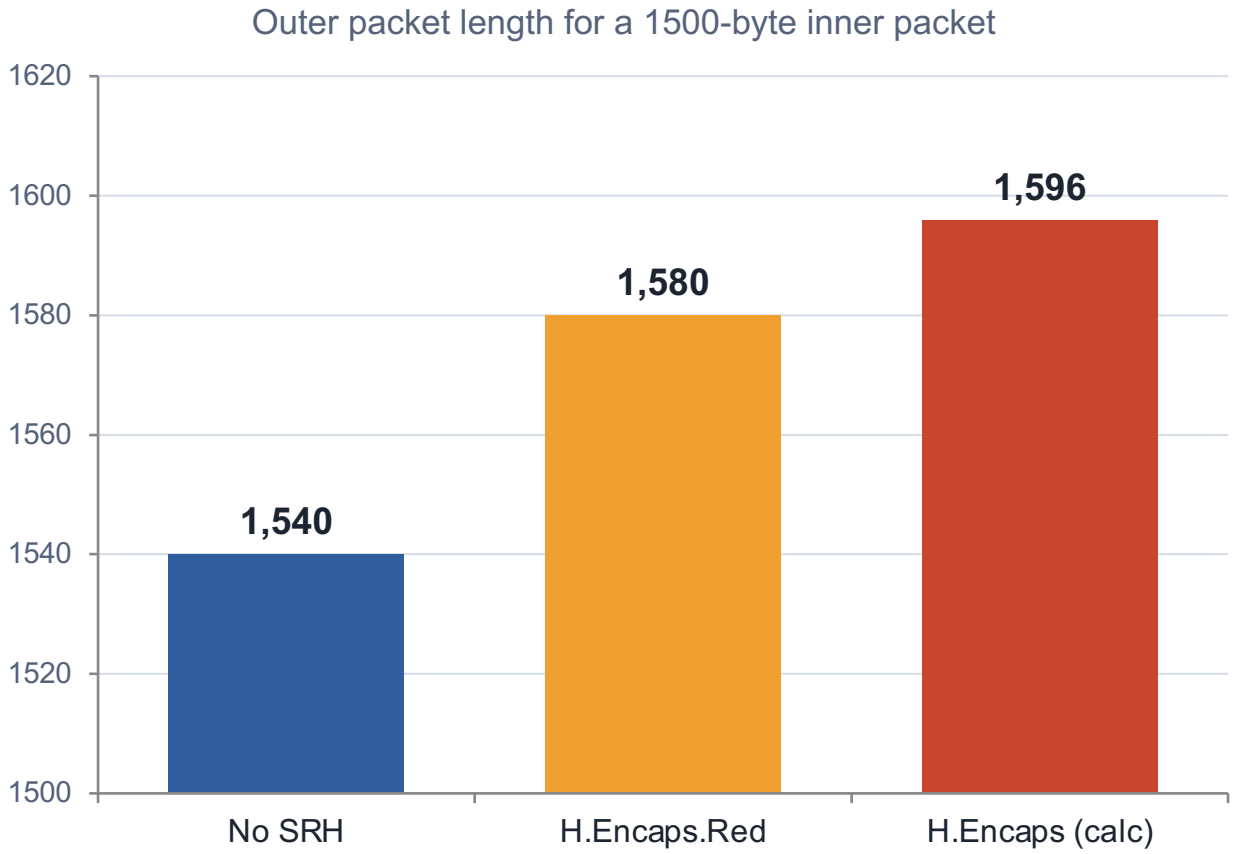
3 **Neither**

This is the last instruction. Examine the payload.

Flavor	What it removes	Evidence
`PSP`	The penultimate endpoint removes the SRH	Lab (Ep 6): after P5 the frame shrinks from 182 to 158 bytes. Earlier Ep 4 capture: 178 to 154
`USP`	The last endpoint removes the SRH	RFC 8986
`USD`	The last endpoint removes the whole outer IPv6 header	**Not verified: none of our services made such a packet**

A flavor acts only when a header exists. The decapsulation in uDT4 is its own function, not USD. With Segments Left at 0 the DA can still hold instructions.

M3 · Encapsulation and MTU: what an SRH costs



The last two assume three 128-bit SIDs: the reduced SRH has 2 entries (40 bytes), the plain one has 3 (56 bytes).

40 bytes: when the uSIDs fit one container there is no SRH, and the only overhead is the outer IPv6 header.

```
40 + 8 + 16 × entries
```

- GREEN frames are 154 bytes with no SRH. Colour 300 frames are 182 bytes on PE1–P3: 64 bytes more.
- P5 is the penultimate endpoint. PSP removes the SRH, so later links add only 40 bytes.
- This headend has only the reduced forms. No plain H.Encaps exists, so 1596 is a calculation, not a capture.

M3 · The SRH on the wire (colour 300)



DA fcbb:bb01:3:5:: · next header 43

Type 4 · next header 41 · length 2 · one entry: PE6 service SID

unchanged

Where	Segments Left	What happens
PE1 → P3	1	DA is the path container (P3, P5)
After P3	1	uSID shift only; the SRH does not change
P5	0	No micro-SID left: SL decreases, the service SID is copied into the DA, PSP removes the SRH

SRH = 8 + 16 × 1 = 24

Why an SRH

- Flex-Algo SIDs: fcbb:bb01; normal: fcbb:bb00
- One container holds one block

Two containers, one entry

- Reduced form: first SID only in the DA
- PE1 encapsulated, then inserted the path

M3 · The MTU budget: 1460, 1436, 1446

1460

one container, no SRH

`1500 − 40`

1436

colour 300, one SRH entry

`1500 − 64`

1446

E-Line (Ep 14)

`1500 − 40 − 14`

Know what your tool means by size

- IOS XR ping: size is the whole IP packet
- Linux ping: size is data only; add 28 for IPv4, 48 for IPv6
- Always confirm with one capture

Core MTU

- The lab uses 1500 so the limits are easy to reach. Real cores use a much larger MTU
- Core 9000, one SRH entry: 9000 − 40 − 24 = 8936
- E-Line: 9000 − 40 − 14 = 8946

M3 · Three MTU failures (Ep 7)

Case	Limit	What happened	Customer
IPv4 with DF at ingress PE1 (RED)	1460	1461 dropped at PE1. No ICMP on any link	**Black hole, no recovery**
IPv6 at ingress, colour 300 (BLUE)	1436	1437 gets Packet Too Big, MTU 1436, from the PE1 loopback. The PTB crosses the core to PE6 and back	**Host fragments, recovers**
IPv4 in the middle: P4 link set to 1300 (RED)	1260	P4 drops 1261. It has no VRF, so it sends ICMPv4 fragmentation needed (MTU 1236, from 10.0.0.4) on in the original outer header to PE1, which returns it through PE6 to CE-B	**CE-B fragments, recovers**

- The worst case is IPv4 at the ingress, not the case most people expect.
- Layer 2 (Ep 14): PE1 fragments the outer IPv6 packet, PE6 drops the fragments, and nobody sends a message.
- Size the core MTU for the largest overhead. Do not depend on ICMP. Record ICMP type, reported MTU, source and path for each case.

M3 · Reported MTU: minus 64 with a repair path

Route state of the destination	ICMPv6 Packet Too Big reports	Forwarding uses	With 1400 set
Installed TI-LFA repair path	**Egress MTU − 64**	The real MTU	1336
No backup	The exact MTU	The real MTU	1400
Plain LFA backup	The exact MTU	The real MTU	1400

Evidence

- XRd vRouter 26.2.2 and XRv9000 26.1.1 give the same result
- With TI-LFA removed from P2, the report changed from 1336 to 1400
- With 1300 set, it reports 1236 (the ICMPv4 value in Ep 7 too)

Inference, not proven

- 64 = 40 + 24: possibly space kept for a repair header
- Cisco does not document it
- Do not use minus 64 as an SRH design rule

Effect

- Hosts use 64 bytes less than possible; nothing breaks
- Record the repair type each time you compare PTB values
- Forwarding always uses the real MTU

M3 · Four conditions for one service

1 Ingress

Locator reachable

PE1 has the PE6 /48 in its IPv6 RIB

```
show route ipv6 <locator>
```

2 Egress

Egress has the service SID

The SID is in the PE6 SID table

```
show segment-routing srv6 sid
```

3 After the egress

The behavior ran

The outer header is gone: 138 → 98 bytes, 40 less

```
capture
```

4 Egress

Lookup in the customer table works

The VRF has a route to the CE

```
show route vrf <VRF> <prefix>
```

- The loopback ping checks none of the four. BGP showing best checks none of them either.
- Healthy: the PE6 locator has metric 31, and BGP received label hex e0080, which is SID function e008.
- A third failure: PE6 moved to a new SID while PE1 kept the old one. For about 25 seconds PE1 got ICMPv6 destination unreachable.

M3 · Two lookups

PE1

Lookup in VRF BLUE

Encapsulate: DA = service SID of PE6

→

P2 · P4

IPv6 longest-match

Hits fcbb:bb00:6::/48, plain transit

→

PE6 · first lookup

Local SID table

Hits uDT4, context = BLUE

→

PE6 · second lookup

VRF BLUE route

After decapsulation, forward on the inner DA to CE-B

Fault A: the remote locator is gone

- PE6 did not attach the locator to IS-IS
- Loopback ping passes; BGP still shows best
- BGP: SID locator inaccessible; CEF: drop
- First divergence: IPv6 RIB on PE1; no packet on the wire

Fault B: the egress service SID is gone

- BGP on PE6 allocated no SRv6 SID for the VRF
- PE1 gets decimal MPLS label 24004, not a SID
- The core has no MPLS: CEF unresolved, drop
- First divergence: local SID table on PE6



Part 2: Reachability

A locator that is up locally is not enough. Every node has to learn it and install it in CEF.

M4 · Bring-up: four acceptance levels

4 **End-to-end packet**

`capture: hop limit 60/59/58`

3 **Remotely reachable**

`show cef ipv6 <locator>`

2 **Local SID installed**

`show segment-routing srv6 sid`

1 **Config accepted**

`commit`

```
! PE1 baseline
interface Loopback0
 ipv6 address 2001:db8:100::1/128
segment-routing
 srv6
  encapsulation
   source-address 2001:db8:100::1
  locators
   locator MAIN
    micro-segment behavior unode psp-usd
    prefix fcbb:bb00:1::/48
router isis CORE
 is-type level-2-only
 net 49.0001.0000.0000.0001.00
 address-family ipv6 unicast
  metric-style wide
  segment-routing srv6
   locator MAIN
```

M4 · Read the manager before you design

What it enables

- Micro-segment mode, format f3216
- Local functions start at e000
- The encapsulation source is Loopback0

What it lacks

- Headends: H.Insert.Red and H.Encaps.Red only; no plain H.Encaps
- Hop limit cannot propagate; traffic class can, off by default

Limits

- Max-SL 3, Max-H-Insert 3, Max-H-Encap 3, Max-End-D 4
- One SID is a full container, so 3 limits containers
- 16 locators and 64000 SIDs are virtual-router limits

- This platform can prove syntax, control-plane state, SID allocation, packet format and forwarding path.
- It cannot prove line-card support, convergence time, hardware resources or scale.
- A behavior in the list is not an accepted feature: uDT2U and uDT2M are listed, but bridge domains are rejected.

M5 · IS-IS: LSP → RIB → CEF

Adjacency
Is the neighbor up?
`show isis adjacency detail`

→

LSP
What PE6 advertised
`show isis database verbose`

→

SPF route
Did SPF compute a path?
`show isis ipv6 route <locator>`

→

RIB
Which route PE1 installed
`show route ipv6 <locator>`

→

CEF
Which interface it leaves on
`show cef ipv6 <locator>`

Three faults, each starting at a different layer

Fault	Adjacency	LSP	SPF route	RIB	CEF
Interface lacks the IPv6 address family	✕ to P2	✓	metric 71	metric 71	exit changes
Locator not attached to IS-IS	✓	✕	✕	✕	✕
Local RIB install is filtered	✓	✓	✓	✕	✕

Advertised SID structure: uN is 32/16/0/80, uA is 32/16/16/64 (not 32/16/16/0). MSD is the per-packet SID depth; Max SIDs is the per-router limit. Locator metric = path cost + 1.



Part 3: Services

Who am I carrying? BGP binds a tenant route to a service SID.

M6 · L3VPN: the service resolution chain

1. **Tenant prefix in the VRF RIB on PE1**
2. **BGP route carries the RT and a service SID (transposed)**
3. **The SID falls inside a reachable remote locator**
4. **VRF CEF on PE1 shows the SRv6 encapsulation**
5. **The local SID table on PE6 holds the same SID**
6. **The VRF on PE6 has a route to CE-B**
7. **CE-B has a return route**

BGP Established proves none of the seven

Start from the tenant prefix and check each link in order. The first one that does not match is the first divergence: stop there and explain it before changing anything else.

M6 · The BGP update: transposition



NLRI

- Label 917568 = 0xE0040
- The label field has 20 bits: function e004 plus four zero bits
- The next hop is the PE6 loopback, not a SID

Prefix-SID attribute

- Type 40, SRv6 L3 Service TLV (type 5)
- The SID value is only fcbb:bb00:6::
- Behavior End.DT4 NEXT-CSID, structure 32/16/16/0

Rebuild

- Transposition length 16, offset 48
- The receiver takes the first 16 bits of the label and puts them after bit 48
- Result: fcbb:bb00:6:e004::

	SID	Label	Source
BLUE IPv4	`fcbb:bb00:6:e004::`	917568	capture
BLUE IPv6	`fcbb:bb00:6:e005::`	917584	calculated
RED IPv6	`fcbb:bb01:6:e004::`	917568	calculated: label as BLUE IPv4

M6 · What the wire shows

Hop limit

- PE1 sets the outer hop limit to 255. It does not copy the customer TTL
- P2 and P4 decrease only the outer value
- Traceroute from CE-A shows only PE1, PE6, CE-B (pipe model, RFC 2983)

Size

- The 98-byte customer ping is 138 bytes in the core
- It is 98 bytes again after PE6
- The customer packet does not change in the tunnel

QoS marking

- CE-A marks EF (DSCP 46)
- In the core the inner header shows 46 and the outer shows 0
- Traffic class is not propagated by default. If the core queues on the outer header, configure it and verify with a capture

Each link fails in its own way. Wrong RT: the update arrives but the VRF imports nothing. Locator unreachable: BGP still shows best but CEF drops. PE1 looks right but CE-B gets nothing: check the SID on PE6.

M6 · Design isolation so it can be proven

BLUE

RT `65000:100`
RD `65000:101 / 65000:106`
Site prefix
`10.1.1.0/24 · 10.1.6.0/24`
Overlapping address (PE6)
`10.99.0.6/32`
Service SIDs on PE6
`uDT4 e004 · uDT6 e005`

RED

RT `65000:200`
RD `65000:201 / 65000:206`
Site prefix
`10.2.1.0/24 · 10.2.6.0/24`
Overlapping address (PE6)
`10.99.0.6/32`
Service SIDs on PE6
`uDT4 e006 · uDT6 e004 (bb01)`

RD keeps overlapping prefixes unique inside BGP; RT decides who imports; the service SID decides the egress table. All three must be correct.

1 **Positive test**

Same-tenant ping in both directions; IPv4 and IPv6 checked separately

2 **Negative test**

A BLUE source pinging a RED address (10.1.1.2 → 10.2.6.2) gets no reply, and VRF BLUE holds no such route. Check both

3 **Right tenant**

Both tenants hold 10.99.0.6 on PE6. Capture the outer DA: e004 for BLUE, e006 for RED

M6 · One VRF is not one SID

SID (PE6)	Behavior	Context
`fcbb:bb00:6:e004::`	uDT4	BLUE
`fcbb:bb00:6:e005::`	uDT6	BLUE
`fcbb:bb00:6:e006::`	uDT4	RED
`fcbb:bb01:6:e004::`	uDT6	RED (FA128)
`fcbb:bb00:6:e007::`	uDX2	300:300 (L2, see M7)
`fcbb:bb00:6:e008::`	uDT4	GREEN

Another trap: the function values differ between PE1 and PE6.

Per-VRF allocation

One SID for each VRF and address family. BLUE has a uDT4 and a uDT6.

Dynamic allocation

Functions follow allocation order and change after a lab rebuild: read them, never copy them.

A VRF alone gets no SID

The SID appears only after SRv6 allocation is enabled under the VRF in BGP.

M6 · The RD: a different value on each PE

VRF	RD on PE1	RD on PE6	Route target
BLUE	65000:101	65000:106	65000:100
RED	65000:201	65000:206	65000:200
GREEN	65000:301	65000:306	65000:300

```
PE1#show bgp vpnv4 unicast vrf BLUE 10.99.0.6/32 detail
BGP routing table entry for 10.99.0.6/32,
  Route Distinguisher: 65000:101
  Received Label 0xe0040
  Extended community: RT:65000:100
  Source AFI: VPNv4 Unicast, Source VRF: default,
    Source Route Distinguisher: 65000:106
```

The RD only makes routes unique

Thus BGP can carry the 10.99.0.6 of BLUE and of RED. The RD is not the isolation.

It need not match on every PE

A different RD for the same VRF on each PE is a usual design. The RD shows which PE sent the route.

The route target decides the import

The route target is the same on both PEs, so each VRF still imports the routes of the other PE.

M7 · EVPN: E-Line built, E-LAN rejected

E-Line (VPWS)

Point-to-point · built

SID	`uDX2`
BGP route	Type 1 (Ethernet A-D)
Check	xconnect up; outer Next Header is Ethernet (143)

E-LAN · known unicast

Bridge domain · theory only

SID	`uDT2U`
BGP route	Type 2 (MAC/IP)
Expect	Once the MAC is learned, unicast uses uDT2U

E-LAN · BUM

Bridge domain · theory only

SID	`uDT2M`
BGP route	Type 3 (IMET)
Expect	The ingress PE sends one copy to each other PE

Lab result

XRd vRouter 26.2.2 and XRv9000 26.1.1 both reject the bridge domain (VPLS bridge domains not supported on this platform), although the manager lists uDT2U and uDT2M. E-LAN is theory only. This is a limit of our CML lab: on a real network, support depends on your hardware.

M7 · E-Line: the route, the SID and the MTU

Type 1 route

- ESI is 0 (the CE has one PE); Ethernet tag 300 is the service number
- Label 917680
- The L2 attributes carry MTU 0: no MTU check
- RT 65000:300 comes from the AS and the instance

Label to SID

- Prefix-SID TLV type 6 (L2 Service)
- Behavior End.DX2 NEXT-CSID, code 65
- 917680 = 0xE00B0 → function e00b → fcbb:bb00:1:e00b::
- On this platform EVPN shows three bytes (e00700): read the bits

The MTU trap

- Core 1500 − 40 − 14 (customer Ethernet header) = 1446; L3VPN allows 1460
- 1447: PE1 fragments the outer packet (1510 + 75), PE6 drops the fragments, the CE gets no message
- The core MTU must carry the customer MTU + 54

CE-A learns the MAC of CE-B and the TTL stays 64: the customer sees no router between the CEs. A 98-byte ping is 152 bytes in the core.

M7 · Why e0070 and e00700 are the same value

The label field in the BGP update has 24 bits. The label inside it has 20 bits.

Label shown by Wireshark: 917616 = 0xE0070 (20 bits)

E	0	0	7	0

XR on PE1 shows the whole field: 0xe00700 (24 bits)

e	0	0	7	0	0

First 16 bits = function e007

Last 4 bits are zero

```
Prefix-SID: transposition offset 48, length 16
SID = fcbb:bb00:6 : e007 :: = fcbb:bb00:6:e007::
PE1:  917680 = 0xE00B0  ->  function e00b
```

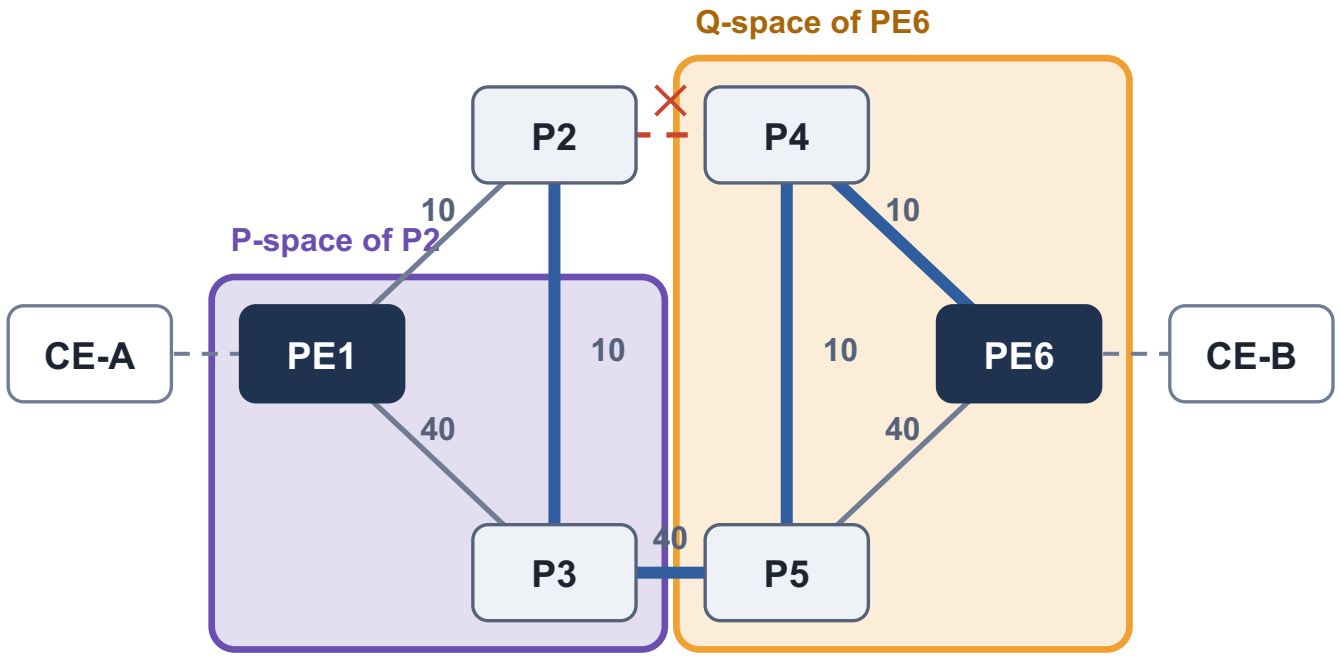
Both numbers are the same value with a different width. The router adds four zero bits at the end. Read the first 16 bits; do not count the digits.



Part 4: Resilience and intent

What if the path breaks, and who chose it in the first place?

M8 · TI-LFA: how a repair list is computed



P2–P4 fails and P2 is the PLR. Repair path: P2–P3–P5–P4–PE6

1 **P-space**

Nodes P2 can reach without the failed link: PE1 and P3.

2 **Q-space**

Nodes that reach PE6 without the failed link: P4 and P5. P3 is not in it, because the shortest path from P3 to P5 runs back over P2–P4.

3 **Bridge the gap**

No node is in both spaces. Between P3 and P5 an adjacency SID (e005) forces that one link.

Repair list (router output)

```
fcbb:bb00:3:e005::
```

The uN of P3 and the uA P3→P5 share one container. Metric 71. It matches the hand calculation.

M8 · Five failures: hand calculation and router repair list

Failure	PLR	Repair path (from the PLR)	Router repair list (Ep 9)	Against the hand calculation
Link PE1–P2	PE1	PE1–P3–P2–P4–PE6	Plain LFA via P3, no SIDs	Same: plain LFA is enough
Link P2–P4	P2	P2–P3–P5–P4–PE6	`fcbb:bb00:3:e005::  (71)`	Same
Link P4–PE6	P4	P4–P5–PE6	`fcbb:bb00:5:e001::  (51)`	Same: the last hop needs a uA too
Node P2	PE1	PE1–P3–P5–P4–PE6	`fcbb:bb00:3:e005::  (101)`	Router is shorter: one container, we had three segments
Node P4	P2	P2–P3–P5–PE6	`fcbb:bb00:3:e005:e001::  (91)`	Same; the second uA has no node uSID before it

The repair is H.Insert.Red

The router inserts the repair into the current header. It adds no second outer header.

Node protection is a request

This release needs a node-protecting tiebreaker. Traffic to PE6 cannot be protected against PE6 failing: path not found.

Check the coverage

show isis ipv6 fast-reroute summary must show 100 percent. With two one-way links, P4 once showed 87 percent.

M8 · TI-LFA on the wire: three phases

Phase	Packets	What the capture shows
1	First 44	P2 inserts the node-protecting repair and sends to P3. The routing header holds the GREEN service SID, Segments Left 1.
2	Next 15	A shorter list: P3 and the adjacency to P5, still with a routing header. We did not verify the cause.
3	After	PE1 own explicit path: fcbb:bb00:3:e005:6:e008::. About 4 seconds later it uses the new shortest path (M8 microloop).

Run 1: the repair appears

- P3 runs its instruction and shifts; e001 is left (P5 to PE6)
- P5 removes the routing header; PE6 gets only the service SID
- All 4000 pings answered; none went from P3 back to P2

Run 2: the same failure

- No repaired packet appears
- 8 pings lost; the wire is empty for about 1 second
- We did not verify why the two runs differ

A software forwarder proves that the repair exists and shows its format. It cannot tell you how fast protection is.

M8 · Microloop avoidance and BFD

Time	Event (P2–P4 fails)
34.253	BFD on P2–P4 goes down; IS-IS removes the adjacency in the same millisecond
34.354	P2 installs its new route (metric 71) with an exp path: P3, then the uA to P5
34.719	PE1 installs its new route (metric 81), also with an exp path; the normal next hop is still P2
40.219	The exp path is gone (5.5 s after install); the route is protected again
Restore	Adjacency up 6.5 s later; PE1 back to metric 31 another 6 s later; no ping lost

Manual example of a microloop: P2 has converged and sends to P3; P3 has not and sends back to P2. This is not a capture.

RIB update delay: 5000 ms

The Cisco default; no RFC sets it. It is how long the router keeps the exp path, not the outage.

BFD: 100 ms × 3

Detection takes 300 ms. Without BFD, IS-IS waits for its hello holdtime: many seconds.

Evidence and gap

With avoidance on, no customer packet went from P3 back to P2. We did not test with avoidance off.

M8 · What a virtual lab cannot time

This lab cannot time

- The forwarder is software on shared vCPUs
- The same failure: 0 lost, then 8 lost
- 5000 ms is the exp-path hold time, not the outage
- The repair of about 0.1 s is one test, not a protection time

How to measure on hardware

- Send a constant-rate stream with sequence numbers; do not use ping
- Know the rate: at 10000 packets per second, one lost packet is 0.1 ms
- Fail the same object many times; report the worst result and the range
- Test the restoration too: traffic can be lost when a link returns

Three kinds of evidence, in time order

Logs

BFD and adjacency events on the two routers next to the failure

Route state

Install times and exp-path lines on the repairing router and a remote PE

The wire

Captures on the old path, the repair path and the new path

M9 · SR Policy: five parts and the Binding SID

1

Headend

The policy lives on a headend (PE1). A color and an endpoint identify it.

2

Candidate path

One or more, each with a preference. The valid path with the highest preference is active.

3

Segment list

The instructions the headend puts in the packet.

4

Binding SID

A local SID that identifies the whole policy.

5

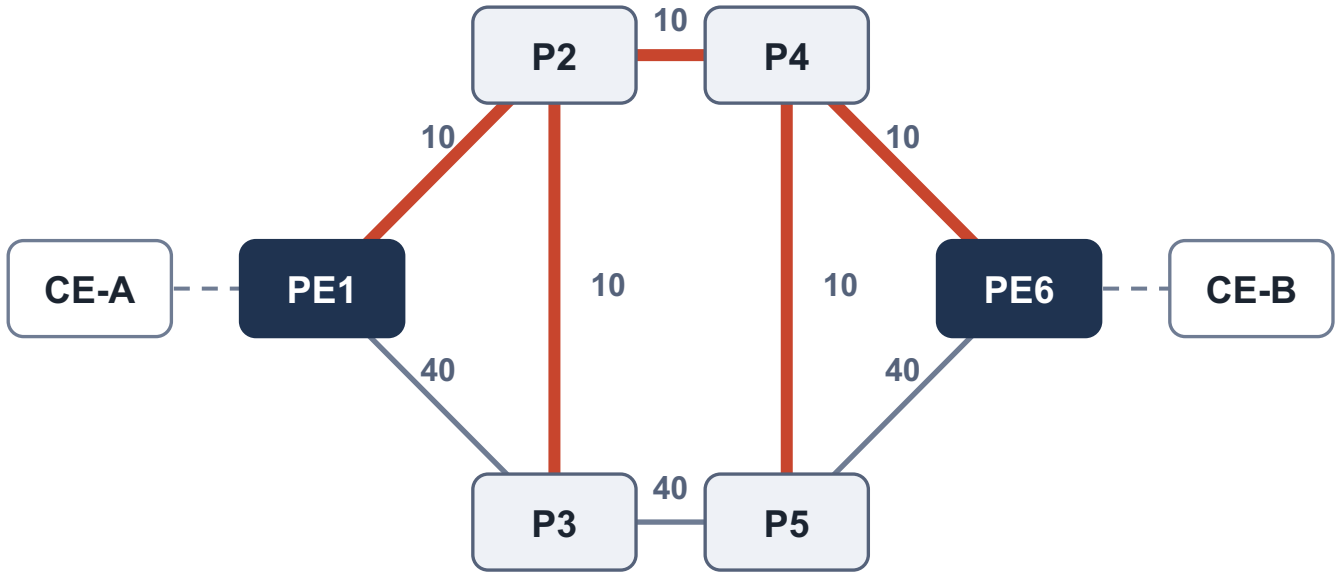
Color

The egress PE adds it to a route. The ingress uses color and next hop to find the policy.

Item	Lab value (Ep 15)
Policy	color 100, endpoint is the PE6 loopback
Candidate path	preference 100, list: P3, P5, PE6 (all uN)
Binding SID	`fcbb:bb00:1:e008::`
Final node SID	In the list, but PE1 does not put it in the packet
PE6 export	Color 100 on BLUE IPv4 only: one update carries three prefixes
CEF (PE1)	H.Encaps.Red with the service SID, then H.Insert.Red {fcbb:bb00:3:5::}

Color does not change what PE6 does with the packet. It changes the path PE1 uses to reach PE6. The Binding SID is dynamic: never design around this value.

M9 · uN alone never takes the lower links



Segment list: P3, P5, PE6 (all uN) · 138 bytes

`PE1–P2–P3–P2–P4–P5–P4–PE6`

Hop limit 255 → 249: seven hops, two U-turns

Segment	IGP shortest path	Cost
PE1 → P3	via P2	20, below the direct 40
P3 → P5	via P2 and P4	30, below the direct 40
P5 → PE6	via P4	20, below the direct 40

uN names a node, nothing more

- Between nodes the packet still follows the IGP shortest path
- To pin a link, use the uA of that link
- uA values are dynamic: read them from the SID table, never hard-code them
- A policy that is up and valid does not mean traffic follows the line you drew

M9 · Strict list, one-way steering and fallback

Strict list (preference 200)

- uA P3→P5 (e005) and uA P5→PE6 (e003)
- Container fcbb:bb00:3:e005:5:e003:6:e004: six micro-SIDs, 138 bytes, no SRH
- Path PE1–P2–P3–P5–PE6, hop limit 255 → 252
- PE1 to P3 still goes through P2: we fixed only the links that need it
- The Binding SID does not change

Color steers one direction

- Only the export of PE6 adds the color
- Replies follow the IGP: PE6, P4, P2, PE1
- To steer both directions, add the color on the other side too

When the policy fails

- Shutdown: admin down, Binding SID released
- The BGP route keeps the color but no longer points to a policy
- CEF falls back to the locator and a plain encapsulation: PE1–P2–P4–PE6, pings pass
- Invalidation drop is off by default. To keep a service on its path, turn it on and test
- After no shutdown the Binding SID has the same value, but it is dynamic

M9 · Adjacency SIDs in a strict list: why, and the risk

```
segment-list VIA-P3-P5-UA
 srv6
  index 10 sid fcbb:bb00:3:e005::   ! uA P3 to P5
  index 20 sid fcbb:bb00:5:e003::   ! uA P5 to PE6
policy BLUE-C100
 candidate-paths
  preference 200
   explicit segment-list VIA-P3-P5-UA
```

Read both values on P3 and P5 immediately before the test.

Why an adjacency SID

- A node SID only says where the packet goes; the IGP decides how
- Only an adjacency SID selects one link

The risk

- On this platform, dynamic uA values can change after a reload or an outage of more than 3 minutes
- The old SID in the list can select another link, or nothing

What to do

- After such an event, read the SIDs again and compare them with the list
- Do not hard-code them: let a PCE compute the list (Ep 16), or use node SIDs and Flex-Algo

M9–M10 · Policy, ODN and Flex-Algo compared

Explicit SR Policy

You write the segment list

- Who computes: a person
- Trigger: the color and next hop of a BGP route match the color and endpoint of the policy
- Check: candidate path valid, traffic really steered, expected DA captured

ODN

A template triggered by color

- Who computes: a PCE. On this release the headend cannot compute an SRv6 path
- Trigger: a colored route arrives; the template holds an intent, not a path
- Check: policy up, PCEP session up, SIDs in CEF, packets on the wire

Flex-Algo

The IGP computes a second topology

- Who computes: every participating router
- Trigger: a FAD is elected and nodes explicitly participate
- Check: FAD winner, participation, the separate locator and its path

M9 · ODN needs a PCE: three tests

Test	Setup	Result	Where the traffic goes
A	No PCE; the headend computes	Policy down: Headend computation not supported for SRv6 policy	Shortest path PE1–P2–P4–PE6, with no alarm
B	P4 is the PCE (three lines); dynamic pcep, metric latency	Policy up: preference 100 delegated to the PCE and active; three protected uA; latency 3000	PE1–P3–P5–PE6 (low latency)
C	B plus constraints segments sid-algorithm 128	Policy down: SRv6 SIDs failed verification; the PCE returns an empty path	Shortest path, with no alarm

The document agrees with the lab

The Cisco 25.x guide lists as unsupported: head-end computed dynamic paths (A) and PCE delegation with a Flex-Algo constraint (C). XRv9000 26.1.1 gave the same result.

A trap when you clean up test A

Removing pcep or the PCE address is not enough: the delegated path stays. Delete the template and create it again.

The silent fallback

The customer pays for low latency and gets high latency, and no device says so. After a boot an ODN policy can need a minute or more to come up.

M9 · The ODN packet and the two methods that work

```
CEF:  H.Insert.Red {fcbb:bb00:e004:e002::}
!     PE1 own uA = outgoing interface to P3
PE1→P3  fcbb:bb00:e004:e002:6:e006::
P3→P5   fcbb:bb00:e002:6:e006::
P5→PE6  fcbb:bb00:6:e006::
```

- Two micro-SIDs cover three adjacencies: PE1 own adjacency is the outgoing interface
- e004 and e002 are in the e000 range: local micro-SIDs, read by each router as its own adjacency
- The PCE picks algorithm 0 adjacency SIDs by latency; it is not a Flex-Algo path

Method 1: a service SID from Algo 128

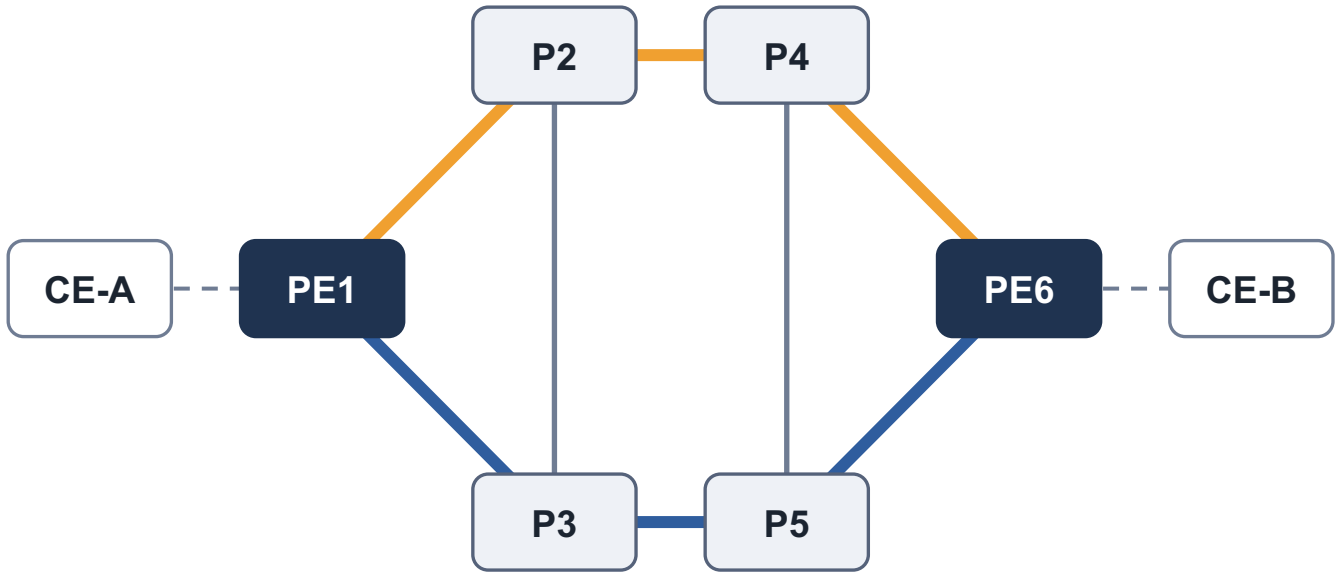
- RED IPv6 uses the FA128 uDT6 fcbb:bb01:6:e004::
- Metric 3001 via P3; no policy and no color

Method 2: ODN through a PCE

- Metric type latency; it finds the same three low-delay links
- The two methods cannot combine (test C)

A color 400 policy with only the Algo 128 node SID of PE6 came up, but the final node SID was left out of the packet: traffic left through P3, then followed Algo 0. Up did not mean the path was used.

M10 · Flex-Algo 128: one graph, two shortest paths



Algo 0: IGP cost 30 **Algo 128: delay 3000 µs**

Advertised static delay: 1000 µs on the three lower links, 5000 µs on the others

	Algo 0	Algo 128
Metric	IGP cost	Delay
Metric of the PE6 locator on PE1	31	3001
Locator of PE6	`fcbb:bb00:6::/48`	`fcbb:bb01:6::/48`
Next hop on PE1	P2	P3

- Receiving a FAD is not participating: every P needs flex-algo 128 configured explicitly
- uSIDs from different blocks cannot share one container
- Algo 128 does not mean low latency: the FAD and the real metrics decide the outcome

M10 · What does 128 mean?

A user range

RFC 9350: 128 to 255 are for the operator, and you choose the meaning. 0 to 127 are not for this use.

Not continuous

You can use 128, 140 and 200 only. You do not need the numbers between them.

One definition wins

Many routers can advertise a definition for one number. The highest priority wins; a tie goes to the highest System-ID.

The number means nothing

The selected definition gives the meaning: metric type and constraints.

Our lab

We use only 128. XRd shows Max Locators: 16, so many algorithms can exist. Your platform can differ.

```
P3#show isis flex-algo 128
  Definition Priority: 200
  Definition Source: PE1.00
  Definition Metric Type: Delay
  Local Priority: 128
```

M10 · Three conditions, and how to put a customer on it

1 Definition

- PE1 (priority 200) and PE6 (100) advertise what 128 means: metric type delay
- The highest priority wins; a tie goes to the highest system ID
- P3 advertises no definition and still sees the selected one (PE1)

2 Participation

- Every router needs flex-algo 128 configured
- Receiving a definition is not participating
- P2–P5 only participate; they advertise no definition

3 Locator

- FA128: fcbb:bb01:n::/48, a separate block (our choice, not an RFC rule)
- Algo 128 has its own node SID and its own uA on each link
- TI-LFA stays in the same topology: the repair SID is fcbb:bb01:4::

Service: no policy and no color

- PE6 allocates the RED IPv6 SID from FA128: uDT6 fcbb:bb01:6:e004::
- PE1 to PE6 goes PE1–P3–P5–PE6
- The return SID is still in MAIN and takes the upper path: the directions differ. For symmetry, allocate on both PEs

Delay test: one number moves the customer

- Raise the advertised delay of P3–P5 from 1000 to 20000 µs on both ends; costs stay
- Algo 128 moves to P2 and P4 at metric 15001; Algo 0 does not change
- RED IPv6 moves to the upper path and GREEN stays; after the restore it moves back



Part 5: Operate

Troubleshooting, boundaries and migration: the same questions applied to real operations.

M11 · The five-layer troubleshooting ladder

1 **Local: locator up, SID allocated**
`show segment-routing srv6 locator / manager`

2 **IGP: locator advertised and installed as a route**
`show isis segment-routing srv6 locators`

3 **Forwarding: CEF has an exit and a backup**
`show cef ipv6 <locator> detail`

4 **Service: BGP path, SID, color, policy**
`show bgp vpnv4 unicast vrf … detail`

5 **End to end: customer traffic and the wire**
`ping both ways, several sizes, packet capture`

Write the hypothesis first

and the evidence that would refute it

Collect evidence from layer 1 up

stop at the first place where expected and actual differ; never skip a layer because the one above looks right

Change one thing at a time

every change is tied to one observation

Regress after the fix

retest the symptom, then a service that was not affected

M11 · How fast the method is: fault A, and a healthy baseline

Layer	Checked	Result
1 Local, PE6	locator MAIN	Up: pass
2 IGP, PE1	`show route ipv6 fcbb:bb00:6::/48`	**Network not in table: FAIL**
2 IGP, PE6	IS-IS configuration	The locator line is missing under IS-IS
Fix	Restore one line on PE6	All layers healthy again

Symptom: GREEN fails, but PE1 gets a reply from the loopback of PE6. Two commands found the fault. Without the method, the troubleshooting in Ep 3 starts at BGP, and BGP showed best.

Keep a healthy baseline

- Record the five layer commands per PE and per service
- Save the running configs: this lab keeps one snapshot per module
- Mark dynamic values as dynamic: uA and Binding SIDs can change after a reboot or an outage over three minutes
- Compare with a diff; do not trust memory

M11 · SRv6 OAM: what ping and traceroute prove

Test (from PE1)	Result	What it proves
ping the uN, uA and uDT4 of PE6	3/3 reply; the source is the PE6 loopback, not the SID	Only that PE6 is reachable and owns the SID; not that the uDT4 looked up BLUE
traceroute to the node SID	P2, P4, PE6 on the IGP path	An ordinary IPv6 traceroute
traceroute ipv6 … srv6	Rejected: % Invalid input	Cisco documents two SRv6 forms; neither is in the ipv6 help tree of XRd 26.2.2. Our claim is only about the ipv6 syntax
traceroute segment-routing srv6 policy name <name>	In the help tree of this release; not run	Traces a policy. A document does not always match your release: use ? and test the syntax
traceroute to fcbb:bb00:3:5:6::	P2, P3, P2, P4, P5, P4, then stars	The U-turns show; the last node drops the probe (checksum uses the original DA)

The document says ping to an SRv6 SID is not supported; the lab got replies. Documents are a start, not a guarantee: try it first.

When the tools are not enough, capture at five places

Ingress access link | **First core link** | **Middle links** | **Last core link** | **Egress access link**

M11 · Four faults, four first divergences

Symptom	First hypothesis	First divergence and evidence	Fix
L2 service: small frames pass, large fail (1000, 1300 pass; 1400 fails)	A core link has a lower IPv6 MTU	Large frames reach P2 but not the P2–P4 link. P2 sends a PTB to the PE1 loopback with MTU field 1336 (not 1400); the CE gets nothing	Remove ipv6 mtu on P2
CE-A cannot ping CE-B	The return path lacks a route	Requests reach CE-B and CE-B replies; the replies stop at PE6. BLUE on PE6 has no route to CE-A: PE1 exports RT 65000:110, PE6 imports 65000:100	Set the RT back to 65000:100
Only GREEN fails	The SID locator is unreachable	Layer 4: locator fcbb:bb00:66::/48 inaccessible, CEF unresolved and drop. The TEST locator on PE6 is Inactive in IS-IS	Move GREEN back to MAIN
Fine until P3–P5 fails, then RED IPv6 stops	Protection or participation was lost	Layer 2 before the failure: after flex-algo 128 left P2 and P4, the Algo 128 locator route lost Protected and its backup	Restore flex-algo 128

M11 · Faults 1 and 3: the configuration changes

```
! Fault 1, on P2, toward P4
interface GigabitEthernet0/0/0/1
 ipv6 mtu 1400
! 1400 - 40 (outer IPv6) - 14 (inner Ethernet) = 1346
! Repair: no ipv6 mtu
```

```
! Fault 3, on PE6. Step 1: new locator, RED uses it
segment-routing srv6 locators locator TEST
 micro-segment behavior unode psp-usd
 prefix fcbb:bb00:66::/48
router bgp 65000 vrf RED address-family ipv4 unicast
 segment-routing srv6 locator TEST
! missing: the locator is not added under IS-IS
! Step 2: RED back to MAIN, GREEN moves to TEST
! Repair: GREEN to MAIN, then: no locator TEST
```

Fault 1

The interface MTU stays 1514 and IPv6 can use only 1400. A customer packet over 1346 bytes cannot cross: 1300 passed, 1400 failed.

Fault 3

The locator was not added to IS-IS: show isis segment-routing srv6 locators shows Inactive. RED survived step 1 because its ODN policy ends with an adjacency SID of P5.

M11 · What the four faults taught

Before fault 1: the lab broke alone

After some idle hours, four links became one-way: 12 adjacencies, not 16. A shut and no shut on both ends fixed them. Run a health check before each recording.

Fault 1: PTB 64 short

The PTB MTU field was 1336, not 1400. With a TI-LFA repair path for the destination, the reported value is the egress MTU minus 64; forwarding still uses the real MTU. For an L2 service the PTB is useless, so the customer sees nothing.

Fault 2: look at both directions

A ping cannot show which direction is broken. A capture can. Examine the PE where the reply stops, also when the customer is on the other side.

Fault 3: a policy can hide a fault

A SID change is an outage too: PE1 used the old SID, already removed by PE6, for about 40 seconds. RED has a color; the last uA of its policy delivered the packet to PE6, so RED survived.

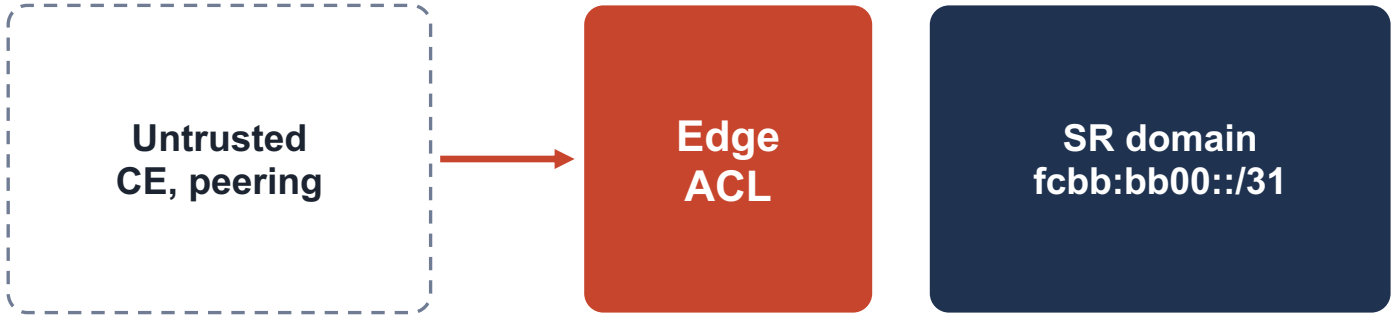
Fault 4: it waits for an event

Nothing failed when we changed it and every test passed. Only layer 2, compared with the reference, showed it. When P3–P5 failed, the customer who paid for the special path lost service.

After a failed commit

Read show configuration failed, then abort. We saw later commits fail in one E-LAN session and did not repeat it: treat it as good practice, not as a proved rule.

M12 · The boundary: who may talk to a SID




```
ipv6 access-list SRV6-BOUNDARY-IN
 10 deny ipv6 any fcbb:bb00::/31 log
 20 deny ipv6 any any routing log
 30 permit ipv6 any any
!
interface GigabitEthernet0/0/0/4
 ipv6 access-group SRV6-BOUNDARY-IN ingress
```

Negative test: pings from the outside host to four SIDs (uN, P4 uN, uA, BLUE uDT4) must get no reply, and the PE6 loopback must still answer. Use -I to set the source address.

A SID is an address, not a label

With no filter, an outside host got replies from the BLUE uDT4 SID too. MPLS blocks this by default; SRv6 does not.

Filter routing headers too

Line 20. This release rejected routing-type 4 and accepted the routing keyword.

A /31 covers both blocks exactly

The whole base block is a /24 and also covers blocks you add later. Choose one and record why.

Verify with a capture, not a counter

The deny lines stayed at zero while packets were dropped. The requests never appeared on the PE1 core link.

M12 · Scale: three different limits

Limit	This lab	Note
Local SIDs per block	7680 (e000 to fdff)	One or two per VRF; per-CE allocation uses many more. Adjacency and Binding SIDs share the range
SID depth	Max-H-Encap 3, Max-H-Insert 3, Max-End-D 4	With micro-SIDs one SID holds up to six instructions; depth is rarely the limit
Number of locators	16 (shown by the XRd manager)	One per algorithm and one per migration step; hardware numbers differ (M12 hardware)

Base block and the ACL

The manager shows the base block fcbb:bb00::/24. The ACL uses a /31 or a /24: see the previous slide.

Arithmetic for 500 VRFs

PE6 used four L3 service SIDs (two BLUE, one RED, one GREEN). 500 dual-stack VRFs need up to 1000, which fits in 7680.

MSD

Max segments left and Max End D limit the routing headers a node can process and remove. RFC 9352 and RFC 8491 define them.

M12 · SID allocation modes: an example with 500 VRFs

Mode	One SID for each	SIDs for 500 VRFs	Tested
per-vrf	VRF and address family	500 × 2 = 1000	Yes
per-vrf-46	VRF; one SID for IPv4 and IPv6 (uDT46)	500 × 1 = 500	No
per-CE	CE and address family	500 × 4 CEs × 2 = 4000	No
Local range	One block	7680	

The numbers are an example, not a measurement. Per-CE allocation can use more than half of the local range, and the adjacency and Binding SIDs share it. The Cisco 8000 guide describes per-vrf-46; we did not test it.

M12 · Migration: the order matters more than the commands

1 Coexist → 2 Verify the new → **3 Shift by batch** → 4 Verify both → 5 Remove the old

LDP → SRv6 (Ep 20)

- The egress decides: each PE moves alone and each direction follows its egress
- LDP stays, so the rollback is one line; remove LDP last
- An SRv6 packet is 32 bytes larger than two MPLS labels: test the MTU first

SR-MPLS → SRv6 (Ep 21)

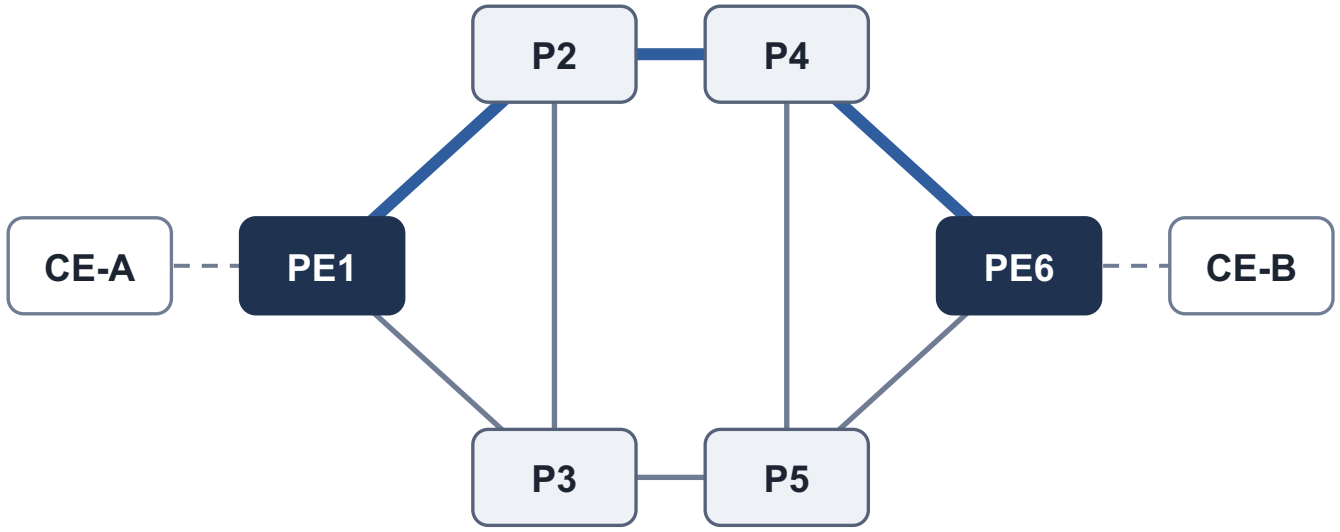
- The service change is the same as in Ep 20
- The same label on each hop needs the same SRGB
- The old transport still carried the IPv4 BGP session: +99 bytes in 60 s
- After the IPv4 VPN session is removed, the counter stops

OSPF → IS-IS (Ep 22)

- Run both IGPs, no redistribution; switch with distance
- OSPF (110) owned the loopback with no backup; IS-IS (115) owned the locator
- Remove OSPF last; the selected routes must not change

Write the rollback trigger before the cutover. A rollback restores the verified previous state and re-verifies it; it is not flipping one switch back.

M12 · The LDP starting point: topology and commands



GREEN path: PE1–P2–P4–PE6; LDP runs on every core link

PE1 and PE6 have a VPNv4 session between the IPv4 loopbacks 10.0.0.1 and 10.0.0.6. BLUE and RED stay on SRv6.

```
PE1#show mpls ldp neighbor brief
10.0.0.2:0  ...   10.0.0.3:0  ...

PE1#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32
detail
  Received Label 24000

PE1#show cef vrf GREEN 10.30.6.6/32 detail
  next hop 10.12.0.1/32 Gi0/0/0/0
    labels imposed {24009 24000}
```

24009 is the LDP label to PE6; 24000 is the VPN label.

M12 · LDP to SRv6: each direction follows its egress

Step	Action (GREEN)	PE1 → PE6	PE6 → PE1
Start	LDP and MPLS VPN: labels {24009 24000}, 122 bytes. BLUE on SRv6: 154 bytes	MPLS	MPLS
1	PE6 allocates an SRv6 SID for GREEN (fcbb:bb00:6:e008::)	**SRv6**	MPLS
2	The same on PE1	**SRv6**	**SRv6**
Rollback of step 1	One line on PE6 removes the allocation (PE1 had moved)	MPLS	**SRv6**
3	Remove the IPv4 BGP session, then LDP	**SRv6**	**SRv6**

The SID selects the encapsulation, not the session

- A GREEN route that arrived on the IPv4 session still used SRv6. Both paths had equal attributes; a late tie-break chose one

What the cleanup leaves behind

- IS-IS still advertises IPv4 and the IPv4 loopbacks are still router IDs. The ICMPv4 message in Ep 7 came from a P4 IPv4 address that exists only because of this setup: retest the MTU after cleanup

M12 · SR-MPLS to SRv6: who was using label 16006

Item	LDP	SR-MPLS	SRv6
Transport	Locally allocated labels	Advertised index plus SRGB	Locator routes
GREEN labels	{24009 24000}	{16006 24000}	fcbb:bb00:6:e008::
100-byte ping	122 bytes	122 bytes	**154 bytes**
Transport signaling	A separate protocol (LDP)	IGP	IGP

- Label 16006 = SRGB start 16000 + index 6; the micro-SID node ID of PE6 is also 6. One number, two data planes
- The label stays 16006 on each hop because the routers use the same SRGB; different SRGBs can give different values. LDP allocates independently on each router

After GREEN moved, 16006 still counted

- 60 seconds with no customer traffic: the counter grew by 99 bytes
- Capture: two labelled packets, both TCP port 179 from 10.0.0.1 to 10.0.0.6: a BGP keepalive of 59 bytes and a TCP ACK of 40 bytes
- Cause: the next hop 10.0.0.6 of the IPv4 BGP session resolved over SR-MPLS
- After removing the session the counter stopped; GREEN still passed

M12 · SR-MPLS to SRv6: four steps with commands

```
! 1  SR-MPLS (here P2, index 2)
router isis CORE address-family ipv4 unicast
 segment-routing mpls
router isis CORE interface Loopback0 address-family
ipv4 unicast
 prefix-sid index 2
! GREEN: labels imposed {16006 24000}

! 2  GREEN to SRv6, on PE6 and PE1
router bgp 65000 vrf GREEN address-family ipv4 unicast
 segment-routing srv6 locator MAIN alloc mode per-vrf
! CEF: SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e008::}
```

```
! 3  remove the IPv4 VPN session, then read the counter
router bgp 65000
 no neighbor 10.0.0.6
PE1#show mpls forwarding prefix 10.0.0.6/32  (00:28:51)
16006  16006  SR Pfx (idx 6)  Gi0/0/0/0  10.12.0.1
1251
PE1#show mpls forwarding prefix 10.0.0.6/32  (00:29:53)
16006  16006  SR Pfx (idx 6)  Gi0/0/0/0  10.12.0.1
1251

! 4  remove SR-MPLS, on every router
no router isis CORE address-family ipv4 unicast
no router isis CORE interface Loopback0 address-family
ipv4 unicast
no router isis CORE interface Gi0/0/0/0 address-family
ipv4 unicast
```

After step 3 the counter stops (1251 → 1251). Do step 4 last. After step 4, IS-IS has only the SRv6 lines; run a ping to confirm that GREEN works.

M12 · SR-MPLS and SRv6 side by side, and the order of work

SR-MPLS	SRv6
Prefix SID	uN: reach this node
Adjacency SID	uA: use this link
TI-LFA repair label stack	Repair SID list, added at the repair point
Binding label	uB6 Binding SID
VPN label	Service SID (routable: this is why a boundary is needed)

The concepts stay. The data plane and the service encoding change.

The order of work

- Migrate every service first, including all VRFs and L2 services
- Find the traffic that still uses labels: growing counters, IPv4 BGP sessions, SR-MPLS policies
- Remove the IPv4 VPN sessions and check that the counters stop
- Remove SR-MPLS and the prefix SIDs last

Step 4: remove SR-MPLS

- On every router, remove the IPv4 address family from IS-IS (instance and each interface)
- This also removes the prefix SIDs and segment routing for MPLS
- Do it after the counter stopped; then IS-IS has only the SRv6 lines

M12 · OSPF to IS-IS: both IGPs together

Route	Both IGPs, default distance	After step 1 (IS-IS distance 100)
PE6 loopback	OSPF (110), metric 30, one path, no backup	IS-IS, with TI-LFA protection
PE6 locator	IS-IS (115), metric 31, with a TI-LFA backup	IS-IS
What BGP sees	Next-hop metric 30 (OSPF), locator metric 31 (IS-IS)	Both from IS-IS

Step 0

Run both IGPs with the same costs (lower links 40, others 10). Check that the selected paths match before you change preference.

Step 1

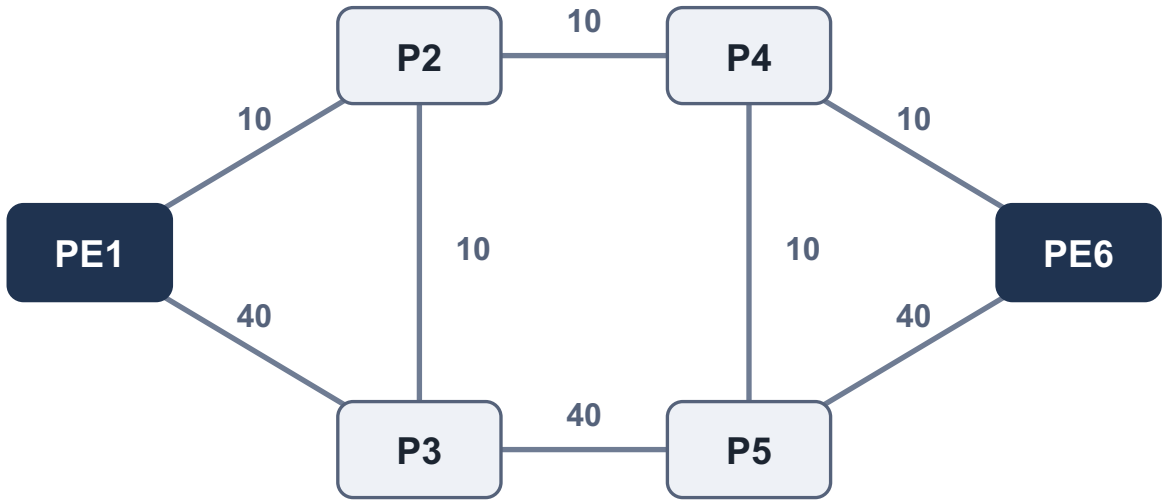
Set IS-IS distance 100 per address family on each router. Verify each router.

Step 2

Wait for the stability period, then remove OSPF. If a selected route changes, stop and find what still needs OSPF.

If the OSPF loopback route disappears, an IS-IS loopback route can replace it. If no usable next-hop route remains, BGP can lose the path. During migration, read both lines.

M12 · OSPF to IS-IS: the topology and three states



The costs are the same in both IGPs: 40 on the lower links, 10 on the others. IS-IS advertises the loopbacks and the locators; OSPF and OSPFv3 are in area 0.

State	PE6 loopback	PE6 locator
1 Both IGPs, default distance	OSPF 110	IS-IS 115
2 Step 1: IS-IS distance 100	IS-IS 100	IS-IS 100
3 Step 2: OSPF removed	IS-IS 100	IS-IS 100

The real change is step 1. Step 2 only removes a protocol that nobody uses, and the selected routes should not change.

M12 · Dynamic SID values: e008 became e002

Test (the two core links of PE1)	Result
T1	Bounce Gi0/0/0/1 only (the policy stays up via P2): every value unchanged
T2	Both links down for about 10 s (policies down, all uA and uB6 released): every value came back unchanged
T3	Both links down for 240 s (past the holdtime), Gi0/0/0/1 restored first, Gi0/0/0/0 a minute later: values were reassigned from the lowest free one, in the order the owners returned

The manager shows SID holdtime: 3 mins

- The color 100 Binding SID was e008 in Ep 15 and e002 in Ep 22
- The configuration did not change: binding-sid dynamic

After an outage over three minutes, check every uA and Binding SID again, or use explicit values.

M12 · What to re-verify on hardware

Topic	Your hardware: read your guide	Our virtual routers
Micro-SID profile	Some hardware needs a hardware profile, effective after a reload	No such step
SIDs per carrier	Depends on the router role: read the units (micro-SIDs or full SIDs)	Max-H-Encap 3, Max-H-Insert 3
Locator	Read your guide: the limits for locators and blocks	XRd reports 16
SID format	Check whether your release allows two formats together	f3216 only
Families to test	Cisco NCS 540, ASR 9000, Cisco 8800: each family has its own guide and limits; check every card that can carry a packet	Not applicable
E-LAN	Needs a supported forwarding implementation	XRd and XRv9000 both reject the bridge domain

M12 · The hardware test plan: six items

1

Profile and reload

Apply the profile and reload in the documented order, verify the state, then configure locators

2

Encapsulation limits

Test the longest policy and repair path against the encapsulation limits

3

MTU and ICMP

Capture every case. Record the installed backup type: TI-LFA, plain LFA, none. With TI-LFA the PTB is 64 short

4

Failure timing

Measure with continuous traffic. One ping per second cannot time a short interruption

5

E-LAN

Test the theory of Ep 14 on supported hardware: route types, MAC learning, broadcast replication, multihoming

6

EVPN IRB anycast gateway

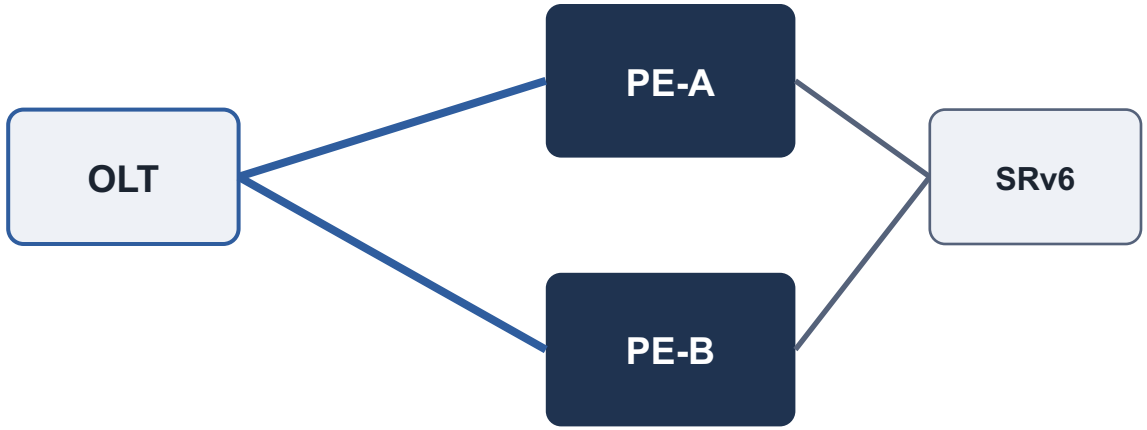
An OLT bundle and two firewalls: route types, failure, MAC moves (next slide)

Virtual results to treat with care

Repair interruption, BFD detection, microloop delay; SID ping replies; ACL deny counters; DSCP not copied by default. Repeat all of them on the target hardware.

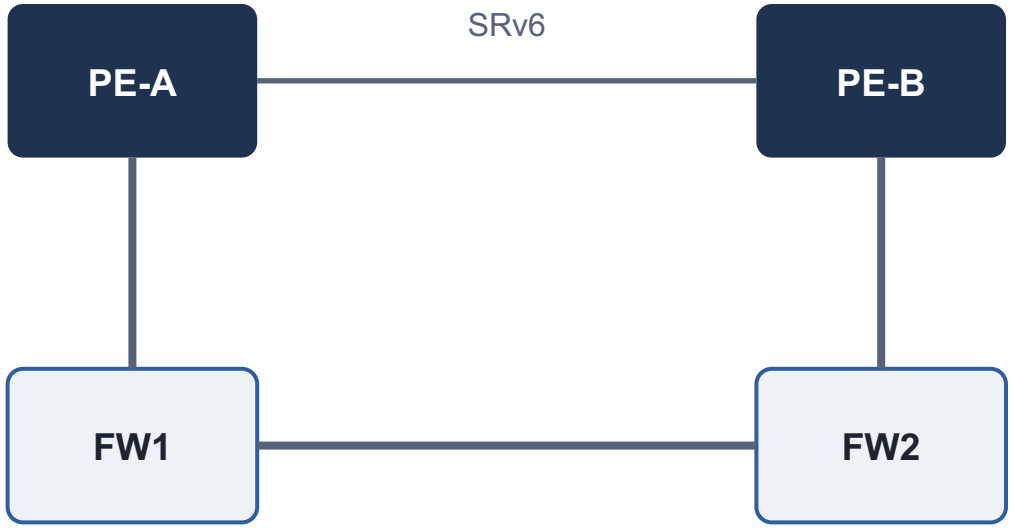
M12 · A hardware-only test: the EVPN IRB anycast gateway

Case 1: an OLT with a bundle to two PEs



One ESI; the IRB has the same IP and the same MAC on both PEs

Case 2: two firewalls in a square



Option 1: both PEs have the same anycast gateway. Option 2: the firewalls run HSRP or VRRP, and the PEs only carry the VLAN. We did not test either.

- Control plane: capture route types 1, 2 and 4 (ESI, MAC and IP of the gateway)
- Gateway: both PEs answer ARP or ND with the same MAC
- Failure: shut one PE link, measure the interruption, check the recovery
- Loops, MAC moves and path (which PE, which SID): capture on both PEs and on the core



Lab and study plan

One lab, from the first day to graduation.

Lab platform: six XRd vRouters and two CE hosts on CML

6 + 2

XRd vRouter 26.2.2 nodes + Alpine CE-A and CE-B

2 / 7

vCPU / GiB per XRd node

2.10.0

the CML version we used

It can prove

- Whether the configuration syntax is accepted
- Control plane: IS-IS, BGP, Flex-Algo, PCEP
- SID allocation and behaviors
- Packet format: every link can be captured
- TI-LFA repair lists, CEF installation and the repair on the wire
- The LDP, SR-MPLS and VPWS data planes
- Results are labelled Virtual lab

It cannot prove

- Convergence time, 50 ms, traffic gaps: the same failure lost 0 pings, then 8
- NPU resources, profiles and reload, line-card differences, scale
- E-LAN: XRd 26.2.2 and XRv9000 26.1.1 both reject the bridge domain
- ODN with a Flex-Algo constraint: not supported
- Links can go one-way with no change: check all 16 adjacencies first

The ladder of 12 labs, mapped to the 24 episodes

L1 State map
Who holds which state
Ep 1

L2 Two lookups
Break the locator, then the SID
Ep 3

L3 MTU and PMTUD
Make the PTB actually appear
Ep 6–7

L4 uSID shift
Ping a container, capture three links
Ep 4–5

L5 Bring-up
Rebuild one node to acceptance
Ep 2

L6 IS-IS five layers
Three single-layer faults
Ep 8

L7 L3VPN
Two tenants, dual stack, isolation
Ep 12–13

L8 EVPN
VPWS; E-LAN is theory only
Ep 14

L9 TI-LFA
Hand-calculated versus CEF repair lists, plus microloop
Ep 9–10

L10 Policy, Flex-Algo
Steer, then prove it; ODN needs a PCE
Ep 11, 15–16

L11 Troubleshooting
Five-layer method, four faults with symptoms only
Ep 17–18

L12 Boundary to graduation
Boundary ACL, three migrations, five cases
Ep 19–24

A four-week rhythm



The packet and bring-up

M1–M4 L1–L5

Ep 1–7

By the end

Can compute a container by hand and confirm it with a capture

Reachability and services

M5–M7 L6–L8

Ep 8, 12–14

By the end

Can walk the resolution chain to the first divergence

Resilience and intent

M8–M10 L9–L10

Ep 9–11, 15–16

By the end

Can predict a repair list and the real path of a policy

Operate, migrate, graduate

M11–M12 L11–L12

Ep 17–24

By the end

Can deliver the graduation lab: five areas and five cases with evidence

About 3 hours of material and 3 hours of lab each week. This is a design estimate, not measured effort. Week 4 holds 8 episodes and is the heaviest: split it over two weeks if needed.

Graduation lab: build five areas

Topology: two PEs, four P routers and two customer hosts, one host on each PE

Give the links different costs so that loose, strict and repair paths have alternative links. The platform must support SRv6 forwarding: we used XRd vRouter. The control-plane-only XRd image lacks the required forwarding.

1 Underlay

IS-IS, locators in two blocks, node SIDs and adjacency SIDs

2 Protection and algorithms

TI-LFA, microloop avoidance and a delay-based Flex-Algo

3 Services

Two L3 tenants with an overlapping address and one point-to-point L2 service

4 Steering

An explicit policy and an on-demand policy calculated by a PCE

5 Operations

A healthy baseline, a boundary ACL and a migration rehearsal with rollback

Build order: pass each checkpoint before the next step, and save every router configuration at each checkpoint

IS-IS and locators → **Protection** → **Tenants** → **VPWS** → **Policies and ACL**

Graduation lab: five cases and grading

Case	Task	Evidence
A	Capture a BGP update and rebuild the service SID from the label field and the Prefix-SID attribute	The capture and its decode (Ep 13)
B	Fail a link during traffic and count lost packets; repeat at least once	Captures of both candidate paths (the two runs in Ep 9 differed)
C	Find the largest L3VPN and L2 packet; test one byte above each	Passing and failing pings, plus captures
D	Ask a colleague to break one service direction; find it with captures at both PEs	Captures of both access links and both PE core links
E	Test SID reachability from an untrusted lab port, before and after filtering	A capture of the first core link; set the source address

Pass

All five areas built, and evidence for cases A, C and E

Merit

Capture evidence for all five cases

Distinction

Also explain a difference from our results, with evidence

Graduation lab: Do, Measure and Proof for each case

Case	Do	Measure	Proof
A	Flap a test loopback in a VRF on PE6	Capture the PE6 core link while the update leaves	Decode the update; 917568 = 0xE0040 = e004
B	Stream from CE-A to CE-B; fail P2–P4	Count lost packets; run at least twice	Captures on both paths: P2–P4 before, P3–P5 after
C	Ping CE-A to CE-B with growing sizes; L3VPN, then VPWS	Find the largest passing size; add one byte	L3VPN 1460 / 1461; VPWS 1446 / 1447
D	A colleague breaks one direction of a service	Ping CE-A to CE-B; find where the packets stop	Captures of the access and core links at both PEs
E	An outside host with a set source address pings a SID	Replies before the ACL, none after; do not trust counters	Capture the first core link of the PE

Graduation lab: reference values from our lab

Item	Our value
Locator metric of PE6 on PE1	31: three links at 10 each plus 1 for the locator
Algo 128 locator metric	3001, through P3 and P5
L3VPN frame	Customer 98 bytes, core 138 bytes (no FCS)
Label 917568	Rebuilds function e004 with the transposition parameters
VPWS	Core frame 152 bytes, outer next header 143
Largest L3VPN packet	1460 bytes; 1436 with one SRH entry
Largest VPWS packet	1446 bytes; one more byte is lost and the customer sees no error
Ping to a multi-node container	No final reply: the checksum used the original destination
ACL deny counters	Stayed at zero while captures showed drops
PTB	Egress IPv6 MTU minus 64 with an installed TI-LFA repair path; exact with no backup or a plain LFA. Forwarding uses the real MTU

Graduation lab: five traps from our recording

One-way links

Links developed one-way failures with no configuration change. Verify every adjacency before you record a result.

dot1q subinterfaces

On the CML vRouter, tagged frames arrive but the replies leave untagged. Untagged ports work.

Stopped nodes

A stopped CML node starts from its day-0 configuration. Keep snapshots and verify persistence for your image.

Hosts with many addresses

A host with several addresses can pick an unexpected source. Set the source address in every test.

Failed commit

Read show configuration failed, then abort. Restore a known configuration from a snapshot and verify the restored state.

Sixteen wrong conclusions people reach most often (1 of 2)

"An SR core holds no state"

IGP, FIB, local SID and protection state are all still there

"SL = 0 means processing is done"

Check the DA, argument, behavior and Next Header as well

"Six slots carry six nodes"

The service function and a block change take slots too

"The DA matches my math, so no SRH"

Whether an SRH exists is read from the Next Header chain

"Commit succeeded, so the data plane works"

Accepted, installed, reachable and end-to-end are four levels

"TI-LFA is always 50 ms"

The same failure lost 0 pings, then 8. Speed is a hardware question

"BGP is up with routes, so the tenant uses SRv6"

Walk the resolution chain, then capture

"Algo 128 means low latency"

The FAD, the participants and the real metrics decide

Sixteen wrong conclusions people reach most often (2 of 2)

"The policy is up, so traffic uses my path"

Our first policy was up and valid, yet it used seven hops and two U-turns

"ODN calculates the path"

This release needs a PCE; without one traffic silently takes the shortest path

"The SID replied, so the SID works"

The loopback replied; the uDT4 did no VRF lookup

"The traceroute stops, so the path is broken"

The last node drops the probe (checksum); confirm with a capture

"The deny counter is zero, so the ACL does nothing"

The capture showed the drops; a counter is not proof either way

"All tests pass, so there is no problem"

Fault 4 passed every test until the link failed

"Both PEs must move in one maintenance window"

Each PE moved alone and GREEN stayed up

"The PTB MTU is the real MTU"

With a TI-LFA repair path it is the egress MTU minus 64; forwarding uses the real MTU

Standards behind the experiments

RFC 8402
Segment Routing architecture

RFC 8754
IPv6 Segment Routing Header

RFC 8986
SRv6 Network Programming

RFC 9800
Compressed SRv6 segment list (uSID)

RFC 9352
IS-IS extensions for SRv6

RFC 9252
BGP overlay services based on SRv6

RFC 9256
SR Policy architecture

RFC 9350
IGP Flexible Algorithm

RFC 9259
OAM in SRv6

RFC 8231
PCEP extensions for stateful PCE

RFC 8664
PCEP extensions for Segment Routing

RFC 8214
VPWS support in EVPN

RFC 8660
SR with the MPLS data plane

RFC 9513
OSPFv3 extensions for SRv6

RFC 8491
Signaling MSD using IS-IS

Implementation claims follow the Cisco IOS XR Segment Routing Configuration Guide of each platform and defer to real output on the target release. The evidence is in the episode videos.



Appendix: Lab sheets

The commands, recorded outputs and readings of every lab, by episode. Follow them in your own lab and compare the output. Dynamic values (SIDs, counters, times) differ: read them, do not copy them.

Ep 1.1 · What P2 actually holds

Router / tool: P2 · Source: lab-evidence/01-state/P2.log

```
RP/0/RP0/CPU0:P2#show bgp summary
% BGP instance 'default' not active

RP/0/RP0/CPU0:P2#show route vrf all summary
VRF: **iid
connected                       0          0          0          0
Total                           0          0          0          0

RP/0/RP0/CPU0:P2#show segment-routing srv6 locator
Name                  ID       Algo  Prefix                    Status   Flags
FA128                 1        128   fcbb:bb01:2::/48          Up       U
MAIN                  2        0     fcbb:bb00:2::/48          Up       U

RP/0/RP0/CPU0:P2#show segment-routing srv6 manager | include Number of SIDs
  Number of SIDs: 12 (0 stale)
```

- Now we examine P2 in the finished lab. All the services in this lab go through it.
- BGP does not run on P2.
- There is no customer VRF. The only entry is an internal entry with no routes.
- P2 has two locators. MAIN is the locator that we use most. The second locator is for Flex-Algo. Episode 11 explains it.
- P2 has 12 local SIDs: its node and adjacency SIDs. With three customers or three thousand customers, P2 keeps the same state.

Ep 1.2 · And what PE1 holds

Source: lab-evidence/01-state/PE1.log (service SIDs and route counts only)

```
RP/0/RP0/CPU0:PE1#show segment-routing srv6 sid
fcbb:bb00:1:e004::          uDT4              'BLUE'                          bgp-65000           InUse  Y
fcbb:bb00:1:e005::          uDT4              'RED'                           bgp-65000           InUse  Y
fcbb:bb00:1:e006::          uDT6              'BLUE'                          bgp-65000           InUse  Y
fcbb:bb00:1:e007::          uDT6              'RED'                           bgp-65000           InUse  Y
fcbb:bb00:1:e00a::          uDT4              'GREEN'                         bgp-65000           InUse  Y

RP/0/RP0/CPU0:PE1#show route vrf all summary
VRF: RED     ...  bgp 65000      2
VRF: BLUE    ...  bgp 65000      2
VRF: GREEN   ...  bgp 65000      1
```

- Now the edge. This is PE1 in the same finished lab.
- PE1 has a service SID for each customer and address family: two for BLUE, two for RED and one for GREEN.
- PE1 also keeps the remote routes of each customer, from BGP. The per-customer state is on the edge, and only on the edge.

Ep 1.3 · What the customer sees

Router / tool: CE-A · Source: lab-evidence/11-ce/v3-CE-A-ping.log

```
CE-A:~$ sudo traceroute -n -s 10.1.1.2 10.1.6.2 -w 1 -q 1 -m 8
traceroute to 10.1.6.2 (10.1.6.2) from 10.1.1.2, 8 hops max, 46 byte packets
 1  10.1.1.1  1.259 ms
 2  10.1.6.1  2.902 ms
 3  10.1.6.2  1.610 ms
```

- We examine the same fact from the customer side, with a traceroute from CE-A to CE-B.
- The first hop is PE1, the gateway of the customer.
- The next hop is PE6, then CE-B. The four core routers do not appear. They do not touch the customer packet. They touch only the outer header.

Ep 2.1 · Know what you are running

Source: lab-evidence/01-baseline/PE1.log

```
RP/0/RP0/CPU0:PE1#show version
Cisco IOS XR Software, Version 26.2.2 LNT
Copyright (c) 2013-2026 by Cisco Systems, Inc.

Build Information:
 Built By     : cisco
 Built On     : Thu Sep 10 00:39:05 UTC 2026
 Workspace    : /auto/srcarchive13/prod/26.2.2/xrd-vrouter/ws/
 Version      : 26.2.2
 Label        : 26.2.2

cisco XRd vRouter
cisco XRd-vRouter-C-01 processor with 9GB of memory
XRd vRouter Container
```

- Before you configure anything, record the software that you use. This is the first line of every lab report.
- The version is IOS XR 26.2.2.
- The platform is XRd vRouter, a container. All the output in this series comes from this platform and this release only.

Ep 2.2 · Step 1: addresses

```
interface Loopback0
 description router-id and SRv6 encapsulation source
 ipv6 address 2001:db8:100::1/128
!
interface GigabitEthernet0/0/0/0
 description TO-P2 cost 10
 ipv6 address 2001:db8:200:12::/127
 no shutdown
!
interface GigabitEthernet0/0/0/1
 description TO-P3 cost 40
 ipv6 address 2001:db8:200:13::/127
 no shutdown
```

- The configuration has three steps. This is PE1. The other five nodes use the same pattern.
- Step 1 is the addresses. Loopback0 gets the /128 from our plan. It is the router ID and the source of every encapsulated packet.
- The two core interfaces get their /127 addresses. There is no IPv4 address, no MPLS and no LDP.

Ep 2.3 · Step 2: SRv6 and the locator

```
segment-routing
 srv6
  encapsulation
   source-address 2001:db8:100::1
  !
  locators
   locator MAIN
    micro-segment behavior unode psp-usd
    prefix fcbb:bb00:1::/48
   !
  !
 !
```

- Step 2 enables SRv6. Only four lines are important.
- The encapsulation source address is the loopback. When this router encapsulates a customer packet, this address is the outer source.
- The locator has the name MAIN. Its prefix is the /48 of this node.
- This line makes it a micro-SID locator. The node SID is a uN with the PSP and USD flavors. Episode 4 explains these flavors.
- On hardware platforms, you can also need a hardware profile and a reload. This virtual router needs neither.

Ep 2.4 · Step 3: IS-IS carries the locator

```
router isis CORE
 is-type level-2-only
 net 49.0001.0000.0000.0001.00
 address-family ipv6 unicast
  metric-style wide
  segment-routing srv6
   locator MAIN
 !
 interface Loopback0
  passive
  address-family ipv6 unicast
 !
 interface GigabitEthernet0/0/0/0
  point-to-point
  address-family ipv6 unicast
   metric 10
```

- Step 3 is IS-IS. Most of this configuration is normal IS-IS.
- We enable the IPv6 address family with wide metrics.
- These two lines are the SRv6 part. They attach the locator to IS-IS. Then IS-IS advertises the locator and creates the adjacency SIDs.
- If you do not include them, the commit is still successful. The locator is on this router, but no other router can reach it.
- Each core interface is point-to-point, with the metric from our plan.

Ep 2.5 · Check 1: IS-IS neighbors

Router / tool: P2 · Source: lab-evidence/01-baseline/P2.log

```
RP/0/RP0/CPU0:P2#show isis neighbors

IS-IS CORE neighbors:
System Id      Interface        SNPA           State Holdtime Type IETF-NSF
PE1            Gi0/0/0/0        *PtoP*         Up    28       L2   Capable
P3             Gi0/0/0/2        *PtoP*         Up    23       L2   Capable
P4             Gi0/0/0/1        *PtoP*         Up    28       L2   Capable

Total neighbor count: 3
```

- Check 1, on P2. First, count the neighbors and compare them with the diagram.
- P2 has three neighbors: PE1, P3 and P4. All are up, level 2 and point-to-point.
- The diagram also shows three. Each PE has two neighbors, and each P router has three.

Ep 2.6 · Check 2: what the router says it can do

Source: lab-evidence/01-baseline/PE1.log

```
RP/0/RP0/CPU0:PE1#show segment-routing srv6 manager
Parameters:
  SRv6 Enabled: Yes
  SRv6 Operational Mode:
    Micro-segment:
      SID Base Block: fcbb:bb00::/24
  Encapsulation:
    Source Address:
      Default: 2001:db8:100::1
      Configured: 2001:db8:100::1
  SID Formats:
    f3216 <32B/16NFA> (2)
      uSID LIB Range:
        LIB Start   : 0xe000
        ELIB Start  : 0xfe00
      uSID WLIB Range:
        EWLIB Start : 0xfff7
```

- Check 2 is one command: show segment-routing srv6 manager. The output is long, so we read it in three parts.
- SRv6 is enabled, and the operational mode is micro-segment.
- The encapsulation source is the loopback that we configured.
- The SID format is f3216. This is a 32-bit block, then 16-bit node and function IDs.
- Local functions start at hexadecimal e000. Remember this number. You will see it in every SID table in this series.

Ep 2.7 · Check 2, continued: behaviors

Source: lab-evidence/01-baseline/PE1.log (uSID entries; the full list also has End and End.DT forms)

```
Platform Capabilities:
  SRv6: Yes
  TILFA: Yes
  Microloop-Avoidance: Yes
  Endpoint behaviors:
    uN (PSP/USD)
    uA (PSP/USD)
    uDT6
    uDT4
    uDT46
    uDX2
    uDT2U
    uDT2M
    uB6 (Insert.Red)
  Headend behaviors:
    T
    H.Insert.Red
    H.Encaps.Red
    H.Encaps.L2.Red
```

- The second part lists the behaviors that this router supports.
- uN and uA are the node and adjacency behaviors. Every path uses them.
- The uDT and uDX entries are service behaviors. They are for Layer 3 VPN and for point-to-point Ethernet.
- uB6 is the Binding SID. A behavior in this list does not prove that every related feature works. For example, this platform rejects bridge domains.
- The headend list has H.Insert.Red and H.Encaps.Red, the reduced forms. It does not have plain H.Encaps.
- This missing line changes the packets in all later episodes. Read this output before you design anything.

Ep 2.8 · Check 2, continued: depth

Source: lab-evidence/01-baseline/PE1.log

```
Signaled parameters:
  Max-SL          : 3
  Max-End-Pop-SRH : 3
  Max-H-Insert    : 3 sids
  Max-H-Encap     : 3 sids
  Max-End-D       : 4
Default parameters (under srv6):
  Encapsulation:
    Hop-Limit     : value=0, propagate=No
    Traffic-class : value=0, propagate=No
Max Locators: 16
Max SIDs: 64000
```

- The third part is about limits.
- The router signals these depths to its neighbors. It can insert or encapsulate 3 SIDs.
- Here, a SID is a full 128-bit container. One container holds several micro-SIDs. Thus the limit of 3 is larger than it looks.
- By default, the router does not copy the hop limit and the traffic class from the customer packet to the outer header. Episode 13 checks this on the wire.
- The limits are 16 locators and 64000 SIDs. These are the limits of a virtual router. Do not use them for hardware.

Ep 2.9 · Check 3: the locator and the local SIDs

Source: lab-evidence/01-baseline/PE1.log

```
RP/0/RP0/CPU0:PE1#show segment-routing srv6 locator
Name                  ID       Algo  Prefix                    Status   Flags
--------------------  -------  ----  ------------------------  -------  --------
MAIN                  1        0     fcbb:bb00:1::/48          Up       U

RP/0/RP0/CPU0:PE1#show segment-routing srv6 sid

*** Locator: 'MAIN' ***

SID                         Behavior          Identification                  Owner               State  RW
--------------------------  ----------------  ------------------------------  ------------------  -----  --
fcbb:bb00:1::               uN (PSP/USD)      'default':1                     sidmgr              InUse  Y
fcbb:bb00:1:e000::          uA (PSP/USD)      [Gi0/0/0/0, Link-Local]:0       isis-CORE           InUse  Y
fcbb:bb00:1:e001::          uA (PSP/USD)      [Gi0/0/0/1, Link-Local]:0       isis-CORE           InUse  Y
```

- Check 3 uses two commands. You will use them more than any other commands in this series.
- The locator MAIN is up, in algorithm 0, with the prefix that we configured.
- The first SID is the node SID, a uN. The SID manager owns it. It is available when the locator is up.
- The next two SIDs are adjacency SIDs, one for each core interface. IS-IS owns them. That is why step 3 is important.
- Their functions are e000 and e001. These are the first two values of the local range.

Ep 2.10 · The remote locator, as PE1 sees it

Source: lab-evidence/01-baseline/PE1.log (four of sixteen routes)

```
RP/0/RP0/CPU0:PE1#show route ipv6 isis

i L2 2001:db8:100::6/128
      [115/30] via fe80::5454:ff:fe9b:6d1f, 01:12:13, GigabitEthernet0/0/0/0
i L2 fcbb:bb00:2::/48
      [115/11] via fe80::5454:ff:fe9b:6d1f, 01:12:32, GigabitEthernet0/0/0/0
i L2 fcbb:bb00:4::/48
      [115/21] via fe80::5454:ff:fe9b:6d1f, 01:12:13, GigabitEthernet0/0/0/0
i L2 fcbb:bb00:6::/48
      [115/31] via fe80::5454:ff:fe9b:6d1f, 01:12:13, GigabitEthernet0/0/0/0
```

- A local SID is useful only when other routers can reach it. Thus, on PE1, we examine the routes from IS-IS.
- PE6's loopback has metric 30, through GigabitEthernet 0. This is the upper path, as we predicted.
- PE6's locator is a different route, with metric 31. On this release, the router adds 1 to the path cost for the locator.
- The locators of P2 and P4 are also here, with metric 11 and 21. The locator of each node is a normal IPv6 route.
- This is an important property of SRv6. A router in the middle needs no new function to forward a packet to a SID. It needs only a route.

Ep 2.11 · Check 4: ping the node SID

Source: 04-behaviors/ua-PE1.log

```
RP/0/RP0/CPU0:PE1#ping fcbb:bb00:6:: source 2001:db8:100::1 count 2
Type escape sequence to abort.
Sending 2, 100-byte ICMP Echos to fcbb:bb00:6::, timeout is 2 seconds:
!!
Success rate is 100 percent (2/2), round-trip min/avg/max = 2/2/3 ms
```

- This is the ping command on PE1. The destination is the node SID of PE6, fcbb:bb00:6::.
- The source option sets the source address. We use the loopback of PE1, because every router can reach it. The reply needs this address to come back. The count option sends two packets.
- The router sends two echo requests, each with 100 bytes.
- Each exclamation mark is one reply that came back.
- The success rate is 100 percent: 2 of 2. The ping works. But the ping alone does not show the path. For this, we need the captures.

Ep 2.12 · Check 4: the packet on three links

Router / tool: tcpdump · Source: 04-behaviors/ua-PE1-P2, ua-P2-P4, ua-P4-PE6 (first request on each link)

```
# PE1 to P2
IP6 (hlim 60) 2001:db8:100::1 > fcbb:bb00:6::: ICMP6, echo request, seq 0
# P2 to P4
IP6 (hlim 59) 2001:db8:100::1 > fcbb:bb00:6::: ICMP6, echo request, seq 0
# P4 to PE6
IP6 (hlim 58) 2001:db8:100::1 > fcbb:bb00:6::: ICMP6, echo request, seq 0
IP6 (hlim 64) 2001:db8:100::6 > 2001:db8:100::1: ICMP6, echo reply, seq 0
```

- This is the output of tcpdump. Each line is one packet. The first part of each line shows the hop limit. Then we see the source address, the destination address, and the packet type. We show the first echo request of each capture.
- On the first link, the destination is the SID, and the hop limit is 60.
- On the second link, the destination is the same, and the hop limit is 59.
- On the third link, the hop limit is 58. P2 and P4 did not do anything special. They routed the packet. Each router lowered the hop limit by 1.
- This last line is the reply, on the third link. It comes from the loopback of PE6, 2001:db8:100::6, not from the SID.

Ep 3.1 · The egress tables behind it

Router / tool: PE6 · Source: lab-evidence/03-faults/v3wire/red-PE6.log

```
RP/0/RP0/CPU0:PE6#show segment-routing srv6 sid fcbb:bb00:6:e006:: detail
fcbb:bb00:6:e006::          uDT4              'RED'                              bgp-65000           InUse  Y
  SID Function: 0xe006
  SID Identification (Context): { table-id=0xe0000001 ('RED':IPv4/Unicast) }

RP/0/RP0/CPU0:PE6#show cef ipv6 fcbb:bb00:6:e006::/64
fcbb:bb00:6:e006::/64, version 179, SRv6 Endpoint uDT4
    next hop VRF - 'RED', table - 0xe0000001
    0     Y   recursive                Lookup in table

RP/0/RP0/CPU0:PE6#show route vrf RED 10.2.6.2
Routing entry for 10.2.6.0/24
  Known via "connected", distance 0, metric 0 (connected)
    directly connected, via GigabitEthernet0/0/0/3
```

- These are the egress tables that did the work.
- The SID is a uDT4. Its context is the IPv4 table of VRF RED.
- In CEF, the SID is a normal /64 entry. Its action is a lookup of the inner packet in the RED table.
- In RED, the customer address is directly connected on the port to CE-B. This is the evidence for condition 4.

Ep 3.2 · Healthy 1: the loopback ping and the locator route

Source: lab-evidence/03-faults/v3/0-healthy-PE1.log

```
RP/0/RP0/CPU0:PE1#ping 2001:db8:100::6 source 2001:db8:100::1 count 2
Type escape sequence to abort.
Sending 2, 100-byte ICMP Echos to 2001:db8:100::6, timeout is 2 seconds:
!!
Success rate is 100 percent (2/2), round-trip min/avg/max = 2/2/3 ms

RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
Routing entry for fcbb:bb00:6::/48
  Known via "isis CORE", distance 100, metric 31, SRv6-locator, type level-2
  Routing Descriptor Blocks
    fe80::5454:ff:fe66:75e1, from 2001:db8:100::6, via GigabitEthernet0/0/0/0, Protected
      Route metric is 31
    fe80::5454:ff:fec1:58eb, from 2001:db8:100::6, via GigabitEthernet0/0/0/1, Backup (TI-LFA)
      Repair Node(s): 2001:db8:100::5
      Route metric is 101
```

- We start with the healthy lab. First, the check that most engineers do first: a ping to the loopback of PE6.
- The destination is 2001:db8:100::6, the loopback of PE6. The source is the loopback of PE1.
- Both replies return, and the success rate is 100 percent. This ping tests only the loopback. It does not use the locator.
- Now the locator. PE1 has a route to fcbb:bb00:6::/48. IS-IS installed it, and the route type is SRv6-locator.
- The primary path leaves PE1 on Gi0/0/0/0. The route metric is 31.
- A backup path leaves on Gi0/0/0/1, with metric 101. Remember this: in the healthy state, PE1 has a primary path and a backup path to the locator.

Ep 3.3 · Healthy 2: the customer route in BGP

Source: lab-evidence/03-faults/v3/0-healthy-PE1.log

```
RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
Paths: (1 available, best #1)
    2001:db8:100::6 (metric 30) from 2001:db8:100::6 (10.0.0.6)
      Received Label 0xe0080
      Origin incomplete, metric 0, localpref 100, valid, internal, best, group-best
      Extended community: RT:65000:300
       SID Locator: fcbb:bb00:6::/48, RIB metric: 31
        T:1(Sid information), Sid:fcbb:bb00:6::(Transposed), Behavior:63
         Length [Loc-blk,Loc-node,Func,Arg]:[32,16,16,0], Tpose-len:16, Tpose-offset:48

RP/0/RP0/CPU0:PE1#show route vrf GREEN 10.30.6.6/32
Routing entry for 10.30.6.6/32
  Known via "bgp 65000", distance 200, metric 0, type internal
    2001:db8:100::6, from 2001:db8:100::6
      Nexthop in Vrf: "default", Table: "default", IPv6 Unicast, Table Id: 0xe0800000
```

- Next, the customer route. On PE1, we read the BGP detail of the prefix 10.30.6.6 in VRF GREEN.
- BGP has one path. It comes from PE6. The next hop is the loopback of PE6, and the IGP metric to it is 30.
- The received label is 0xe0080. It holds the SID function e008. Fault B changes this line.
- The path is valid and best. The route target 65000:300 puts the route into VRF GREEN.
- BGP also checks the locator of the SID. The locator route is in the RIB, with metric 31. Fault A changes this line.
- In the VRF, the route is a BGP route. Its next hop is in the default VRF, in the IPv6 table. PE1 must resolve the next hop in that table.

Ep 3.4 · Healthy 3: the forwarding entry and the customer ping

Source: lab-evidence/03-faults/v3/0-healthy-PE1.log

```
RP/0/RP0/CPU0:PE1#show cef vrf GREEN 10.30.6.6/32
10.30.6.6/32, version 9, SRv6 Headend
   via fcbb:bb00:6::/128, 3 dependencies, recursive
    next hop VRF - 'default', table - 0xe0800000
    next hop fcbb:bb00:6::/128 via fcbb:bb00:6::/48
    SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e008::}
    Hash  OK  Interface                 Address
    0     Y   GigabitEthernet0/0/0/0    fe80::5454:ff:fe66:75e1

RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Type escape sequence to abort.
Sending 3, 100-byte ICMP Echos to 10.30.6.6 timeout is 2 seconds:
!!!
Success rate is 100 percent (3/3), round-trip min/avg/max = 1/1/3 ms
```

- Now forwarding. The CEF entry shows what PE1 does with a packet for this customer.
- The entry is an SRv6 Headend. It has 3 dependencies. This means that the router resolved the entry.
- The next hop is the locator of PE6. PE1 resolves it with the /48 route. This is condition 1.
- The action is H.Encaps.Red. The SID list has one SID: fcbb:bb00:6:e008::. This is the service SID of GREEN on PE6.
- The last line shows the result. The packet leaves PE1 on Gi0/0/0/0.
- Now the customer ping. The source is an address of PE1 in VRF GREEN. All three packets succeed.

Ep 3.5 · Healthy 4: the customer ping on the wire

Router / tool: tshark · Source: /healthy2-PE1-P2.pcap (fields of two frames)

```
# PE1 to P2: the first request and its reply
frame 113, 154 bytes, request
  IPv6  2001:db8:100::1 > fcbb:bb00:6:e008::    hlim 255
  IPv4  10.30.1.1 > 10.30.6.6                   ICMP echo request
frame 114, 154 bytes, reply
  IPv6  2001:db8:100::6 > fcbb:bb00:1:e00a::    hlim 253
  IPv4  10.30.6.6 > 10.30.1.1                   ICMP echo reply
```

- Now the packets. We captured the customer ping on the link from PE1 to P2. Each frame has 154 bytes: 14 for Ethernet, 40 for the outer IPv6 header, and 100 for the customer packet.
- The request has an outer IPv6 header. Its destination is the service SID of PE6, fcbb:bb00:6:e008::. The source is the loopback of PE1.
- Inside is the customer packet: an IPv4 echo request from 10.30.1.1 to 10.30.6.6. There is no SRH, because the SID list has only one SID.
- The reply goes the other way. Its outer destination is fcbb:bb00:1:e00a::. This is the uDT4 SID of GREEN on PE1.
- The hop limit of the reply is 253. PE6 sent it with 255. P4 and P2 each lowered it by 1.
- This is the healthy reference. In the two faults, we capture the same link, and we compare.

Ep 3.6 · Fault A: PE6 stops advertising its locator

```
! PE6
router isis CORE
 address-family ipv6 unicast
  segment-routing srv6
   no locator MAIN
!
commit
```

- Fault A. On PE6, we remove one line: the locator under IS-IS. Episode 2 told you about this line.
- IS-IS still advertises the loopback. The locator is still on PE6. Only the advertisement of the locator is gone.

Ep 3.7 · Fault A: what PE1 sees

Source: lab-evidence/03-faults/v3/A-PE1.log

```
RP/0/RP0/CPU0:PE1#ping 2001:db8:100::6 source 2001:db8:100::1 count 2
Success rate is 100 percent (2/2), round-trip min/avg/max = 2/2/3 ms

RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
% Network not in table

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
Paths: (1 available, best #1)
      Received Label 0xe0080
      Origin incomplete, metric 0, localpref 100, valid, internal, best, group-best
       SID Locator: fcbb:bb00:6::/48, RIB metric: inaccessible
```

- Now we do the same checks from PE1.
- The loopback ping is successful. This is the check that gave the wrong answer in the phone call.
- The locator route is gone: Network not in table. Condition 1 is false.
- But BGP still shows the path as valid and best.
- Only this line shows the problem: the SID locator is inaccessible. You see it only when you read the detail.

Ep 3.8 · Fault A: forwarding

Source: lab-evidence/03-faults/v3/A-PE1.log

```
RP/0/RP0/CPU0:PE1#show cef vrf GREEN 10.30.6.6/32
10.30.6.6/32, version 9, SRv6 Headend
   via fcbb:bb00:6::/128, 0 dependencies, recursive
    unresolved
    SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e008::}
    0     Y   recursive                drop

RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 0 percent (0/3)
```

- What does forwarding do with a best path whose locator is not reachable?
- The entry is still there, but nothing resolves it: zero dependencies, unresolved.
- The SID list is still in the entry. A fast look can make you think that the encapsulation is ready.
- The last column shows drop.
- The customer ping fails.

Ep 3.9 · Fault A: PE6 still looks healthy

Router / tool: PE6 · Source: lab-evidence/03-faults/v3/A-PE6.log (FA128 locator and other SIDs omitted)

```
RP/0/RP0/CPU0:PE6#show segment-routing srv6 locator
Name                  ID       Algo  Prefix                    Status   Flags
--------------------  -------  ----  ------------------------  -------  --------
MAIN                  2        0     fcbb:bb00:6::/48          Up       U

RP/0/RP0/CPU0:PE6#show segment-routing srv6 sid
SID                         Behavior          Identification                  Owner               State  RW
--------------------------  ----------------  ------------------------------  ------------------  -----  --
fcbb:bb00:6::               uN (PSP/USD)      'default':6                     sidmgr              InUse  Y
fcbb:bb00:6:e008::          uDT4              'GREEN'                         bgp-65000           InUse  Y
```

- Now we examine the egress during fault A. This check tells you where the fault is not.
- The locator MAIN is up. PE6 still owns the prefix fcbb:bb00:6::/48.
- The service SID e008 for GREEN is a uDT4. BGP owns it, and it is in use. Condition 2 is true.
- PE6 owns everything that it needs. But the rest of the network does not know the locator. The fault is in the advertisement.

Ep 3.10 · Fault B: PE6 stops allocating the service SID

```
! PE6
router bgp 65000
 vrf GREEN
  address-family ipv4 unicast
   no segment-routing srv6
!
commit
```

- Fault B is different. On PE6, under BGP, we remove the SRv6 allocation for VRF GREEN.
- The locator, IS-IS and the loopback do not change. Only the service SID for this one customer is gone.

Ep 3.11 · Fault B: the egress

Router / tool: PE6 · Source: lab-evidence/03-faults/v3/B-PE6.log (adjacency SIDs omitted)

```
RP/0/RP0/CPU0:PE6#show segment-routing srv6 sid
*** Locator: 'MAIN' ***
fcbb:bb00:6::              uN (PSP/USD)      'default':6                    sidmgr
fcbb:bb00:6:e004::         uDT4              'BLUE'                         bgp-65000
fcbb:bb00:6:e005::         uDT6              'BLUE'                         bgp-65000
fcbb:bb00:6:e006::         uDT4              'RED'                          bgp-65000
fcbb:bb00:6:e007::         uDX2              300:300                        l2vpn_srv6

RP/0/RP0/CPU0:PE6#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
    Local Label: 24004 (with rewrite);
  Advertised to peers (in unique update groups):
    2001:db8:100::1
```

- Start at the location of the fault: the egress.
- PE6's SID table has service SIDs for BLUE, for RED and for a Layer 2 service. It has nothing for GREEN. Condition 2 is false.
- This result is not expected. Without an SRv6 SID, BGP used an MPLS label, 24004.
- BGP advertised this label to PE1. No warning appeared.

Ep 3.12 · Fault B: the ingress

Source: lab-evidence/03-faults/v3/B-PE1.log

```
RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
  Known via "isis CORE", distance 100, metric 31, SRv6-locator, type level-2

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
      Received Label 24004
      Origin incomplete, metric 0, localpref 100, valid, internal, best, group-best

RP/0/RP0/CPU0:PE1#show cef vrf GREEN 10.30.6.6/32
   via 2001:db8:100::6/128, 0 dependencies, recursive
    unresolved
     labels imposed {24004}
    0     Y   recursive                drop
```

- Now PE1.
- This time, the locator route is correct. Condition 1 is true.
- BGP received label 24004. The path is still valid and best.
- Forwarding tries to push an MPLS label to PE6's loopback. This core has no MPLS path. Thus the entry is unresolved, and it drops the packet.
- One missing line on the egress changed an SRv6 service into an MPLS service that cannot work.

Ep 3.13 · Fault B: the ping and the wire

Source: lab-evidence/03-faults/v3/B-PE1.log, B-cap-PE1-P2.pcap, B-cap-P4-PE6.pcap

```
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Type escape sequence to abort.
Sending 3, 100-byte ICMP Echos to 10.30.6.6 timeout is 2 seconds:
...
Success rate is 0 percent (0/3)

# Captures on PE1 to P2 and on P4 to PE6, during this ping
no echo request on either link
```

- We run the customer ping during fault B.
- Each dot is one timeout. There are three dots, and no reply.
- The success rate is 0 percent.
- We also captured on the link PE1 to P2, and on the link P4 to PE6. Neither link has an echo request. PE1 drops the packet before the wire.
- In both faults, the customer sees the same symptoms: no ping, and no packet on the wire. Only the router outputs show the difference.

Ep 3.14 · Read the label field

Source: lab-evidence/03-faults/v3 (A-PE1.log, B-PE1.log)

```
Healthy:
      Received Label 0xe0080
       SID Locator: fcbb:bb00:6::/48, RIB metric: 31
        T:1(Sid information), Sid:fcbb:bb00:6::(Transposed), Behavior:63
           Length [Loc-blk,Loc-node,Func,Arg]:[32,16,16,0], Tpose-len:16, Tpose-offset:48

Fault B:
      Received Label 24004
```

- This is a fast method to identify the two cases. Read the received label.
- In the healthy state, the label is a hexadecimal value, e0080. This is the SID function, e008, in the label field.
- The next lines show the method. BGP sends the locator one time, and puts the 16-bit function in the label field. The receiver builds the full SID again.
- RFC 9252 calls this method transposition. Episode 13 reads it field by field from a captured update.
- In fault B, the label is a normal decimal MPLS label. In a core with only SRv6, this tells you that there is a problem.

Ep 3.15 · The moment in between: on the wire

Router / tool: tshark · Source: lab-evidence/03-faults/stale-sid-P4-PE6.pcap (fields of two frames)

```
# P4 to PE6, in an earlier version of the lab (VRF RED)
frame 116, 154 bytes
  IPv6  2001:db8:100::1 > fcbb:bb00:6:e004::    hlim 253
  IPv6  2001:db8:a:201::1 > 2001:db8:a:206::6   ICMPv6 echo request
frame 117, 206 bytes
  IPv6  2001:db8:200:46::1 > 2001:db8:100::1    hlim 63
  ICMPv6 type 1, code 0: destination unreachable
```

- This capture is from an earlier version of the lab. The customer was in VRF RED, with IPv6 addresses. The idea is the same.
- The request arrives at PE6. The outer destination is the old SID, fcbb:bb00:6:e004::.
- Inside is the customer packet: an ICMPv6 echo request.
- PE6 does not own this SID. It replies from its own address with ICMPv6 type 1, code 0: destination unreachable.
- The packet reached the correct router, but the router has no instruction for the SID. This is why you read the SID table on PE6.

Ep 4.1 · UN in the forwarding table

Router / tool: P3 · Source: lab-evidence/04-behaviors/P3-cef.log

```
RP/0/RP0/CPU0:P3#show segment-routing srv6 sid
fcbb:bb00:3::               uN (PSP/USD)      'default':3                       sidmgr             InUse  Y

RP/0/RP0/CPU0:P3#show cef ipv6 fcbb:bb00:3::/48 detail
fcbb:bb00:3::/48, version 5, SRv6 Endpoint uN (shift)
   via ::/128, 0 dependencies, weight 0, class 0 [flags 0x0]
    next hop ::/128
```

- We now look at these behaviors in the forwarding table of a real router, P3.
- P3 has the uN fcbb:bb00:3::, with the PSP and USD flavors. We explain the flavors later in this episode.
- In CEF, it is a /48 entry. Its action is SRv6 Endpoint uN, shift. This is step 1 of the procedure, as the router shows it.
- There is no outgoing interface here. After the shift, the router looks up the new address.

Ep 4.2 · UA in the forwarding table

Router / tool: P3 · Source: lab-evidence/04-behaviors/P3-cef.log

```
RP/0/RP0/CPU0:P3#show segment-routing srv6 sid
fcbb:bb00:3:e004::          uA (PSP/USD)      [Gi0/0/0/1, Link-Local]:0:P       isis-CORE           InUse  Y
fcbb:bb00:3:e005::          uA (PSP/USD)      [Gi0/0/0/1, Link-Local]:0         isis-CORE           InUse  Y

RP/0/RP0/CPU0:P3#show isis neighbors
P5             Gi0/0/0/1        *PtoP*         Up    29       L2   Capable

RP/0/RP0/CPU0:P3#show cef ipv6 fcbb:bb00:3:e005::/64 detail
fcbb:bb00:3:e005::/64, version 114, SRv6 Endpoint uA (shift)
   via fe80::5454:ff:feb1:be7a/128, GigabitEthernet0/0/0/1, 8 dependencies
```

- Now we look at an adjacency SID on the same router.
- P3 has two uA SIDs for interface GigabitEthernet 0/0/0/1. One has the mark P, for protected. The other has no mark.
- The router can repair a protected adjacency around a failure. Use the unprotected one when you want only that link. Episode 9 uses this difference.
- That interface goes to P5.
- In CEF, the uA is a /64. The block, the node and the function make 64 bits.
- The uA has a fixed next hop, out of GigabitEthernet 0/0/0/1 to P5. The uN has no fixed next hop.

Ep 4.3 · Ping the uA address

Source: lab-evidence/04-behaviors/ua-PE1.log

```
RP/0/RP0/CPU0:PE1#ping fcbb:bb00:3:e005:6:: source 2001:db8:100::1 count 2
Type escape sequence to abort.
Sending 2, 100-byte ICMP Echos to fcbb:bb00:3:e005:6::, timeout is 2 seconds:
..
Success rate is 0 percent (0/2)

RP/0/RP0/CPU0:PE1#ping fcbb:bb00:6:: source 2001:db8:100::1 count 2
Type escape sequence to abort.
Sending 2, 100-byte ICMP Echos to fcbb:bb00:6::, timeout is 2 seconds:
!!
Success rate is 100 percent (2/2), round-trip min/avg/max = 2/2/3 ms
```

- Now we use that uA. You can do the same in your lab. First, in CML, start a packet capture on every link of the path. Then ping from PE1.
- The destination is fcbb:bb00:3:e005:6::. After the block, it has three parts. 3 is the node P3, e005 is the uA of P3 toward P5, and 6 is the node PE6.
- The ping fails: two dots, and 0 percent. This does not mean that the packet is lost. We show the reason later in this episode.
- For comparison, we ping the node SID of PE6 in the same way.
- This ping works: two replies, and 100 percent. A normal address gets replies. The first address gets none. The captures show the difference.

Ep 4.4 · The capture, link by link

Router / tool: tcpdump · Source: lab-evidence/04-behaviors/ua-PE1-P2, ua-P2-P3, ua-P3-P5, ua-P4-P5, ua-P4-PE6

```
$ tcpdump -nn -t -v -r <capture file> icmp6
# PE1 to P2
IP6 (hlim 60) 2001:db8:100::1 > fcbb:bb00:3:e005:6::: [icmp6 sum ok] ICMP6, echo request, seq 0
# P2 to P3
IP6 (hlim 59) 2001:db8:100::1 > fcbb:bb00:3:e005:6::: [icmp6 sum ok] ICMP6, echo request, seq 0
# P3 to P5
IP6 (hlim 58) 2001:db8:100::1 > fcbb:bb00:6::: [bad icmp6 cksum 0x0316 -> 0xe31e!] ICMP6, echo request, seq 0
# P5 to P4
IP6 (hlim 57) 2001:db8:100::1 > fcbb:bb00:6::: [bad icmp6 cksum 0x0316 -> 0xe31e!] ICMP6, echo request, seq 0
# P4 to PE6
IP6 (hlim 56) 2001:db8:100::1 > fcbb:bb00:6::: [bad icmp6 cksum 0x0316 -> 0xe31e!] ICMP6, echo request, seq 0
```

- Now we read the captures. We save each capture from CML, and we read it with tcpdump on our own computer. We show the first echo request of each file.
- On the link PE1 to P2, the destination holds all three instructions. The hop limit is 60. tcpdump says that the checksum is correct.
- On the link P2 to P3, the destination is the same. P2 has no matching SID. It routes the packet, and it lowers the hop limit by one.
- After P3, the destination is fcbb:bb00:6::. P3 removed its node instruction and the uA, and it shifted PE6 to the front. Now tcpdump reports a bad checksum. Remember this.
- P5 and P4 route the packet like a normal address. The packet arrives at PE6 with hop limit 56, after four routers. Each change agrees with the prediction.

Ep 4.5 · The shifted ping from PE1

Source: lab-evidence/05-usid-shift/checksum (direct-PE1.log, shifted-PE1.log)

```
RP/0/RP0/CPU0:PE1#ping fcbb:bb00:6:: source 2001:db8:100::1 count 3
Sending 3, 100-byte ICMP Echos to fcbb:bb00:6::, timeout is 2 seconds:
!!!
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/3/3 ms

RP/0/RP0/CPU0:PE1#ping fcbb:bb00:2:4:6:: source 2001:db8:100::1 count 3
Sending 3, 100-byte ICMP Echos to fcbb:bb00:2:4:6::, timeout is 2 seconds:
...
Success rate is 0 percent (0/3)
```

- The capture showed a strange result. The shifted packet arrived at PE6, but PE6 did not reply. We compare two pings from PE1.
- First, a ping directly to the node SID of PE6. All three packets get a reply.
- Second, a ping to fcbb:bb00:2:4:6::. This address holds three instructions: P2, then P4, then PE6. It gets no reply. The cause is on PE6.

Ep 4.6 · PE6 counters after each ping

Router / tool: PE6 · Source: lab-evidence/05-usid-shift/checksum

```
RP/0/RP0/CPU0:PE6#show ipv6 traffic      (ICMP statistics, three times)
before the pings
  Rcvd: 23 input, 6 checksum errors, 0 too short
        13 echo request, 0 echo reply
after 3 direct pings
  Rcvd: 26 input, 6 checksum errors, 0 too short
        16 echo request, 0 echo reply
after 3 shifted pings
  Rcvd: 29 input, 9 checksum errors, 0 too short
        16 echo request, 0 echo reply
```

- On PE6, we read the ICMP statistics before the pings, and after each ping. The command is show ipv6 traffic.
- This is the starting point: 6 checksum errors, and 13 echo requests.
- After three direct pings, the echo request counter increased by three, to 16. The checksum error counter did not change.
- After three shifted pings, the checksum error counter increased by three, to 9. The echo request counter did not change. PE6 discarded the packets as damaged.

Ep 4.7 · Why PE6 discards the packet

Router / tool: analysis · Source: lab-evidence/05-usid-shift/checksum/checksum-analysis.txt

```
# ICMPv6 checksum, calculated again from the capture on P4 to PE6
direct: arriving DA fcbb:bb00:6::, checksum in packet 0xfd85
   computed with the arriving DA                    0xfd85  match
   computed with the sender's DA fcbb:bb00:6::      0xfd85  match
shifted: arriving DA fcbb:bb00:6::, checksum in packet 0xfd34
   computed with the arriving DA                    0xfd3a  MISMATCH
   computed with the sender's DA fcbb:bb00:2:4:6::  0xfd34  match
```

- We calculated the ICMPv6 checksum again from the capture of the link P4 to PE6.
- For the direct ping, the packet arrives with the same address that PE1 sent to. The checksum matches.
- For the shifted ping, the arriving destination gives 0xfd3a. The packet carries 0xfd34. They do not match.
- The destination that PE1 sent to, fcbb:bb00:2:4:6::, gives 0xfd34. This matches. PE1 calculated the checksum with its own destination.
- RFC 9800, section 6.5, says that the source must calculate the checksum with the final destination: the address of the packet when it arrives. Our ping used the container as a normal address.
- This does not affect customer traffic, because its checksum is in the inner packet. The lesson is for tests: use captures, not ping replies, to examine a shifted path.

Ep 4.8 · PSP: the policy that PE1 uses

Source: lab-evidence/06-srh/v3/PE1.log

```
RP/0/RP0/CPU0:PE1#show cef vrf BLUE ipv6 2001:db8:c:106::/64 detail
2001:db8:c:106::/64, version 22, SRv6 Headend
   via fcbb:bb00:1:e009::/128, 3 dependencies, recursive
    SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e005::}
      SRv6 H.Insert.Red SID-list {fcbb:bb01:3:5::}
    Hash  OK  Interface                 Address
    0     Y   GigabitEthernet0/0/0/1    fe80::5454:ff:fec1:58eb
```

- Now PSP, in the lab. The route to the BLUE customer in IPv6 uses an SR policy. Episode 6 explains the policy. Here we only read the forwarding entry.
- The first SID list is the service SID of PE6, fcbb:bb00:6:e005::. PE1 puts it in the routing header.
- The second SID list is the policy. It is one container, fcbb:bb01:3:5::, with two instructions: P3, then P5. PE1 puts it in the destination address.
- The packet needs both lists, so it needs a routing header. Now we ping from CE-A, and we capture on the links.

Ep 4.9 · PSP on the wire

Router / tool: tcpdump · Source: lab-evidence/06-srh/v3/c300-PE1-P3, c300-P3-P5, c300-P4-P5, c300-P4-PE6

```
# CE-A: sudo ping6 -c 2 2001:db8:c:106::2   (2 received, 0% packet loss)
# PE1 to P3, 182 bytes
IP6 (hlim 255, next-header Routing (43)) 2001:db8:100::1 > fcbb:bb01:3:5::: RT6 (segleft=1, [0]fcbb:bb00:6:e005::)
# P3 to P5, 182 bytes
IP6 (hlim 254, next-header Routing (43)) 2001:db8:100::1 > fcbb:bb01:5::: RT6 (segleft=1, [0]fcbb:bb00:6:e005::)
# P5 to P4, 158 bytes
IP6 (hlim 253, next-header IPv6 (41)) 2001:db8:100::1 > fcbb:bb00:6:e005:::
# P4 to PE6, 158 bytes
IP6 (hlim 252, next-header IPv6 (41)) 2001:db8:100::1 > fcbb:bb00:6:e005:::
```

- In the lab, we start captures on the links of the path. Then, on CE-A, we ping the customer address on the far side. Both replies return.
- On the link PE1 to P3, the destination is the container fcbb:bb01:3:5::. The routing header has one entry: the service SID of PE6. Segments Left is 1. The frame has 182 bytes.
- After P3, the destination is fcbb:bb01:5::. P3 shifted out its own instruction. The routing header is still there, with one segment left.
- P5 is the penultimate endpoint. It copies the last SID into the destination, and it removes the routing header. This is PSP. The frame is 24 bytes smaller: 158 bytes.
- On the link P4 to PE6, there is no routing header. PE6 receives a normal encapsulated packet, and the service SID e005 does the rest.

Ep 4.10 · uDT4: a service decapsulates by itself

Router / tool: PE6 · Source: lab-evidence/03-faults/v3wire/red-PE6.log

```
RP/0/RP0/CPU0:PE6#show cef ipv6 fcbb:bb00:6:e006::/64
fcbb:bb00:6:e006::/64, version 179, SRv6 Endpoint uDT4
    next hop VRF - 'RED', table - 0xe0000001
    0     Y   recursive                Lookup in table
```

- Do not mix up USD and service decapsulation.
- A uDT4 is a different behavior. It does not need a flavor to remove the outer header. To remove the outer header is its function.
- Then it looks up the inner packet in a specified VRF, here RED. Thus it is a service, not a waypoint.

Ep 5.1 · Test 1: ping and capture

Source: lab-evidence/05-usid-shift/v3/PE1.log and the first echo request in shift-*.pcap

```
RP/0/RP0/CPU0:PE1#ping fcbb:bb00:2:4:6:: source 2001:db8:100::1 count 1
Type escape sequence to abort.
Sending 1, 100-byte ICMP Echos to fcbb:bb00:2:4:6::, timeout is 2 seconds:
.
Success rate is 0 percent (0/1)

$ tcpdump -nn -t -v -r shift-PE1-P2.pcap icmp6
IP6 (hlim 60) 2001:db8:100::1 > fcbb:bb00:2:4:6:::  [icmp6 sum ok] ICMP6, echo request
$ tcpdump -nn -t -v -r shift-P2-P4.pcap icmp6
IP6 (hlim 59) 2001:db8:100::1 > fcbb:bb00:4:6:::  [bad icmp6 cksum 0xde68 -> 0xde6a!] ICMP6, echo request
$ tcpdump -nn -t -v -r shift-P4-PE6.pcap icmp6
IP6 (hlim 58) 2001:db8:100::1 > fcbb:bb00:6:::  [bad icmp6 cksum 0xde68 -> 0xde6e!] ICMP6, echo request
```

- Now we run the first test. In CML, start a packet capture on each core link. Then ping the test address from PE1.
- The destination is fcbb:bb00:2:4:6::. It holds three instructions: P2, then P4, then PE6.
- The ping gets no reply. Episode 4 showed the reason: the checksum. Thus we read the path from the captures.
- On the link PE1 to P2, the destination is the full address. The hop limit is 60.
- On the link P2 to P4, P2 shifted the address. The destination is fcbb:bb00:4:6::, and the hop limit is 59. tcpdump now reports a bad checksum.
- On the link P4 to PE6, the destination is fcbb:bb00:6::, with hop limit 58. All eight links agree with the table, and no other link carried the packet.

Ep 5.2 · Why normal VPN traffic looks simpler

Source: lab-evidence/03-faults/v3wire/healthy2-PE1-P2.pcap

```
Healthy GREEN customer ping, captured on PE1 to P2:
  154 bytes   outer destination fcbb:bb00:6:e008::   inner 10.30.1.1 to 10.30.6.6

Shortest path PE1 to PE6:  PE1, P2, P4, PE6   (cost 30)
```

- Why did the VPN traffic in Episode 3 show no shift?
- The customer packets had only one instruction: PE6's service SID. P2 and P4 had nothing to do.
- The shortest path already goes through P2 and P4, so it is not necessary to name them. A headend adds waypoints only for a different path. Thus, in this episode, we make test addresses manually.

Ep 5.3 · Test 2: ping the waypoints

Source: lab-evidence/05-usid-shift/v3/PE1.log

```
RP/0/RP0/CPU0:PE1#ping fcbb:bb00:3:5:6:: source 2001:db8:100::1 count 1
Type escape sequence to abort.
Sending 1, 100-byte ICMP Echos to fcbb:bb00:3:5:6::, timeout is 2 seconds:
.
Success rate is 0 percent (0/1)
```

- Test 2 is the ping to fcbb:bb00:3:5:6::. Start the captures on all eight core links first.
- The destination has three instructions: P3, then P5, then PE6.
- Again, there is no reply. We read the path from the captures, not from the ping.

Ep 5.4 · The capture for fcbb:bb00:3:5:6::

Source: lab-evidence/05-usid-shift/v3/shift-*.pcap, one ping, read in time order

```
$ tcpdump -nn -t -v -r shift-<link>.pcap icmp6     (eight files, merged in time order)
link        destination            hop limit
PE1 - P2    fcbb:bb00:3:5:6::      60
P2 - P3     fcbb:bb00:3:5:6::      59     toward P3
P3 - P2     fcbb:bb00:5:6::        58     P3 shifted, sends it back
P2 - P4     fcbb:bb00:5:6::        57
P4 - P5     fcbb:bb00:5:6::        56     toward P5
P5 - P4     fcbb:bb00:6::          55     P5 shifted, sends it back
P4 - PE6    fcbb:bb00:6::          54

PE1 - P3, P3 - P5, P5 - PE6:  nothing
```

- This is one ping. We read the capture of each core link with tcpdump, and we put the results in time sequence.
- PE1 to P2, then P2 to P3, with no change. P2 had no instruction here, so it only routed the packet.
- P3 shifts and sends the packet back to P2, then to P4.
- P4 forwards the packet to P5. P5 shifts and sends it back to P4.
- P4 sends it to PE6 with hop limit 54. The packet used seven links. The shortest path has three links.
- The three lower links carried nothing. The prediction is correct on each link.

Ep 5.5 · A full container

Source: lab-evidence/05-usid-shift/v3/shift-*.pcap, one ping, read in time order

```
RP/0/RP0/CPU0:PE1#ping fcbb:bb00:2:3:5:4:6:: source 2001:db8:100::1 count 1
Sending 1, 100-byte ICMP Echos to fcbb:bb00:2:3:5:4:6:0, timeout is 2 seconds:
.
Success rate is 0 percent (0/1)

fcbb:bb00:0002:0003:0005:0004:0006:0000    five instructions, one zero group

link        destination             hop limit
PE1 - P2    fcbb:bb00:2:3:5:4:6:0    60
P2 - P3     fcbb:bb00:3:5:4:6::      59     P2 executed its own instruction
P3 - P2     fcbb:bb00:5:4:6::        58
P2 - P4     fcbb:bb00:5:4:6::        57
P4 - P5     fcbb:bb00:5:4:6::        56
P5 - P4     fcbb:bb00:4:6::          55
P4 - PE6    fcbb:bb00:6::            54
```

- How many instructions can one address contain? We fill one address.
- Test 3 is the ping to fcbb:bb00:2:3:5:4:6::. The router shows the address with the last zero group, :0. The ping gets no reply, as before.
- After the 32-bit block, there are six 16-bit positions. We put five node SIDs in them: P2, P3, P5, P4 and PE6. One zero group shows the end.
- This time, P2 is first. Thus P2 does its instruction and shifts the address before it sends the packet to P3.
- Then the packet uses the same path as before, through P3, P2, P4, P5 and P4 again.
- PE6 receives fcbb:bb00:6:: with hop limit 54. Five instructions are in one address, and there is no routing header.

Ep 5.6 · Test 4: ping with a service SID

Source: lab-evidence/05-usid-shift/v3/PE1.log

```
RP/0/RP0/CPU0:PE1#ping fcbb:bb00:2:4:6:e008:: source 2001:db8:100::1 count 1
Type escape sequence to abort.
Sending 1, 100-byte ICMP Echos to fcbb:bb00:2:4:6:e008::, timeout is 2 seconds:
.
Success rate is 0 percent (0/1)
```

- Test 4 puts a service function at the end of the list. The ping is to fcbb:bb00:2:4:6:e008::.
- The address has four instructions: P2, P4, PE6, and the service function e008 of PE6.
- There is no reply, as in the other tests. The captures show the shifts.

Ep 6.1 · Run the pings

Router / tool: PE1 and CE-A · Source: lab-evidence/06-srh/v3/green.log, CE-A.log

```
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 2
!!
Success rate is 100 percent (2/2), round-trip min/avg/max = 2/2/3 ms

CE-A:~$ sudo ping6 -c 2 2001:db8:c:106::2
64 bytes from 2001:db8:c:106::2: seq=0 ttl=62 time=7.698 ms
2 packets transmitted, 2 packets received, 0% packet loss

CE-A:~$ sudo ping6 -c 2 -s 1000 2001:db8:c:106::2
1008 bytes from 2001:db8:c:106::2: seq=0 ttl=62 time=2.140 ms
2 packets transmitted, 2 packets received, 0% packet loss
```

- Three pings produce the frames in this episode. In CML, first start a capture on every core link, and on the link between CE-A and PE1.
- First, a GREEN ping from PE1. The source is the GREEN interface of PE1. Both packets succeed.
- Second, a BLUE IPv6 ping from the host CE-A to the far side of the BLUE customer. The default ping packet has 56 data bytes.
- Third, the same ping with 1000 data bytes. We use the two sizes to measure how the frame size changes. Both pings succeed.

Ep 6.2 · Configure the policy on PE1

```
! PE1 (trimmed)
segment-routing
 traffic-eng
  segment-lists
   srv6
    sid-format usid-f3216
   segment-list TWO-BLOCKS
    srv6
     index 10 sid fcbb:bb01:3::
     index 20 sid fcbb:bb01:5::
     index 30 sid fcbb:bb00:6::
  policy BLUE6-C300
   srv6
    binding-sid dynamic behavior ub6-insert-reduced locator MAIN
   color 300 end-point ipv6 2001:db8:100::6
   candidate-paths
    preference 100
     explicit segment-list TWO-BLOCKS
```

- To make condition 2 on purpose, we configure one explicit policy on PE1. Episode 15 explains each command.
- First, a segment list that uses the micro-SID format f3216. Its name is TWO-BLOCKS.
- The first two SIDs, P3 and P5, are from the block fcbb:bb01. They are Flex-Algo SIDs.
- The last SID, PE6, is from the block fcbb:bb00. It is a normal SID.
- The policy BLUE6-C300 has color 300 and the end point PE6. Its Binding SID is dynamic, from the locator MAIN.
- The only candidate path uses the segment list TWO-BLOCKS.

Ep 6.3 · Color the BLUE IPv6 routes on PE6

```
! PE6 (trimmed)
extcommunity-set opaque COLOR-300
 300
end-set
route-policy BLUE6-EXPORT-C300
 set extcommunity color COLOR-300
end-policy
vrf BLUE
 address-family ipv6 unicast
  export route-policy BLUE6-EXPORT-C300
```

- The policy needs traffic. PE6 adds the color 300 to the BLUE IPv6 routes that it advertises. PE1 then matches the color and the end point with the policy.
- First, an extended community with the value 300.
- Second, a route policy that sets this color.
- Third, PE6 applies the route policy when it exports the BLUE IPv6 routes.

Ep 6.4 · A policy that mixes two blocks

Source: lab-evidence/06-srh/v3/PE1.log

```
RP/0/RP0/CPU0:PE1#show segment-routing traffic-eng policy color 300 detail
Color: 300, End-point: 2001:db8:100::6
    Admin: up  Operational: up
      Explicit: segment-list TWO-BLOCKS (valid)
          SID[0]: fcbb:bb01:3::/48
          SID[1]: fcbb:bb01:5::/48
          SID[2]: fcbb:bb00:6::/48
      Binding SID: fcbb:bb00:1:e009::
```

- Now we check the policy on PE1. BLUE IPv6 traffic to PE6 goes into this policy.
- The first two waypoints, P3 and P5, are Flex-Algo SIDs from the fcbb:bb01 block.
- The last waypoint, PE6, is a normal SID from fcbb:bb00.
- The policy is up, and it has a Binding SID. There are two blocks, so there must be at least two containers.

Ep 6.5 · What PE1 programs

Source: lab-evidence/06-srh/v3/PE1.log

```
RP/0/RP0/CPU0:PE1#show cef vrf BLUE ipv6 2001:db8:c:106::/64 detail
    SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e005::}
      SRv6 H.Insert.Red SID-list {fcbb:bb01:3:5::}
```

- Now the forwarding entry for the BLUE destination. It has two important lines.
- Line 1: H.Encaps.Red to PE6's service SID, e005. This is the VPN part. PE1 encapsulates the customer packet and sends it to the BLUE service on PE6.
- Line 2: H.Insert.Red with the path container fcbb:bb01:3:5::. This is the policy. PE1 inserts the path in front of the service SID.
- The third SID of the policy, PE6's node SID, is not there. The service SID already ends at PE6, so the router does not use it.

Ep 6.6 · The SRH, field by field

Router / tool: capture · Source: lab-evidence/06-srh/v3/srh-decode.txt, from c300-PE1-P3.pcap

```
$ tshark -r c300-PE1-P3.pcap -Y "ipv6.nxt == 43" -V     (first packet, trimmed)
Internet Protocol Version 6, Src: 2001:db8:100::1, Dst: fcbb:bb01:3:5::
    Payload Length: 128
    Next Header: Routing Header for IPv6 (43)
    Hop Limit: 255
    Routing Header for IPv6 (Segment Routing)
        Next Header: IPv6 (41)
        Length: 2
        [Length: 24 bytes]
        Type: Segment Routing (4)
        Segments Left: 1
        Last Entry: 0
        Address[0]: fcbb:bb00:6:e005::
Internet Protocol Version 6, Src: 2001:db8:c:101:5054:ff:fe0b:8f87, Dst: 2001:db8:c:106::2
```

- Now we decode one of the 182-byte packets from the capture of the PE1 to P3 link. We use tshark with the verbose option.
- The outer destination is the path container: P3, then P5, in the Flex-Algo block. Next header 43 means that a routing header follows.
- This is the SRH. Type 4 means segment routing. Its next header is 41, an IPv6 packet: the customer packet. Its length is 24 bytes.
- It has one entry: the service SID. Segments Left is 1. One more SID is in the list after the current destination.
- Then there is the customer IPv6 header, with no change.

Ep 6.7 · Segments Left along the path

Router / tool: tcpdump · Source: lab-evidence/06-srh/v3/c300-PE1-P3, c300-P3-P5, c300-P4-P5, c300-P4-PE6

```
$ tcpdump -nn -t -v -r <capture file> 'ip6[6] == 43 or ip6[6] == 41'   (first packet, trimmed)
# PE1 to P3, 182 bytes
IP6 (hlim 255, next-header Routing (43)) 2001:db8:100::1 > fcbb:bb01:3:5::: RT6 (segleft=1, [0]fcbb:bb00:6:e005::)
# P3 to P5, 182 bytes
IP6 (hlim 254, next-header Routing (43)) 2001:db8:100::1 > fcbb:bb01:5::: RT6 (segleft=1, [0]fcbb:bb00:6:e005::)
# P5 to P4, 158 bytes
IP6 (hlim 253, next-header IPv6 (41)) 2001:db8:100::1 > fcbb:bb00:6:e005:::
# P4 to PE6, 158 bytes
IP6 (hlim 252, next-header IPv6 (41)) 2001:db8:100::1 > fcbb:bb00:6:e005:::
```

- Follow the Segments Left field on the path. We read the capture of each link with tcpdump. The RT6 part is the routing header.
- When the packet leaves PE1, the destination is the container fcbb:bb01:3:5::. Segments Left is 1, and the entry is the service SID of PE6.
- After P3, the destination has shifted to fcbb:bb01:5::, and Segments Left is still 1. A shift in a container does not change the SRH.
- At P5, the container has no more micro-SIDs. P5 decreases Segments Left to zero, and it copies the next SID into the destination. P5 is the penultimate endpoint, thus it also removes the SRH.
- The next header is now 41, and the frame is 24 bytes smaller. This is step 2 of the procedure from Episode 4, on the wire.

Ep 6.8 · The BLUE packet, measured

Router / tool: tshark · Source: lab-evidence/06-srh/v3/c300-PE1-CE-A, c300-PE1-P3, c300-P3-P5, c300-P4-P5, c300-P4-PE6 (summary)

```
$ tshark -r <capture file> -Y "icmpv6.type == 128 and icmpv6.echo.sequence_number == 0" -T fields -e frame.len
link                  small  large  what is added
CE-A to PE1           118    1062   nothing: the customer packet
PE1 to P3, P3 to P5   182    1126   + 40 outer IPv6 + 24 SRH = 64
P5 to P4, P4 to PE6   158    1102   + 40 outer IPv6 only
```

- Now we count each byte. For each capture file, tshark prints the frame length of the echo requests. The small ping and the large ping give two values.
- On the access link, the frames are 118 and 1062 bytes. This is the customer packet.
- From PE1 to P5, each frame is 64 bytes larger: 40 for the outer IPv6 header and 24 for the SRH.
- After P5, each frame is only 40 bytes larger. P5 was the penultimate endpoint, and PSP removed the SRH, as Episode 4 showed.
- Thus the overhead of the same flow changes on the path: 64 bytes in the first part, 40 bytes in the second part.

Ep 6.9 · Why our lab never shows 1596

Source: lab-evidence/01-baseline/PE1.log

```
RP/0/RP0/CPU0:PE1#show segment-routing srv6 manager
  Headend behaviors:
    T
    H.Insert.Red
    H.Encaps.Red
    H.Encaps.L2.Red
  Signaled parameters:
    Max-H-Insert    : 3 sids
    Max-H-Encap     : 3 sids
```

- Our captures do not show a 1596-byte packet. This is the reason.
- This headend has only the reduced forms. There is no plain H.Encaps, so the first SID is never in the SRH also. In this series, the 1596 case is only a calculation.
- It can insert or encapsulate a maximum of 3 SIDs. Each SID can be a full container of micro-SIDs. Thus 3 is a limit on containers, not on instructions.

Ep 7.1 · Case 1: what PE1 programs

Source: lab-evidence/07-mtu/v3/A-PE1-cef.log

```
RP/0/RP0/CPU0:PE1#show cef vrf RED 10.2.6.0/24 detail | include SID-list
    SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e006::}
      SRv6 H.Insert.Red SID-list {fcbb:bb00:e004::}
```

- Case 1 uses IPv4 in VRF RED. First, we check what PE1 programs for the RED prefix 10.2.6.0/24.
- The service SID is fcbb:bb00:6:e006::. It is the uDT4 of RED on PE6.
- The path is one container, fcbb:bb00:e004::. PE1 inserts it in front of the service SID.
- Each list has one SID, so the packet has no routing header. The overhead is 40 bytes, and the limit is 1460.

Ep 7.2 · Case 1: the pings

Router / tool: CE-A · Source: lab-evidence/07-mtu/v3/A-CE-A.log, A2-CE-A.log

```
CE-A:~$ sudo ping -c 1 -I 10.2.1.2 -s 1432 10.2.6.2
1440 bytes from 10.2.6.2: seq=0 ttl=62 time=1.495 ms
1 packets transmitted, 1 packets received, 0% packet loss

CE-A:~$ sudo ping -c 3 -I 10.2.1.2 -s 1433 10.2.6.2
3 packets transmitted, 0 packets received, 100% packet loss

CE-A:~$ sudo cat /proc/sys/net/ipv4/ip_no_pmtu_disc
0
```

- Now CE-A sends IPv4 pings to the RED far side. The option -s is the data size, and -I sets the source address.
- A data size of 1432 makes a 1460-byte IPv4 packet. This is the limit, and the reply returns.
- A data size of 1433 makes a 1461-byte packet. All three pings are lost. The packet loss is 100 percent.
- The Linux host sets the don't-fragment bit, because this kernel setting is 0.

Ep 7.3 · Case 1: the captures

Router / tool: tshark · Source: lab-evidence/07-mtu/v3/A-PE1-Gi0_0_0_3-CE-A, A-PE1-P3, A2-*

```
$ tshark -r <capture file> -Y icmp
# PE1 to CE-A, the 1460-byte ping
1474 bytes  10.2.1.2 > 10.2.6.2  echo request
1474 bytes  10.2.6.2 > 10.2.1.2  echo reply
# PE1 to CE-A, the 1461-byte ping
1475 bytes  10.2.1.2 > 10.2.6.2  echo request   (no reply follows)
# PE1 to P3, the 1460-byte ping
1514 bytes  10.2.1.2 > 10.2.6.2  echo request, inside the outer IPv6 header
# PE1 to P2, PE1 to P3, P4 to PE6, P5 to PE6, the 1461-byte ping
no echo request on any core link
```

- We captured on the access link and on the core links, and we read each file with tshark. The frame length includes 14 bytes of Ethernet header.
- On the access link, the 1460-byte packet is a 1474-byte frame. The reply returns.
- The 1461-byte packet is a 1475-byte frame. It arrives at PE1, and no reply follows.
- The 1460-byte packet crosses the core in a 1514-byte frame. This is the largest frame that the link carries.
- For the 1461-byte packet, no core link carries an echo request. PE1 dropped it before the encapsulation.

Ep 7.4 · Case 2: the pings

Router / tool: CE-A · Source: lab-evidence/07-mtu/v3/B-CE-A.log

```
CE-A:~$ sudo ping6 -c 1 -s 1388 2001:db8:c:106::2
1396 bytes from 2001:db8:c:106::2: seq=0 ttl=62 time=1.545 ms
1 packets transmitted, 1 packets received, 0% packet loss

CE-A:~$ sudo ping6 -c 2 -s 1389 2001:db8:c:106::2
1397 bytes from 2001:db8:c:106::2: seq=1 ttl=62 time=1.566 ms
2 packets transmitted, 1 packets received, 50% packet loss
```

- Case 2 uses IPv6 in VRF BLUE, through colour 300. The limit is 1436. Linux ping6 adds 48 bytes to the data size.
- A data size of 1388 makes a 1436-byte packet. The reply returns.
- A data size of 1389 makes a 1437-byte packet. The first packet is lost, and the second packet gets a reply. The loss is 50 percent.

Ep 7.5 · Case 2: the captures

Router / tool: tshark · Source: lab-evidence/07-mtu/v3/B-PE1-CE-A.pcap

```
$ tshark -r B-PE1-CE-A.pcap -Y icmpv6      (CE-A is 2001:db8:c:101:5054:ff:fe0b:8f87)
1450 bytes  CE-A > 2001:db8:c:106::2  echo request, seq 0
1450 bytes  2001:db8:c:106::2 > CE-A  echo reply, seq 0
1451 bytes  CE-A > 2001:db8:c:106::2  echo request, the 1437-byte packet
1294 bytes  2001:db8:100::1 > CE-A    Packet Too Big, MTU 1436
1446 and 75 bytes  CE-A > 2001:db8:c:106::2  the second request, in two fragments
1451 bytes  2001:db8:c:106::2 > CE-A  echo reply, seq 1
```

- On the access link, we read the packets with tshark. CE-A is the address that the first line shows.
- The 1436-byte packet is a 1450-byte frame. The request and the reply both pass.
- The 1437-byte packet is a 1451-byte frame. This is the first try.
- PE1 answers with an ICMPv6 Packet Too Big. The source is the loopback of PE1. The MTU is 1436, which is our budget.
- CE-A sends the second request in two fragments, and the reply returns. IPv6 path MTU discovery worked.

Ep 7.6 · Case 2: the long way home

Router / tool: tshark · Source: lab-evidence/07-mtu/v3/B-PE1-P2.pcap, B-P4-PE6.pcap, B-PE1-CE-A.pcap

```
$ tshark -r <capture file> -Y icmpv6.type==2      (outer header, trimmed)
# PE1 to P2, 1358 bytes
2001:db8:100::1 > fcbb:bb00:6:e005::   hop limit 255   Packet Too Big, toward PE6
# P4 to PE6, 1358 bytes
2001:db8:100::1 > fcbb:bb00:6:e005::   hop limit 253   the same message
# P4 to PE6, 1334 bytes
2001:db8:100::6 > fcbb:bb00:1:e006::   hop limit 255   the message, back toward PE1
# PE1 to P2, 1334 bytes
2001:db8:100::6 > fcbb:bb00:1:e006::   hop limit 253   the same message
# PE1 to CE-A, 1294 bytes
2001:db8:100::1 > CE-A   Packet Too Big, delivered
```

- Now look at the path of that Packet Too Big. PE1 made it for a host on its own access link. You expect it to go directly to the host. It did not.
- PE1 encapsulated it to the BLUE service SID of PE6, and it left PE1 toward P2.
- The same message crosses the core and arrives on the link P4 to PE6. The hop limit is 253.
- PE6 decapsulated it, looked up CE-A's address in BLUE, and sent it back to the service SID of PE1.
- It crosses the core again, and it arrives on the link PE1 to P2.
- Then PE1 sent it to CE-A. It worked, but it went through the core two times. We did not verify why the ingress makes the message this way. It probably depends on the platform.

Ep 7.7 · Case 3: a smaller link in the middle

```
! P4, the link toward P2
interface GigabitEthernet0/0/0/0
 ipv6 mtu 1300
!
! RED traffic from CE-B to CE-A crosses P4 to P2
! inner 1260 + outer 40 = 1300   fits
! inner 1261 + outer 40 = 1301   does not fit
```

- Case 3. The ingress has no problem, but a link in the middle is too small. On P4, we set the IPv6 MTU to P2 to 1300.
- This time, the traffic goes in the opposite direction, from CE-B to CE-A in RED. That direction uses the upper path through P4 and P2.
- An inner packet of 1260 bytes becomes 1300 bytes. It fits.
- At 1261 bytes, it does not fit, and P4 must decide what to do. P4 is a P router. It has no VRF, and it does not know CE-B.

Ep 7.8 · Case 3: the pings

Router / tool: CE-B · Source: lab-evidence/07-mtu/v3/C-CE-B.log

```
CE-B:~$ sudo ping -c 1 -I 10.2.6.2 -s 1232 10.2.1.2
1240 bytes from 10.2.1.2: seq=0 ttl=62 time=1.796 ms
1 packets transmitted, 1 packets received, 0% packet loss

CE-B:~$ sudo ping -c 2 -I 10.2.6.2 -s 1233 10.2.1.2
1241 bytes from 10.2.1.2: seq=1 ttl=62 time=1.558 ms
2 packets transmitted, 1 packets received, 50% packet loss
```

- Now CE-B sends IPv4 pings to CE-A in RED. This direction crosses P4 and P2.
- A data size of 1232 makes a 1260-byte packet. It fits, and the reply returns.
- A data size of 1233 makes a 1261-byte packet. The first packet is lost. The second packet gets a reply. The loss is 50 percent.

Ep 7.9 · Case 3: the captures

Router / tool: tshark · Source: lab-evidence/07-mtu/v3/C-PE6-Gi0_0_0_3-CE-B, C-P2-P4, C-PE1-P2, C-P4-PE6

```
$ tshark -r C-PE6-Gi0_0_0_3-CE-B.pcap -Y icmp      (PE6 to CE-B)
1275 bytes  10.2.6.2 > 10.2.1.2  echo request, the 1261-byte packet
110 bytes   10.0.0.4 > 10.2.6.2  destination unreachable, fragmentation needed, MTU 1236
1250 and 59 bytes  10.2.6.2 > 10.2.1.2  the second request, in two fragments
1275 bytes  10.2.1.2 > 10.2.6.2  echo reply

$ tshark -r <capture file> -Y icmp.type==3      (the error in the core, outer header)
P2 to P4   190 bytes  2001:db8:100::6 > fcbb:bb00:1:e005::  hop limit 254
PE1 to P2  190 bytes  2001:db8:100::6 > fcbb:bb00:1:e005::  hop limit 253
P4 to PE6  150 bytes  2001:db8:100::1 > fcbb:bb00:6:e006::  hop limit 252
```

- We captured on five links, and we read the files with tshark. First, the access link of CE-B, on PE6.
- The 1261-byte request leaves CE-B. This is the first try.
- An ICMPv4 error comes back to CE-B: fragmentation needed, with an MTU of 1236. Its source is 10.0.0.4, the IPv4 loopback of P4.
- Now the core. P4 sent the error in the same outer header as the dropped packet. It goes from P4 through P2 toward PE1, to the service SID of PE1.
- PE1 decapsulated it, found CE-B in RED, and sent it back through the core to the service SID of PE6.
- CE-B learns the MTU, and it sends the second request in two fragments, 1250 and 59 bytes. The reply returns.

Ep 8.1 · Where the LSP content comes from

```
! PE6 (trimmed)
router isis CORE
 address-family ipv6 unicast
  metric-style wide
  segment-routing srv6
   locator MAIN
   locator FA128
 flex-algo 128
  metric-type delay
  advertise-definition
 interface GigabitEthernet0/0/0/0
  point-to-point
  address-family ipv6 unicast
   fast-reroute per-prefix ti-lfa
   metric 10
performance-measurement
 interface GigabitEthernet0/0/0/0
  delay-measurement
   advertise-delay 5000
```

- Before we read the LSP, we look at the configuration that creates it. This is PE6, trimmed to the lines that matter. You can copy the same structure on each router.
- Wide metrics are necessary for segment routing. The IPv6 address family carries everything.
- Two locators are attached to IS-IS: MAIN for algorithm 0, and FA128 for Flex-Algo 128. Each one becomes a locator in the LSP.
- The flex-algo 128 block gives the metric type, delay, and it advertises the definition. We use it in Episode 11.
- Each core interface is point-to-point, with a metric. The metric is the link metric that we read in the LSP.
- The link delay of 5000 microseconds comes from the performance-measurement block. It is a static value that we configured, not a measurement.

Ep 8.2 · Layer 1: the adjacency

Source: lab-evidence/08-isis/rerecord/PE1-healthy.log

```
RP/0/RP0/CPU0:PE1#show isis adjacency detail
P2             Gi0/0/0/0               *PtoP*         Up    25   06:50:33 Yes None Up
  Neighbor IPv6 Address:  fe80::5454:ff:fe66:75e1*
  END.X SID:              fcbb:bb00:1:e000::/64 uA (PSP/USD), Algorithm 0
  Non-FRR END.X SID:      fcbb:bb00:1:e001::/64 uA (PSP/USD), Algorithm 0
  END.X SID:              fcbb:bb01:1:e000::/64 uA (PSP/USD), Algorithm 128
  Non-FRR END.X SID:      fcbb:bb01:1:e001::/64 uA (PSP/USD), Algorithm 128
  Topology:               IPv6 Unicast
    Usable by Flex-algo:  128
  BFD Status:             Neighbor Useable (BFD Required)
```

- Layer 1, on PE1, for the neighbor P2.
- The adjacency is up. It uses IPv6 link-local addresses. There is no IPv4 here.
- IS-IS allocated two adjacency SIDs for this link in algorithm 0. Fast reroute can protect one of them. The SID with the Non-FRR mark is never protected.
- There are two more for algorithm 128, the Flex-Algo of Episode 11. One link has four adjacency SIDs.
- The topology is IPv6 unicast, and Flex-Algo 128 can use it. BFD is necessary, and it is up. Each of these lines can be the cause of a missing path.

Ep 8.3 · Layer 2: the LSP, capabilities

Source: lab-evidence/08-isis/rerecord/PE1-healthy.log

```
RP/0/RP0/CPU0:PE1#show isis database verbose PE6.00-00
PE6.00-00             0x00000034   0x359c        1145 /1200         0/0/0
  IPv6 Router ID: 2001:db8:100::6
  Router Cap:     0.0.0.0 D:0 S:0
    SR Algorithm:
      Algorithm: 0
      Algorithm: 1
      Algorithm: 128
    SRv6: O:0
    Node Maximum SID Depth:
      SRH Max SL:       3
      SRH Max End Pop:  3
      SRH Max T.insert: 3
      SRH Max T.encaps: 3
      SRH Max End D:    4
```

- Layer 2 is PE6's link-state packet on PE1. We read it in three parts. Use verbose. The detail option does not show most of the information.
- This is PE6's LSP, with a sequence number, a remaining lifetime and an IPv6 router ID.
- PE6 gives the algorithms that it uses: 0 and 1, which are the defaults, and 128, which we configured.
- RFC 9352 defines this SRv6 capability flag. It tells that PE6 can process SRv6.
- These are the maximum SID depths. They are the same values that the manager showed in Episode 2. Each router learns the limits of the other routers from here.

Ep 8.4 · Layer 2: the LSP, locators

Source: lab-evidence/08-isis/rerecord/PE1-healthy.log

```
SRv6 Locator:   MT (IPv6 Unicast) fcbb:bb00:6::/48 D:0 Metric: 1 Algorithm: 0
  END SID: fcbb:bb00:6:: uN (PSP/USD)
    SID Structure:
      Block Length: 32, Node-ID Length: 16, Func-Length: 0, Args-Length: 80
SRv6 Locator:   MT (IPv6 Unicast) fcbb:bb01:6::/48 D:0 Metric: 1 Algorithm: 128
  END SID: fcbb:bb01:6:: uN (PSP/USD)

Metric: 1          MT (IPv6 Unicast) IPv6 fcbb:bb00:6::/48
```

- Part 2: the locators.
- The SRv6 locator TLV contains fcbb:bb00:6::/48, in algorithm 0, with a metric of 1. This is the additional 1 in the metric 31 from Episode 2.
- Below it is the End SID, PE6's uN, with its structure: block 32, node 16, function 0, argument 80. The 80 bits of argument are the space for the next micro-SIDs.
- This is a second locator, in algorithm 128, for Flex-Algo.
- The same locator is here again, as a normal IPv6 prefix. RFC 9352 permits this. Then routers without SRv6 can also route to the locator.

Ep 8.5 · Loopback and locator, side by side

Source: lab-evidence/08-isis/rerecord/PE1-healthy.log

```
  Metric: 0          MT (IPv6 Unicast) IPv6 2001:db8:100::6/128
    Prefix Attribute Flags: X:0 R:0 N:1 E:0 A:0
    Source Router ID: 2001:db8:100::6
  Metric: 1          MT (IPv6 Unicast) IPv6 fcbb:bb00:6::/48
    Prefix Attribute Flags: X:0 R:0 N:0 E:0 A:0
```

- In the LSP, compare PE6's two most important prefixes. The first fault in Episode 3 is here.
- The loopback has metric 0 and the N flag. The N flag means that this prefix identifies the node.
- The locator has metric 1, and it is a different prefix. These are two lines in the same LSP, from different parts of the configuration. One can disappear while the other stays.

Ep 8.6 · Layer 2: the LSP, links

Source: lab-evidence/08-isis/rerecord/PE1-healthy.log

```
Metric: 10         MT (IPv6 Unicast) IS-Extended P4.00
  Interface IPv6 Address: 2001:db8:200:46::1
  Neighbor IPv6 Address: 2001:db8:200:46::
  Link Average Delay: 5000 us
  END.X SID: fcbb:bb01:6:e000:: B:1 S:0 P:0 uA (PSP/USD) Alg:128
  END.X SID: fcbb:bb01:6:e001:: B:0 S:0 P:0 uA (PSP/USD) Alg:128
  END.X SID: fcbb:bb00:6:e000:: B:1 S:0 P:0 uA (PSP/USD) Alg:0
  END.X SID: fcbb:bb00:6:e001:: B:0 S:0 P:0 uA (PSP/USD) Alg:0
    SID Structure:
      Block Length: 32, Node-ID Length: 16, Func-Length: 16, Args-Length: 64
```

- Part 3: each link. This is PE6's link to P4.
- The metric is 10, with the IPv6 addresses of the two ends.
- The link delay is 5000 microseconds. This is the static value that we configured for Flex-Algo. It is not a measurement. Episode 11 uses it.
- There are four End.X SIDs, two for each algorithm. The B flag shows the SIDs that can have protection.
- Their structure is block 32, node 16, function 16, argument 64. Our first plan expected an argument of zero. The router shows 64, and the router is correct.

Ep 8.7 · Layers 3, 4 and 5

Source: lab-evidence/08-isis/rerecord/PE1-healthy.log

```
RP/0/RP0/CPU0:PE1#show isis ipv6 route fcbb:bb00:6::/48 detail
L2 fcbb:bb00:6::/48 [31/115]
     via fe80::5454:ff:fe66:75e1, GigabitEthernet0/0/0/0, P2

RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48 detail
  Known via "isis CORE", distance 115, metric 31, SRv6-locator, type level-2
    fe80::5454:ff:fe66:75e1, from 2001:db8:100::6, via GigabitEthernet0/0/0/0, Protected

RP/0/RP0/CPU0:PE1#show cef ipv6 fcbb:bb00:6::/48
   via fe80::5454:ff:fe66:75e1/128, GigabitEthernet0/0/0/0
```

- Now the last three layers, for the same locator, on PE1.
- Layer 3: IS-IS calculated the route, with metric 31, through P2.
- Layer 4: the RIB installed it. It shows the route as an SRv6 locator, protected. Episode 9 explains the word protected.
- Layer 5: CEF forwards it out of GigabitEthernet 0/0/0/0. All layers show the same next hop. When the layers do not agree, you have found the fault.

Ep 8.8 · Fault 1: the interface leaves the IPv6 topology

```
! PE1
router isis CORE
 interface GigabitEthernet0/0/0/0
  no address-family ipv6 unicast
```

- Now we break the network, one layer at a time. Fault 1: on PE1, we remove the IPv6 address family from the IS-IS interface to P2.
- The interface stays up, with its addresses. Only IS-IS stops the use of IPv6 on it. Predict the result before the next page.

Ep 8.9 · Fault 1: what changed

Source: lab-evidence/08-isis/rerecord/F1-PE1.log

```
RP/0/RP0/CPU0:PE1#show isis neighbors
P3             Gi0/0/0/1        *PtoP*         Up    22       L2   Capable

RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
  Known via "isis CORE", distance 115, metric 71, SRv6-locator, type level-2

RP/0/RP0/CPU0:PE1#traceroute fcbb:bb00:6:: source 2001:db8:100::1
 1  2001:db8:200:13::1
 2  2001:db8:200:23::
 3  2001:db8:200:24::1
 4  2001:db8:100::6

RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 2
Success rate is 100 percent (2/2), round-trip min/avg/max = 2/2/3 ms
```

- This is PE1, twenty seconds later.
- The adjacency to P2 is gone. When IS-IS runs only IPv6, the IPv6 topology is the only reason to keep a neighbor.
- PE6's locator now has metric 71, not 31.
- The traceroute shows the new path: down the expensive link to P3, back to P2, then P4 and PE6. Each packet now goes through two more hops.
- The customer ping still works. Nobody complains. This fault is quiet, thus it is dangerous. When we restored the address family, P2 came back and the metric became 31 again.

Ep 8.10 · Fault 2: what PE1 shows

Source: lab-evidence/03-faults/v3/A-PE1.log

```
RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
% Network not in table

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
       SID Locator: fcbb:bb00:6::/48, RIB metric: inaccessible

RP/0/RP0/CPU0:PE1#show cef vrf GREEN 10.30.6.6/32
    unresolved
    0     Y   recursive                drop

RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 0 percent (0/3)
```

- You saw fault 2 in Episode 3: on PE6, we removed the locator from IS-IS. Here are the outputs on PE1, in the layer model.
- Layer 4: the RIB has no route to the locator.
- BGP still has the customer path, but the locator is inaccessible.
- Layer 5: CEF is unresolved, and the last column shows drop.
- The customer ping fails.

Ep 8.11 · Fault 2: layers 1 to 5 on PE1

Source: lab-evidence/08-isis/fault2/before-PE1.log, after-PE1.log

```
RP/0/RP0/CPU0:PE1#show isis adjacency
P2             Gi0/0/0/0    *PtoP*   Up
P3             Gi0/0/0/1    *PtoP*   Up

RP/0/RP0/CPU0:PE1#show isis database verbose PE6.00-00 | include Locator
  SRv6 Locator:   MT (IPv6 Unicast) fcbb:bb01:6::/48 D:0 Metric: 1 Algorithm: 128
! before the fault: a second line, fcbb:bb00:6::/48, Algorithm: 0

RP/0/RP0/CPU0:PE1#show isis ipv6 route fcbb:bb00:6::/48
The requested route fcbb:bb00:6::/48 was not found in the IS-IS CORE IPv6 Unicast Routing Table

RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
% Network not in table
```

- Now we read all five layers of fault 2. On PE6, the locator MAIN is removed from IS-IS.
- Layer 1: the adjacencies are up. PE1 still has its neighbors P2 and P3.
- Layer 2: the LSP of PE6 has lost the locator. Only the Flex-Algo locator remains. Before the fault, the LSP also had fcbb:bb00:6::/48 in algorithm 0.
- Layer 3: SPF has no prefix to calculate. The IS-IS route table does not contain the route.
- Layer 4: the RIB has no route. Layer 5 follows: CEF drops the service, as the previous page showed.
- The fault started at layer 2, on the far router. When the locator is added back, the route returns with metric 31, and the ping succeeds.

Ep 8.12 · Fault 3: a filter on the way into the RIB

```
! PE1
ipv6 prefix-list BLOCK-PE6-LOCATOR
 10 deny fcbb:bb00:6::/48
 20 permit ::/0 le 128
!
router isis CORE
 address-family ipv6 unicast
  distribute-list prefix-list BLOCK-PE6-LOCATOR in
```

- Fault 3 is the most difficult to see. On PE1, we add a filter between IS-IS and the RIB.
- It is a prefix list. It denies only PE6's locator and permits all other prefixes.
- We apply it with distribute-list in, under the IPv6 address family. It filters the routes that IS-IS installs. It does not stop the flooding of LSPs.

Ep 8.13 · Fault 3: the layers disagree

Source: lab-evidence/08-isis/rerecord/F3-PE1.log

```
RP/0/RP0/CPU0:PE1#show isis database verbose PE6.00-00 | include Locator
  SRv6 Locator:   MT (IPv6 Unicast) fcbb:bb00:6::/48 D:0 Metric: 1 Algorithm: 0

RP/0/RP0/CPU0:PE1#show isis ipv6 route fcbb:bb00:6::/48
L2 fcbb:bb00:6::/48 [31/115]

RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
% Network not in table

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
      valid, internal, best, group-best
       SID Locator: fcbb:bb00:6::/48, RIB metric: inaccessible

RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 2
Success rate is 0 percent (0/2)
```

- Now read the five layers on PE1, from top to bottom.
- Layer 2: PE6's locator is in the LSP. The filter did not change the flooding.
- Layer 3: IS-IS still calculates the route, with metric 31.
- Layer 4: the RIB does not have it. Network not in table.
- BGP keeps the customer path as best, with the locator inaccessible, as in Episode 3.
- The customer has no service. Layers 2 and 3 say yes, and layer 4 says no. This difference shows that there is a filter.

Ep 9.1 · Link P2 to P4, repaired by P2

Router / tool: P2 · Source: lab-evidence/09-tilfa/v3/link-P2.log

```
RP/0/RP0/CPU0:P2#show isis ipv6 fast-reroute fcbb:bb00:6::/48 detail
       Backup path: TI-LFA (link), via ..., GigabitEthernet0/0/0/2 P3, Metric: 71
         P node: P3.00 [2001:db8:100::3], SRv6 SID: fcbb:bb00:3:: uN (PSP/USD)
         Q node: P5.00 [2001:db8:100::5], SRv6 SID: fcbb:bb00:3:e005:: uA (PSP/USD)

RP/0/RP0/CPU0:P2#show cef ipv6 fcbb:bb00:6::/48 detail
    next hop ..., Repair Node(s): 2001:db8:100::5
    SRv6 H.Insert.Red SID-list {fcbb:bb00:3:e005::}
```

- Now the router. First, without node protection, the example that we drew.
- P2 has a TI-LFA backup through P3. The metric after the repair is 71.
- The P node is P3, the Q node is P5, and the adjacency SID e005 goes from P3 to P5. This agrees with our prediction.
- In CEF, the repair is H.Insert.Red with one container, fcbb:bb00:3:e005::. P3's node SID and its adjacency to P5 are in one address.
- On this platform, the repair inserts a routing header into the packet. It does not add a second outer header.

Ep 9.2 · Link P4 to PE6, and link PE1 to P2

Router / tool: P4 and PE1 · Source: lab-evidence/09-tilfa/v3/link-P4.log, link-PE1.log

```
RP/0/RP0/CPU0:P4#show isis ipv6 fast-reroute fcbb:bb00:6::/48 detail
       Backup path: TI-LFA (link), via ..., GigabitEthernet0/0/0/2 P5, Metric: 51
         P node: P5.00 [2001:db8:100::5], SRv6 SID: fcbb:bb00:5:: uN (PSP/USD)
         Q node: PE6.00 [2001:db8:100::6], SRv6 SID: fcbb:bb00:5:e001:: uA (PSP/USD)
    SRv6 H.Insert.Red SID-list {fcbb:bb00:5:e001::}

RP/0/RP0/CPU0:PE1#show isis ipv6 fast-reroute fcbb:bb00:6::/48 detail
       Backup path: LFA, via ..., GigabitEthernet0/0/0/1, P3, Metric: 71
```

- Two more link examples.
- P4 protects its link to PE6: go to P5, then use the adjacency from P5 to PE6, in one container. The metric is 51. This agrees with the prediction.
- PE1 protects its link to P2. The backup is a normal LFA through P3, with no SIDs. Not every failure needs a segment list. This example proves it.

Ep 9.3 · Asking for node protection

```
router isis CORE
 address-family ipv6 unicast
  fast-reroute per-prefix tiebreaker node-protecting index 100
```

- Link protection expects that only the link failed. If the full neighbor router fails, a link-protecting backup can send traffic back to that router.
- On this release, we had to request node protection with a tiebreaker. We added it on all six routers.

Ep 9.4 · Node P2, repaired by PE1

Source: lab-evidence/09-tilfa/v3/node-PE1.log

```
RP/0/RP0/CPU0:PE1#show isis ipv6 fast-reroute fcbb:bb00:6::/48 detail
       Backup path: TI-LFA (node), via ..., GigabitEthernet0/0/0/1 P3, Metric: 101
         P node: P3.00 [2001:db8:100::3], SRv6 SID: fcbb:bb00:3:: uN (PSP/USD)
         Q node: P5.00 [2001:db8:100::5], SRv6 SID: fcbb:bb00:3:e005:: uA (PSP/USD)
    SRv6 H.Insert.Red SID-list {fcbb:bb00:3:e005::}
```

- Now the node examples. First, P2 fails, as PE1 sees it.
- PE1 now has a node-protecting backup through P3, with metric 101.
- The list is one container: go to P3, then use the adjacency to P5. It is shorter than our manual calculation, which had three segments.
- Why? P3 is PE1's direct neighbor, so the first hop needs no adjacency SID; the outgoing interface is sufficient. From P5, the normal path to PE6 goes through P4, not through P2. Thus PE6 needs no segment.
- The router found the shortest list. Our manual calculation was correct, but not the shortest. Remember this difference when you check your predictions.

Ep 9.5 · Node P4, repaired by P2

Router / tool: P2 · Source: lab-evidence/09-tilfa/v3/node-P2.log

```
RP/0/RP0/CPU0:P2#show isis ipv6 fast-reroute fcbb:bb00:6::/48 detail
       Backup path: TI-LFA (node), via ..., GigabitEthernet0/0/0/2 P3, Metric: 91
         P node: P3.00 [2001:db8:100::3], SRv6 SID: fcbb:bb00:3:: uN (PSP/USD)
         Q node: P5.00 [2001:db8:100::5], SRv6 SID: fcbb:bb00:3:e005:: uA (PSP/USD)
         Q node: PE6.00 [2001:db8:100::6], SRv6 SID: fcbb:bb00:5:e001:: uA (PSP/USD)
    SRv6 H.Insert.Red SID-list {fcbb:bb00:3:e005:e001::}
```

- Now P4 fails, as P2 sees it.
- There is a node-protecting backup through P3, with metric 91.
- The repair has three parts: go to P3, use the adjacency from P3 to P5, then use the adjacency from P5 to PE6. This agrees with the manual calculation.
- Look at the encoding: fcbb:bb00:3:e005:e001::. The second adjacency, e001, is P5's adjacency. P5's node SID is not in front of it. After P3 shifts the address, P5 finds e001 as its own local SID.
- Three instructions are in one container, and there is no routing header.

Ep 9.6 · When node protection is impossible

Router / tool: P4 · Source: lab-evidence/09-tilfa/v3/node-P4.log

```
RP/0/RP0/CPU0:P4#show isis ipv6 fast-reroute fcbb:bb00:6::/48 detail
       Backup path: TI-LFA (link), via ..., GigabitEthernet0/0/0/2 P5, Metric: 51
       TI-LFA Path Fallback Information:
         TI-LFA link: Path valid
         TI-LFA node: Path not found
```

- One more example, which was not in the predictions. P4 protects traffic to PE6, with node protection requested.
- P4 still uses a link-protecting backup.
- P4 gives the reason: it did not find a node-protecting path. The destination, PE6, is the neighbor. You cannot protect traffic to a router against the failure of that router.

Ep 9.7 · How much is protected

Router / tool: P2 · Source: lab-evidence/09-tilfa/v3/node-P2.log

```
RP/0/RP0/CPU0:P2#show isis ipv6 fast-reroute summary
IS-IS CORE IPv6 Unicast FRR summary
                          Critical   High       Medium     Low        Total
                          Priority   Priority   Priority   Priority
Prefixes reachable in L2
  All paths protected     0          0          5          10         15
  Some paths protected    0          0          0          0          0
  Unprotected             0          0          0          0          0
  Protection coverage     0.00%      0.00%      100.00%    100.00%    100.00%
```

- One command shows if some prefixes have no protection. Use it on each router after a change.
- On P2, all 15 IPv6 prefixes have a protected path.
- No prefix is unprotected.
- The coverage is 100 percent. Before, when two links had changed to one-way without a warning, P4 showed only 87 percent here. When this number decreases, the topology is often not what you think.

Ep 9.8 · Failure test: the ping and the shutdown

Router / tool: PE1 and P4 · Source: lab-evidence/09-tilfa/v3/fail-ping.log, fail-P4-shut.log

```
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 4000 timeout 1
Sending 4000, 100-byte ICMP Echos to 10.30.6.6 timeout is 1 seconds:

# while the ping runs, on P4:
RP/0/RP0/CPU0:P4(config)#interface GigabitEthernet0/0/0/0 shutdown
RP/0/RP0/CPU0:P4(config)#commit

# when the ping ends, on PE1:
Success rate is 100 percent (4000/4000), round-trip min/avg/max = 1/2/13 ms
```

- Now the main test. We make a failure while traffic flows. First, in CML, we start captures on the core links.
- Then, on PE1, we start a long GREEN ping: 4000 packets, with a timeout of 1 second.
- While the ping runs, we shut the interface of P4 toward P2. This is the failure. P2 is the point of local repair.
- When the ping ends, all 4000 packets got a reply. Next, we see what the routers recorded.

Ep 9.9 · Failure test: what the routers saw

Router / tool: P2 and PE1 · Source: lab-evidence/09-tilfa/v3/fail-P2-log.log, fail-PE1-after.log

```
RP/0/RP0/CPU0:P2#show logging last 12
%L2-BFD-6-SESSION_STATE_DOWN : BFD session to neighbor ... has gone down. Reason: Nbor signalled down
%ROUTING-ISIS-5-ADJCHANGE : ISIS (CORE): Adjacency to P4 (GigabitEthernet0/0/0/1) (L2) Down, BFD session DOWN

RP/0/RP0/CPU0:PE1#show isis ipv6 route fcbb:bb00:6::/48 detail
L2 fcbb:bb00:6::/48 [81/115] Label: None, low priority
     via fe80::5454:ff:fe66:75e1, GigabitEthernet0/0/0/0, P2, SRGB Base: 16000, Weight: 0

RP/0/RP0/CPU0:PE1#traceroute fcbb:bb00:6:: source 2001:db8:100::1
 1  2001:db8:200:12::1
 2  2001:db8:200:23::1
 3  2001:db8:200:35::1
 4  2001:db8:200:45::
 5  2001:db8:100::6
```

- What did the routers record? First, the log of P2.
- BFD on P2 reports that the session to P4 is down. Then IS-IS removes the adjacency to P4.
- A few seconds later, PE1 has a new IS-IS route. The metric is 81, not 31. The next hop is still P2.
- The traceroute shows the new path: P2, P3, P5, P4, and then PE6. It has five hops, not three.

Ep 9.10 · A repair, caught on the wire

Router / tool: tshark · Source: lab-evidence/09-tilfa/v3/fail-P2-P3.pcap (columns aligned)

```
$ tshark -r fail-P2-P3.pcap -Y "icmp.type == 8" -T fields -e ipv6.dst -e ipv6.routing.segleft -e ipv6.nxt | sort | uniq -c
   44  fcbb:bb00:3:e005:e001::   1   43
   15  fcbb:bb00:3:e005::        1   43
 1481  fcbb:bb00:3:e005:6:e008::     4

# time of the first packet of each group, in seconds
9.645  fcbb:bb00:3:e005:e001::
9.749  fcbb:bb00:3:e005::
9.787  fcbb:bb00:3:e005:6:e008::
```

- Now the most important part: the packets on the link from P2 to P3. We count the echo requests by destination. The last two columns are Segments Left and the next header.
- Phase 1: 44 packets. P2 inserted its node-protecting repair. The routing header has Segments Left 1, and the next header is 43.
- Phase 2: 15 packets with a shorter list: P3 and the adjacency to P5, still with a routing header. We did not verify which router state caused this short phase.
- Phase 3: 1481 packets with no routing header. The next header is 4. PE1 uses its new path, and P2 stopped the repair. Episode 10 explains this phase.
- The times show how short the repair was. Phase 1 starts at 9.645 seconds, and phase 3 starts at 9.787. The repair carried traffic for about 0.14 seconds.

Ep 9.11 · Following phase 1 to PE6

Router / tool: tshark · Source: lab-evidence/09-tilfa/v3/fail-P2-P3, fail-P3-P5, fail-P5-PE6 (columns aligned)

```
$ tshark -r <capture file> -Y "icmp.type == 8" -T fields -e ipv6.dst -e ipv6.routing.segleft -e ipv6.nxt | sort | uniq -c
# P2 to P3
   44  fcbb:bb00:3:e005:e001::   1   43
# P3 to P5
   44  fcbb:bb00:e001::          1   43
 1496  fcbb:bb00:6:e008::            4
# P5 to PE6
   44  fcbb:bb00:6:e008::            4
```

- Follow the 44 repaired packets to PE6. We read the capture of each link with the same command.
- On the link P2 to P3, P2 inserted the repair in front of the service SID.
- P3 did its adjacency instruction and shifted the address. The remaining instruction is e001, P5's adjacency to PE6.
- P5 did e001 and sent the packet on the direct link to PE6. P5 is the penultimate endpoint, thus it removed the routing header. PE6 received only the service SID.
- The other 1496 packets on the link P3 to P5 are the later traffic of phases 2 and 3. They have the same destination, and they have no routing header.
- The functions from Episodes 4, 5 and 6 worked together in less than a quarter of a second.

Ep 9.12 · Run it twice

Source: lab-evidence/09-tilfa/v3/fail-ping.log, fail2-ping.log

```
# run 1
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 4000 timeout 1
Success rate is 100 percent (4000/4000), round-trip min/avg/max = 1/2/13 ms

# run 2, the same failure
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 6000 timeout 1
Success rate is 99 percent (5992/6000), round-trip min/avg/max = 1/2/17 ms
```

- We did the same failure again, and we got a different result. You must see this too.
- In the first test, the repair carried traffic for about 0.14 seconds, and no ping was lost.
- In the second test, no repaired packet appeared, and 8 pings were lost. There were no packets on the wire for approximately one second. Then PE1's new path started.
- We did not verify why the two tests were different. This is a software forwarder in a virtual lab. It proves that the repair is there and shows its format. It cannot tell you the speed of protection. Measure the speed on hardware.

Ep 10.1 · The configuration and the delay

Source: lab-evidence/10-microloop/PE1-config.log

```
RP/0/RP0/CPU0:PE1#show running-config router isis CORE address-family ipv6 unicast
  microloop avoidance segment-routing

RP/0/RP0/CPU0:PE1#show isis protocol
        Microloop avoidance:       Enabled
          Configuration:           Type: Segment Routing, RIB update delay: 5000 msec
```

- This is the configuration, and the router output.
- There is one line under the IPv6 address family, on each router.
- The router shows a RIB update delay of 5000 milliseconds. This is the Cisco default value. No RFC gives this value.

Ep 10.2 · The failure test

Router / tool: P4 and P2 · Source: lab-evidence/10-microloop/P4-shut.log, P2-log.log

```
RP/0/RP0/CPU0:P4(config)#interface GigabitEthernet0/0/0/0 shutdown
RP/0/RP0/CPU0:P4(config)#commit

RP/0/RP0/CPU0:P2#show logging last 12
03:46:34.253  %L2-BFD-6-SESSION_STATE_DOWN : BFD session to neighbor ... has gone down. Reason: Nbor signalled down
03:46:34.253  %ROUTING-ISIS-5-ADJCHANGE : ISIS (CORE): Adjacency to P4 (GigabitEthernet0/0/0/1) (L2) Down, BFD session DOWN
```

- Now the test. As in Episode 9, we make the link between P2 and P4 fail while a GREEN ping runs from PE1. In CML, we start captures on the core links first.
- On P4, we shut the interface toward P2, and we commit.
- On P2, the log shows the BFD session to P4 going down, with the reason: the neighbor signalled down.
- IS-IS removes the adjacency in the same millisecond. This is time zero for everything that follows.

Ep 10.3 · P2, a fraction of a second later

Router / tool: P2 · Source: lab-evidence/10-microloop/t1-P2.log

```
RP/0/RP0/CPU0:P2#show isis ipv6 route fcbb:bb00:6::/48 detail
L2 fcbb:bb00:6::/48 [71/115]
   Installed Oct 03 03:46:34.354
     via fe80::5454:ff:fea8:d13a, GigabitEthernet0/0/0/2, P3
     exp fe80::5454:ff:fea8:d13a, GigabitEthernet0/0/0/2, P3
       P node: P3.00 [2001:db8:100::3], SRv6 SID: fcbb:bb00:3:: uN (PSP/USD)
       Q node: P5.00 [2001:db8:100::5], SRv6 SID: fcbb:bb00:3:e005:: uA (PSP/USD)
    SRv6 H.Insert.Red SID-list {fcbb:bb00:3:e005::}
```

- We made the P2 to P4 link fail again, and we read the routers immediately after.
- P2 installed its new route to PE6, with metric 71, at 34.354 seconds after the minute.
- The new next hop is P3. There is a second line with the mark exp: an explicit path.
- The explicit path is P3, then the adjacency to P5. These are the same segments as the repair. Thus packets get to P5 without the old table of P3.

Ep 10.4 · PE1 does the same

Source: lab-evidence/10-microloop/t1-PE1.log

```
RP/0/RP0/CPU0:PE1#show isis ipv6 route fcbb:bb00:6::/48 detail
L2 fcbb:bb00:6::/48 [81/115]
   Installed Oct 03 03:46:34.719
     via fe80::5454:ff:fe66:75e1, GigabitEthernet0/0/0/0, P2
     exp fe80::5454:ff:fe66:75e1, GigabitEthernet0/0/0/0, P2
       P node: P3.00 [2001:db8:100::3], SRv6 SID: fcbb:bb00:3:: uN (PSP/USD)
       Q node: P5.00 [2001:db8:100::5], SRv6 SID: fcbb:bb00:3:e005:: uA (PSP/USD)
```

- PE1 is two hops from the failure, and it also changes.
- PE1 installed its new route, with metric 81, at 34.719. This is a little more than one third of a second after P2.
- PE1 also keeps an explicit path through P3 and the adjacency to P5. Its normal next hop is still P2.

Ep 10.5 · PE1, a few seconds later

Source: lab-evidence/10-microloop/t3-PE1.log

```
RP/0/RP0/CPU0:PE1#show isis ipv6 route fcbb:bb00:6::/48 detail
L2 fcbb:bb00:6::/48 [81/115]
   Installed Oct 03 03:46:40.219
     via fe80::5454:ff:fe66:75e1, GigabitEthernet0/0/0/0, P2

RP/0/RP0/CPU0:PE1#show cef ipv6 fcbb:bb00:6::/48 detail
   via fe80::5454:ff:fe66:75e1/128, GigabitEthernet0/0/0/0, protected
   via fe80::5454:ff:fec1:58eb/128, GigabitEthernet0/0/0/1, backup (Local-LFA)
```

- This is PE1 again, some seconds later.
- There is a new installation at 40.219. The explicit path is gone. Only the normal path through P2 is there.
- CEF shows the route as protected again, with a new backup for the new topology. The network is stable.

Ep 10.6 · The explicit path on the wire

Router / tool: tshark · Source: lab-evidence/09-tilfa/v3/fail2-P2-P3.pcap (second test)

```
$ tshark -r fail2-P2-P3.pcap -Y "icmp.type == 8" -T fields -e ipv6.dst | sort | uniq -c
  979  fcbb:bb00:3:e005:6:e008::
 2621  fcbb:bb00:6:e008::

# first and last packet of the explicit path, in seconds
9.897   first   fcbb:bb00:3:e005:6:e008::
14.087  last    fcbb:bb00:3:e005:6:e008::
```

- Now we can explain phase 3 from Episode 9. This capture is from the second test of that failure. We count the echo requests on the link P2 to P3 by destination.
- Immediately after the failure, GREEN traffic is in one container: P3, the adjacency to P5, then PE6's service SID. This is the explicit path, with the service, in one address. There are 979 packets.
- Later, the packets use only the service SID again. They follow the new shortest path through P2, P3, P5 and P4. There are 2621 packets.
- The explicit path ran from 9.897 to 14.087 seconds, which is about 4.2 seconds. The router state showed 5.5 seconds. Use both numbers as approximate values, not as measurements.

Ep 10.7 · Did anything loop?

Router / tool: tshark · Source: lab-evidence/09-tilfa/v3/fail-P2-P3.pcap, fail2-P2-P3.pcap

```
$ tshark -r <capture file> -Y "icmp.type == 8" -T fields -e eth.src -e ipv6.hlim | sort | uniq -c
# link P2 to P3, run 1
 1540  56:54:00:1c:70:f9  254
# link P2 to P3, run 2
 3600  56:54:00:1c:70:f9  254
```

- A microloop leaves evidence on the wire. Thus we looked for it on the link between P2 and P3, the two routers from our manual example. We count the packets by source MAC address and hop limit.
- In the first test, one source sent all 1540 packets, with hop limit 254. The source is P2.
- The second test gave the same result: 3600 packets, one source, hop limit 254.
- No packet came from P3. A looping packet would appear in the two directions, with a decreasing hop limit. With microloop avoidance on, we saw none.
- We did not verify if a loop occurs with avoidance off. This is a good test for your lab.

Ep 10.8 · BFD

Source: lab-evidence/10-microloop/PE1-config.log

```
RP/0/RP0/CPU0:PE1#show running-config router isis CORE interface GigabitEthernet0/0/0/0
  bfd minimum-interval 100
  bfd multiplier 3
  bfd fast-detect ipv6

RP/0/RP0/CPU0:PE1#show bfd ipv6 session
Gi0/0/0/0            fe80::5454:ff:fe66:75e1
No                   n/a             0s(0s*0)         300ms(100ms*3)   UP
```

- Detection is the first step. This is how we made it fast.
- BFD runs on each IS-IS interface, every 100 milliseconds. After 3 missed packets, BFD declares the session down.
- Thus the detection time is 300 milliseconds, and the session is up. Without BFD, IS-IS waits for its hello holdtime, which is many seconds.
- In our test, P4 shut its own interface, so P2 learned about the failure from BFD. Thus the BFD event and the adjacency event occurred in the same millisecond.

Ep 10.9 · Restoration is an event too

Router / tool: P4, P2 and PE1 · Source: lab-evidence/10-microloop/restore

```
RP/0/RP0/CPU0:P4(config)#no interface GigabitEthernet0/0/0/0 shutdown
RP/0/RP0/CPU0:P4(config)#commit

RP/0/RP0/CPU0:P2#show logging last 12
03:52:07.575  %L2-BFD-6-SESSION_STATE_UP : BFD session to neighbor ... on interface GigabitEthernet0/0/0/1 is up
03:52:14.102  %ROUTING-ISIS-5-ADJCHANGE : ISIS (CORE): Adjacency to P4 (GigabitEthernet0/0/0/1) (L2) Up, New adjacency

RP/0/RP0/CPU0:PE1#show isis ipv6 route fcbb:bb00:6::/48 detail
L2 fcbb:bb00:6::/48 [31/115] Label: None, low priority
   Installed Oct 03 03:52:20.122 for 00:00:04

RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 6000 timeout 1
Success rate is 100 percent (6000/6000), round-trip min/avg/max = 1/2/12 ms
```

- Then we restored the link while the pings continued. The restoration is also an event.
- On P4, we remove the shutdown, and we commit.
- BFD came up first, at 07.575 seconds after 03:52.
- IS-IS brought the adjacency up at 14.102, six and a half seconds later.
- PE1 changed its route back to metric 31 at 20.122, six seconds after that. This agrees with the same 5000-millisecond delay for an up event.
- No ping was lost: 6000 of 6000. For the whole test, the traffic stayed on the detour. A repaired network changes back step by step, more than ten seconds later.

Ep 11.1 · The configuration

```
! PE1: defines and advertises algorithm 128
router isis CORE
 flex-algo 128
  priority 200
  metric-type delay
  advertise-definition
!
! P3: takes part, does not advertise a definition
router isis CORE
 flex-algo 128
  metric-type delay
!
! every router: a locator in algorithm 128
segment-routing srv6 locators locator FA128
 prefix fcbb:bb01:1::/48
 algorithm 128
```

- This is our configuration, in three parts.
- PE1 defines algorithm 128 with the delay metric, at priority 200, and advertises the definition. PE6 does the same, at priority 100.
- P3 participates, with the same metric type, but it does not advertise a definition. P2, P4 and P5 have the same configuration.
- Each router has a second locator in the fcbb:bb01 block, for algorithm 128. In this series, we use a separate block for Flex-Algo. This is our choice, not a rule of the RFC.

Ep 11.2 · Who defines algorithm 128?

Router / tool: P3 · Source: lab-evidence/11-flexalgo/P3.log

```
RP/0/RP0/CPU0:P3#show isis flex-algo 128
Flex-Algo 128:
  Level-2:
    Definition Priority: 200
    Definition Source: PE1.00
    Definition Equal to Local: Yes
    Definition Calculation Type: SPF
    Definition Metric Type: Delay
  Topologies supported:
    IPv6 Unicast
  Local Priority: 128
```

- Two routers advertise a definition. Which definition does the network use? We ask P3, which advertises none.
- The selected definition comes from PE1, at priority 200.
- It is the same as P3's local configuration. If it were different, P3 would still use the selected definition, and this line would tell you to compare them.
- The metric type is delay.
- P3's local priority is 128, the default value, because P3 does not compete. RFC 9350 selects the highest priority. If two are equal, it selects the highest system ID.

Ep 11.3 · The delays we advertise

```
! PE1
performance-measurement
 interface GigabitEthernet0/0/0/0
  delay-measurement
   advertise-delay 5000
 interface GigabitEthernet0/0/0/1
  delay-measurement
   advertise-delay 1000
```

- The delay values must come from a source. In a real network, the router measures them. In this series, we advertise fixed values. Then you can build the same lab.
- PE1 advertises 5000 microseconds for its upper link to P2.
- It advertises 1000 for its lower link to P3. Each lower link has 1000. Each upper link and cross link has 5000. This is the opposite of the costs, on purpose.

Ep 11.4 · Two routes, two paths

Source: lab-evidence/11-flexalgo/PE1.log

```
RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
  Known via "isis CORE", distance 115, metric 31, SRv6-locator, type level-2
RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb01:6::/48
  Known via "isis CORE", distance 115, metric 3001, SRv6-locator (algo 128), type level-2

RP/0/RP0/CPU0:PE1#traceroute fcbb:bb00:6:: source 2001:db8:100::1
 1  2001:db8:200:12::1
 2  2001:db8:200:24::1
 3  2001:db8:100::6
RP/0/RP0/CPU0:PE1#traceroute fcbb:bb01:6:: source 2001:db8:100::1
 1  2001:db8:200:13::1
 2  2001:db8:200:35::1
 3  2001:db8:100::6
```

- The router gives the same result.
- There are two routes to PE6, one for each locator. Algorithm 0 has metric 31. Algorithm 128 has metric 3001: 3000 microseconds plus 1 for the locator.
- A traceroute to PE6's algorithm 0 SID goes through P2 and P4.
- A traceroute to its algorithm 128 SID goes through P3 and P5. The routers and the links are the same, but there are two topologies. The destination address selects the topology.

Ep 11.5 · Every algorithm has its own SIDs

Source: lab-evidence/11-flexalgo/PE1-frr128.log (FA128 locator, node and adjacency SIDs)

```
RP/0/RP0/CPU0:PE1#show segment-routing srv6 sid
*** Locator: 'FA128' ***
fcbb:bb01:1::              uN (PSP/USD)      'default':1                       sidmgr
fcbb:bb01:1:e000::         uA (PSP/USD)      [Gi0/0/0/0, Link-Local]:128:P     isis-CORE
fcbb:bb01:1:e001::         uA (PSP/USD)      [Gi0/0/0/0, Link-Local]:128       isis-CORE
fcbb:bb01:1:e002::         uA (PSP/USD)      [Gi0/0/0/1, Link-Local]:128:P     isis-CORE
fcbb:bb01:1:e003::         uA (PSP/USD)      [Gi0/0/0/1, Link-Local]:128       isis-CORE
```

- Each algorithm has its own SIDs. Episode 8 showed this in the LSP.
- PE1 has a second node SID, fcbb:bb01:1::, in algorithm 128.
- It also has a second pair of adjacency SIDs for each link, with the mark 128. A packet with these SIDs follows the algorithm 128 paths on each hop, not only on the first hop.

Ep 11.6 · Protection stays inside the algorithm

Source: lab-evidence/11-flexalgo/PE1-frr128.log

```
RP/0/RP0/CPU0:PE1#show isis ipv6 fast-reroute flex-algo 128 fcbb:bb01:6::/48 detail
     via ..., GigabitEthernet0/0/0/1, P3
       Backup path: TI-LFA (node), via ..., GigabitEthernet0/0/0/0 P2, Metric: 15001
         P node: P4.00 [2001:db8:100::4], SRv6 SID: fcbb:bb01:4:: uN (PSP/USD)

RP/0/RP0/CPU0:PE1#show cef ipv6 fcbb:bb01:6::/48 detail
    SRv6 H.Insert.Red SID-list {fcbb:bb01:4::}
```

- TI-LFA also works in each algorithm.
- The primary path to PE6's algorithm 128 locator goes through P3. Its node-protecting backup goes through P2, with a delay metric of 15001.
- The repair SID is P4's algorithm 128 node SID, fcbb:bb01:4::. The repair stays in the same topology as the traffic that it protects.

Ep 11.7 · Putting a customer on algorithm 128

Router / tool: PE6 · Source: lab-evidence/11-flexalgo/PE6.log

```
RP/0/RP0/CPU0:PE6#show running-config router bgp 65000 vrf RED
  address-family ipv4 unicast
   segment-routing srv6
    locator MAIN
  address-family ipv6 unicast
   segment-routing srv6
    locator FA128

RP/0/RP0/CPU0:PE6#show segment-routing srv6 sid | include RED
fcbb:bb01:6:e004::          uDT6              'RED'                          bgp-65000
fcbb:bb00:6:e006::          uDT4              'RED'                          bgp-65000
```

- Now the result. How do you put a customer on the low-delay topology? In SRv6, the simplest method is one line on the egress PE.
- For RED IPv4, PE6 allocates its service SID from MAIN, algorithm 0.
- For RED IPv6, PE6 allocates it from FA128, algorithm 128.
- Thus RED IPv6's service SID is in the fcbb:bb01 block. Each packet that PE1 sends to it follows algorithm 128 automatically. No policy and no colour are necessary.

Ep 11.8 · RED IPv6, both directions

Router / tool: CE-A and tshark · Source: lab-evidence/11-flexalgo/red6-before-CE-A.log, red6-before-PE1-P3.pcap, red6-before-PE1-P2.pcap

```
CE-A:~$ sudo ping6 -c 3 2001:db8:c:206::2
3 packets transmitted, 3 packets received, 0% packet loss

$ tshark -r <capture file> -Y "icmpv6.type == 128 or icmpv6.type == 129" -T fields -e ipv6.dst
# PE1 to P3, CE-A to CE-B
fcbb:bb01:6:e004::     (outer destination, algorithm 128)
# PE1 to P2, CE-B to CE-A
fcbb:bb00:1:e007::     (outer destination, algorithm 0)
```

- Now the result on the wire. We pinged from CE-A to CE-B in RED IPv6. We captured the two core links of PE1, and we read each file with tshark.
- All three pings succeed.
- To CE-B, the packets go out on the lower link, to the algorithm 128 service SID.
- The replies come back on the upper link. PE1's own RED IPv6 SID is still in MAIN, so the return path uses algorithm 0. The two directions are different, because we configured only one end.
- If you want low delay in the two directions, allocate from the algorithm 128 locator on the two PEs.

Ep 11.9 · Experiment: make one link slow

```
! P3 and P5, the link between them
performance-measurement
 interface GigabitEthernet0/0/0/1      ! on P5: GigabitEthernet0/0/0/0
  delay-measurement
   advertise-delay 20000
```

- Now a test. We increase the advertised delay of the P3 to P5 link from 1000 to 20000 microseconds, on the two ends. The costs do not change.
- Predict the result before the next page. What occurs to each algorithm, and to RED IPv6?

Ep 11.10 · One number changed

Source: lab-evidence/11-flexalgo/after-delay-PE1.log, red6-after-CE-A.log, red6-after-PE1-P2.pcap

```
RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
  Known via "isis CORE", distance 115, metric 31, SRv6-locator, type level-2
RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb01:6::/48
  Known via "isis CORE", distance 115, metric 15001, SRv6-locator (algo 128), type level-2
    fe80::5454:ff:fe66:75e1, from 2001:db8:100::6, via GigabitEthernet0/0/0/0, Protected

RP/0/RP0/CPU0:PE1#traceroute fcbb:bb01:6:: source 2001:db8:100::1
 1  2001:db8:200:12::1
 2  2001:db8:200:24::1
 3  2001:db8:100::6

CE-A:~$ sudo ping6 -c 3 2001:db8:c:206::2
3 packets transmitted, 3 packets received, 0% packet loss

# captures after the change, from the same ping
PE1 to P2: fcbb:bb01:6:e004::  and  fcbb:bb00:1:e007::
PE1 to P3: no echo request
```

- These are the results, after the delay of the P3 to P5 link became 20000. We start with the routes on PE1.
- Algorithm 0 did not change. It has the same metric 31 and the same path. It does not use delay.
- Algorithm 128 now has metric 15001, and the next hop is P2, the upper link. The lower path has a total of 22000 microseconds, and the upper path, at 15000, is better.
- The traceroute to the algorithm 128 SID confirms this: P2, P4, then PE6.
- The customer ping still works. The service did not change.
- On the wire, the RED IPv6 traffic also moved. Both directions are now on the link PE1 to P2, and the link PE1 to P3 carries none. When we restored the delay, the traffic moved back.

Ep 12.1 · The VRF

```
vrf BLUE
 address-family ipv4 unicast
  import route-target
   65000:100
  export route-target
   65000:100
 address-family ipv6 unicast
  import route-target
   65000:100
  export route-target
   65000:100
```

- This is VRF BLUE on PE1.
- There is one VRF with the name BLUE.
- For IPv4, it imports and exports route target 65000:100.
- IPv6 is the same. RED uses 65000:200. In this series, these values are examples. Compare them with your own numbering plan.

Ep 12.2 · The attachment

```
interface GigabitEthernet0/0/0/2
 description TO-CE BLUE
 vrf BLUE
 ipv4 address 10.1.1.1 255.255.255.0
 ipv6 address 2001:db8:c:101::1/64
```

- This is the attachment to CE-A.
- First, the interface goes into VRF BLUE. Then it gets an address.
- Then it gets an IPv4 and an IPv6 address. RED has its own port and its own addresses.

Ep 12.3 · BGP

```
router bgp 65000
 address-family vpnv4 unicast
 address-family vpnv6 unicast
 neighbor 2001:db8:100::6
  remote-as 65000
  update-source Loopback0
  address-family vpnv4 unicast
  address-family vpnv6 unicast
 vrf BLUE
  rd 65000:101
  address-family ipv4 unicast
   segment-routing srv6
    locator MAIN
    alloc mode per-vrf
   redistribute connected
```

- BGP connects all the parts.
- One IPv6 session between the PE loopbacks carries VPNv4 and VPNv6. There is no IPv4 in this core.
- Under the VRF, there is a route distinguisher: 65000:101 on PE1, and 65000:106 on PE6.
- This is the SRv6 part. BGP allocates the service SID from locator MAIN, one for each VRF.
- Redistribute connected advertises the CE subnet and the tenant loopback on the PE. In a real network, you also add static or eBGP routes to the CE.

Ep 12.4 · The service SIDs

Router / tool: PE1 and PE6 · Source: lab-evidence/12-l3vpn/PE1-build.log, PE6-sids.log (service SIDs only)

```
RP/0/RP0/CPU0:PE1#show segment-routing srv6 sid
fcbb:bb00:1:e004::          uDT4              'BLUE'                          bgp-65000
fcbb:bb00:1:e005::          uDT6              'BLUE'                          bgp-65000
fcbb:bb00:1:e006::          uDT6              'RED'                           bgp-65000
fcbb:bb00:1:e007::          uDT4              'RED'                           bgp-65000

RP/0/RP0/CPU0:PE6#show segment-routing srv6 sid
fcbb:bb00:6:e004::          uDT4              'BLUE'                          bgp-65000
fcbb:bb00:6:e005::          uDT6              'BLUE'                          bgp-65000
fcbb:bb00:6:e006::          uDT4              'RED'                           bgp-65000
fcbb:bb01:6:e004::          uDT6              'RED'                           bgp-65000
```

- These are the service SIDs on the two PEs.
- On PE1, BLUE has a uDT4 and a uDT6. BGP owns both.
- RED has its own pair. The function values, e004 to e007, come from the dynamic range, in the sequence of allocation.
- PE6 has its own SIDs, with different values. On PE6, RED IPv6 is in the fcbb:bb01 block. This is the algorithm 128 setting from Episode 11. Do not write these values in your configuration. Read them.

Ep 12.5 · BGP and the VRF table

Source: lab-evidence/12-l3vpn/PE1-build.log

```
RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast summary
Neighbor        Spk    AS MsgRcvd MsgSent       TblVer  InQ OutQ  Up/Down  St/PfxRcd
2001:db8:100::6   0 65000      31      21           70    0    0 00:02:51          5

RP/0/RP0/CPU0:PE1#show route vrf BLUE
C    10.1.1.0/24 is directly connected, 00:05:38, GigabitEthernet0/0/0/2
L    10.1.1.1/32 is directly connected, 00:05:38, GigabitEthernet0/0/0/2
B    10.1.6.0/24 [200/0] via 2001:db8:100::6 (nexthop in vrf default), 00:01:13
L    10.100.1.1/32 is directly connected, 00:05:43, Loopback101
B    10.100.6.6/32 [200/0] via 2001:db8:100::6 (nexthop in vrf default), 00:01:13
```

- Now the control plane.
- The VPNv4 session to PE6 is up. It received five prefixes. The VPNv6 session shows four.
- VRF BLUE on PE1 has the local CE subnet and the tenant loopback.
- It also has two BGP routes from PE6. Their next hop is in the default table: PE6's loopback. The router resolves this next hop through PE6's locator and service SID. Episode 13 follows this chain step by step.

Ep 12.6 · Tests from CE-A

Router / tool: CE-A · Source: lab-evidence/12-l3vpn/CE-A-tests.log

```
CE-A:~$ sudo ping -c 2 -I 10.1.1.2 10.1.6.2
2 packets transmitted, 2 packets received, 0% packet loss
CE-A:~$ sudo ping -c 2 -I 10.2.1.2 10.2.6.2
2 packets transmitted, 2 packets received, 0% packet loss
CE-A:~$ sudo ping6 -c 2 2001:db8:c:106::2
2 packets transmitted, 2 packets received, 0% packet loss
CE-A:~$ sudo ping6 -c 2 2001:db8:c:206::2
2 packets transmitted, 2 packets received, 0% packet loss
CE-A:~$ sudo ping -c 2 -W 1 -I 10.1.1.2 10.2.6.2
2 packets transmitted, 0 packets received, 100% packet loss
```

- Now the important tests, from the customer side. On CE-A, the option -I sets the source address, and the source address selects the BLUE or RED port.
- BLUE IPv4, site to site: both packets are answered.
- RED IPv4: both packets are answered.
- BLUE and RED IPv6: the pings are successful.
- Now the negative test: from a BLUE source to a RED address, there is no reply. A test of reachability only is half a test. Always prove the isolation too.

Ep 12.7 · Experiment: the same address in both customers

```
! PE6
interface Loopback199
 description BLUE overlap test
 vrf BLUE
 ipv4 address 10.99.0.6 255.255.255.255
!
interface Loopback299
 description RED overlap test
 vrf RED
 ipv4 address 10.99.0.6 255.255.255.255
```

- Customers do not plan their addresses together. Thus we give the two customers the same address on PE6, on purpose.
- 10.99.0.6 in BLUE.
- 10.99.0.6 in RED. The same router has the same address two times.

Ep 12.8 · Two routes for one address

Source: lab-evidence/12-l3vpn/PE1-overlap.log

```
RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf BLUE 10.99.0.6/32 detail
      Received Label 0xe0040
      Extended community: RT:65000:100
      Source Route Distinguisher: 65000:106

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf RED 10.99.0.6/32 detail
      Received Label 0xe0060
      Extended community: RT:65000:200
      Source Route Distinguisher: 65000:206
```

- How does BGP keep them separate?
- The BLUE route arrived with PE6's BLUE route distinguisher. The RED route arrived with RED's. Thus BGP has two different routes.
- Each route has its own route target. Each VRF on PE1 imports only its own route.
- Each route has a different service SID function: e004 for BLUE and e006 for RED. They are in the label field, as Episode 3 showed.

Ep 12.9 · One address, two destinations

Router / tool: CE-A and tshark · Source: lab-evidence/12-l3vpn/CE-A-tests.log, build-PE1-P2.pcap

```
CE-A:~$ sudo ping -c 2 -I 10.1.1.2 10.99.0.6
2 packets transmitted, 2 packets received, 0% packet loss
CE-A:~$ sudo ping -c 2 -I 10.2.1.2 10.99.0.6
2 packets transmitted, 2 packets received, 0% packet loss

# outer destination on PE1 to P2, from the capture
from 10.1.1.2, BLUE   fcbb:bb00:6:e004::
from 10.2.1.2, RED    fcbb:bb00:6:e006::
```

- Now the wire. CE-A pings the same address, 10.99.0.6, with two different source addresses.
- From its BLUE address, the ping is answered by the BLUE loopback.
- From its RED address, the ping is answered by the RED loopback.
- In the capture, PE1 sends the BLUE packet to PE6's BLUE service SID, e004. It sends the RED packet to the RED service SID, e006. The same address, two customers and two SIDs: the router does not mix them up.

Ep 12.10 · Experiment: forget one import

```
! PE1
vrf BLUE
 address-family ipv4 unicast
  no import route-target 65000:100
```

- One more test shows the function of route targets. On PE1, we remove only BLUE's IPv4 import route target. The export and IPv6 do not change.
- Before the next page, predict what fails, and in which direction.

Ep 12.11 · Half a VPN

Router / tool: PE1, PE6 and CE-A · Source: lab-evidence/12-l3vpn/rt/

```
RP/0/RP0/CPU0:PE1#show route vrf BLUE
C    10.1.1.0/24 is directly connected, 00:11:03, GigabitEthernet0/0/0/2
L    10.1.1.1/32 is directly connected, 00:11:03, GigabitEthernet0/0/0/2
L    10.100.1.1/32 is directly connected, 00:11:07, Loopback101

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf BLUE 10.1.6.0/24
Paths: (0 available, no best path)

RP/0/RP0/CPU0:PE6#show route vrf BLUE
B    10.1.1.0/24 [200/0] via 2001:db8:100::1 (nexthop in vrf default), 00:08:19
B    10.100.1.1/32 [200/0] via 2001:db8:100::1 (nexthop in vrf default), 00:08:19

CE-A:~$ sudo ping -c 2 -W 1 -I 10.1.1.2 10.1.6.2
2 packets transmitted, 0 packets received, 100% packet loss
CE-A:~$ sudo ping6 -c 2 2001:db8:c:106::2
2 packets transmitted, 2 packets received, 0% packet loss
```

- These are the results from the two ends, after we removed the import of 65000:100 for BLUE IPv4 on PE1.
- PE1's BLUE table lost all the IPv4 routes from PE6. Only the local routes are there.
- BGP has no path for CE-B's subnet. PE1 did not keep PE6's route, because no VRF on PE1 imported that route target.
- PE6 does not know about the problem. It still has PE1's routes, because PE1 still exports them. The VPN is broken in one direction only.
- BLUE IPv4 pings fail.
- But BLUE IPv6 still works, because we did not change the IPv6 import. When we restored the line, all the routes came back in some seconds.
- When one address family of one customer fails, first examine the route targets of that address family on the two ends.

Ep 13.1 · Step 1: PE6 allocates and advertises

Router / tool: PE6 · Source: lab-evidence/13-chain/PE6-egress.log

```
RP/0/RP0/CPU0:PE6#show bgp vpnv4 unicast vrf BLUE 10.1.6.0/24 detail
BGP routing table entry for 10.1.6.0/24, Route Distinguisher: 65000:106
    SRv6-VPN SID: fcbb:bb00:6:e004::/64
    Format: f3216
    Alloc Mode/Locator ID: per-vrf/2
  Advertised to peers (in unique update groups):
    2001:db8:100::1
      Origin incomplete, metric 0, localpref 100, weight 32768, valid, redistributed, best
      Extended community: RT:65000:100
```

- Step 1, on PE6, where the prefix starts.
- The route is in BLUE, with PE6's BLUE route distinguisher.
- PE6 adds its BLUE service SID, fcbb:bb00:6:e004::. The SID comes from locator MAIN, with per-VRF allocation.
- PE6 advertises the route to PE1.
- The route target is 65000:100. Now we capture this advertisement on the wire.

Ep 13.2 · Step 2: make PE6 send an update

Router / tool: PE6 and tshark · Source: lab-evidence/13-chain/flap-shut.log, flap-noshut.log

```
RP/0/RP0/CPU0:PE6(config)#interface Loopback199 shutdown
RP/0/RP0/CPU0:PE6(config)#commit
RP/0/RP0/CPU0:PE6(config)#no interface Loopback199 shutdown
RP/0/RP0/CPU0:PE6(config)#commit

$ tshark -r bgp-update-P4-PE6.pcap -Y "bgp.type == 2" -V
```

- Step 2. To make PE6 send an update, we flap one of its BLUE loopbacks. In CML, we first start a capture on the link from PE6 to P4.
- We shut the loopback, and PE6 withdraws its routes.
- Then we remove the shutdown, and PE6 advertises the routes again. This is the update that we want.
- We read the capture with tshark, in verbose mode, with a filter for BGP updates. Next, the decoded update.

Ep 13.3 · Step 2: the update on the wire, part 1

Router / tool: capture · Source: lab-evidence/13-chain/bgp-update-decode.txt, from bgp-update-P4-PE6.pcap

```
Border Gateway Protocol - UPDATE Message
    Path Attribute - MP_REACH_NLRI
        Address family identifier (AFI): IPv4 (1)
        Subsequent address family identifier (SAFI): Labeled VPN Unicast (128)
        Next hop:  RD=0:0 IPv6=2001:db8:100::6
        Network Layer Reachability Information (NLRI)
            Label Stack: 917568 (bottom)
            Route Distinguisher: 65000:106
            MP Reach NLRI IPv4 prefix: 10.99.0.6
    Path Attribute - EXTENDED_COMMUNITIES
        Route Target: 65000:100
```

- This update carries 10.99.0.6, the overlap address from Episode 12. CE-B's subnet uses the same format.
- The address family is IPv4, labeled VPN unicast. MPLS VPNs use the same family.
- The next hop is PE6's IPv6 loopback. It is not a SID.
- The NLRI has a label, the route distinguisher and the prefix. The label is 917568. Remember this number.
- The route target is an extended community.

Ep 13.4 · Step 2: the update on the wire, part 2

Router / tool: capture · Source: lab-evidence/13-chain/bgp-update-decode.txt

```
Path Attribute - BGP Prefix-SID
    Type Code: BGP Prefix-SID (40)
    SRv6 L3 Service
        Type: SRv6 L3 Service (5)
        SRv6 Service Sub-TLV - SRv6 SID Information
            SRv6 SID Value: fcbb:bb00:6::
            SRv6 Endpoint Behavior: End.DT4 with NEXT-CSID (0x003f)
            SRv6 SID Structure
                Locator Block Length: 32
                Locator Node Length: 16
                Function Length: 16
                Argument Length: 0
                Transposition Length: 16
                Transposition Offset: 48
```

- This is part 2 of the same update: the SRv6 information.
- It is in the BGP Prefix-SID attribute, type 40, in an SRv6 L3 Service TLV, type 5. RFC 9252 defines them.
- The SID value is only fcbb:bb00:6::, PE6's locator. The behavior code is End.DT4 with NEXT-CSID. This is uDT4 on IOS XR.
- The structure is block 32, node 16, function 16, argument 0.
- There are two more numbers: transposition length 16 and offset 48. The function is not in the SID value. These two numbers tell where it is.

Ep 13.5 · Step 3: PE1's BGP path

Source: lab-evidence/13-chain/PE1-ingress.log

```
RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf BLUE 10.1.6.0/24 detail
BGP routing table entry for 10.1.6.0/24, Route Distinguisher: 65000:101
      Received Label 0xe0040
      Origin incomplete, metric 0, localpref 100, valid, internal, best, group-best, imported
      Extended community: RT:65000:100
       SID Locator: fcbb:bb00:6::/48, RIB metric: 31
        T:1(Sid information), Sid:fcbb:bb00:6::(Transposed), Behavior:63
           Length [Loc-blk,Loc-node,Func,Arg]:[32,16,16,0], Tpose-len:16, Tpose-offset:48
      Source AFI: VPNv4 Unicast, Source VRF: default, Source Route Distinguisher: 65000:106
```

- Step 3, on PE1.
- PE1 imported the route into BLUE, from PE6's route distinguisher into PE1's own route distinguisher.
- The received label is e0040. This is the same number as on the wire, in hexadecimal.
- BGP checked the locator: it is reachable, with metric 31. In Episode 3, this line showed inaccessible.
- This is the received transposed SID and its structure. PE1 now has all the information to build the full SID.

Ep 13.6 · Step 4: the VRF route

Source: lab-evidence/13-chain/PE1-ingress.log

```
RP/0/RP0/CPU0:PE1#show route vrf BLUE 10.1.6.0/24 detail
Routing entry for 10.1.6.0/24
  Known via "bgp 65000", distance 200, metric 0, type internal
    2001:db8:100::6, from 2001:db8:100::6
      Nexthop in Vrf: "default", Table: "default", IPv6 Unicast
      Source RD attributes: 0x0000:65000:106
      SRv6 Headend: H.Encaps.Red [f3216], SID-list {fcbb:bb00:6:e004::}
```

- Step 4, the VRF routing table.
- The route is in BLUE. PE1 learned it from BGP.
- Its next hop is PE6's loopback, in the default table. BGP next hops are loopbacks, as in MPLS VPNs.
- This is the full SID, fcbb:bb00:6:e004::, with the instruction to encapsulate: reduced form, f3216. This line is all the SRv6 information in the route.

Ep 13.7 · Step 5: CEF resolves through the locator

Source: lab-evidence/13-chain/PE1-ingress.log

```
RP/0/RP0/CPU0:PE1#show cef vrf BLUE 10.1.6.0/24 detail
10.1.6.0/24, version 31, SRv6 Headend
   via fcbb:bb00:6::/128, 9 dependencies, recursive
    next hop fcbb:bb00:6::/128 via fcbb:bb00:6::/48
    SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e004::}
    0     Y   GigabitEthernet0/0/0/0    fe80::5454:ff:fe66:75e1

RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
  Known via "isis CORE", distance 115, metric 31, SRv6-locator, type level-2
```

- Step 5, CEF.
- The entry does not resolve through PE6's loopback. It resolves through PE6's locator, fcbb:bb00:6::/48. Thus the locator must be reachable, not the loopback.
- This is the encapsulation instruction, to the service SID.
- This is the outgoing interface, GigabitEthernet 0/0/0/0 to P2. It comes from the IS-IS route to the locator. The chain on PE1 ends here: VPN route, locator route, interface.

Ep 13.8 · Step 6: the wire

Router / tool: CE-A and tshark · Source: lab-evidence/13-chain/CE-A.log, dp-*.pcap

```
CE-A:~$ sudo ping -c 2 -I 10.1.1.2 10.1.6.2
2 packets transmitted, 2 packets received, 0% packet loss

$ tshark -r <capture file> -Y "icmp.type == 8" -T fields -e frame.len -e ipv6.dst -e ipv6.hlim
# CE-A to PE1       98
# PE1 to P2        138   fcbb:bb00:6:e004::   255
# P4 to PE6        138   fcbb:bb00:6:e004::   253
# PE6 to CE-B       98
```

- Step 6, the packet. CE-A sends a ping, and we captured it at four locations.
- Both pings are answered.
- From CE-A, it is a normal 98-byte IPv4 ping.
- When it leaves PE1, it is 40 bytes larger, with the service SID as destination and an outer hop limit of 255.
- When it arrives at PE6, it is the same, but two routers decreased the outer hop limit.
- On the link to CE-B, it is a normal packet again. The customer packet did not change in the tunnel.

Ep 13.9 · Step 7: PE6 receives

Router / tool: PE6 · Source: lab-evidence/13-chain/PE6-egress.log

```
RP/0/RP0/CPU0:PE6#show cef ipv6 fcbb:bb00:6:e004::/64
fcbb:bb00:6:e004::/64, version 136, SRv6 Endpoint uDT4
    next hop VRF - 'BLUE', table - 0xe0000002
    0     Y   recursive                Lookup in table

RP/0/RP0/CPU0:PE6#show route vrf BLUE 10.1.6.2
Routing entry for 10.1.6.0/24
  Known via "connected", distance 0, metric 0 (connected)
    directly connected, via GigabitEthernet0/0/0/2
```

- Step 7, on PE6 again.
- The destination agrees with PE6's uDT4 entry. Its action is a lookup of the inner packet in BLUE.
- In BLUE, CE-B is directly connected. The chain is complete: it starts with an allocation on PE6, and it ends at a port of PE6.

Ep 13.10 · TTL, hop limit and DSCP

Router / tool: CE-A and tshark · Source: lab-evidence/13-chain/CE-A.log, dp-PE1-CE-A.pcap, dp-PE1-P2.pcap, dp-P4-PE6.pcap

```
CE-A:~$ sudo traceroute -n -t 184 -s 10.1.1.2 10.1.6.2 -w 1 -q 1 -m 5
 1  10.1.1.1
 2  10.46.0.1
 3  10.1.6.2

$ tshark -r <capture file> -Y "ip.dst == 10.1.6.2 and udp" -T fields \
    -e frame.len -e ipv6.dst -e ipv6.hlim -e ipv6.tclass -e ip.dsfield.dscp -e ip.ttl
# CE-A to PE1   60                                  DSCP 46   TTL 1, 2, 3
# PE1 to P2    100   fcbb:bb00:6:e004::   hop limit 255   tclass 0x0   DSCP 46   TTL 1, 2
# P4 to PE6    100   fcbb:bb00:6:e004::   hop limit 253   tclass 0x0   DSCP 46   TTL 1, 2
```

- Two header fields are important: the hop limit and the quality of service marking. We send traceroute probes from CE-A, with the DSCP mark EF, which is 46.
- The traceroute shows PE1, then PE6, then CE-B. The customer cannot see P2 and P4. RFC 2983 calls this the pipe model.
- On the access link, the probes have DSCP 46 and the TTL 1, 2 and 3.
- In the core, PE1 sets the outer hop limit to 255. It does not copy the customer TTL. P2 and P4 decrease only the outer hop limit: it is 253 at PE6.
- The inner TTL did not change. The inner DSCP is still 46, but the outer traffic class is 0. A core router that queues on the outer header sends this traffic as best effort.
- On this platform, the traffic class propagation is configurable, as the manager showed in Episode 2. If your core queues on the outer header, configure it, and verify it with a capture.

Ep 14.1 · The configuration on PE1

Source: lab-evidence/14-vpws/PE1-commit.log, PE1-state.log

```
interface GigabitEthernet0/0/0/4
 description TO-CE L2 attachment circuit
 l2transport
!
l2vpn
 xconnect group SRV6-VPWS
  p2p ELINE-300
   interface GigabitEthernet0/0/0/4
   neighbor evpn evi 300 service 300 segment-routing srv6
!
evpn
 segment-routing srv6
  locator MAIN
```

- This is all the configuration on PE1. PE6 has the same configuration, in the opposite direction. The BGP sessions already carry the Layer 2 VPN EVPN family.
- The customer port becomes a Layer 2 transport port. It has no IP address. PE1 does not route on this port.
- A point-to-point cross-connect connects the port to one remote service.
- This is EVPN instance 300, service 300, over SRv6. The service number must be the same on the two PEs.
- EVPN allocates its SIDs from locator MAIN. We did not configure route targets. EVPN made 65000:300 from the AS number and the instance.

Ep 14.2 · Route type 1 on the wire, part 1

Router / tool: capture · Source: lab-evidence/14-vpws/bgp-update-decode.txt, frame 179 of build-PE1-P2.pcap

```
Border Gateway Protocol - UPDATE Message
    Path Attribute - MP_REACH_NLRI
        Address family identifier (AFI): Layer-2 VPN (25)
        Subsequent address family identifier (SAFI): EVPN (70)
        Next hop: 2001:db8:100::1
        EVPN NLRI: Ethernet AD Route
            Route Type: Ethernet AD Route (1)
            Route Distinguisher: 10.0.0.1:300
            ESI: 00:00:00:00:00:00:00:00:00:00
            Ethernet Tag ID: 300
            MPLS Label 1: 917680
    Path Attribute - EXTENDED_COMMUNITIES
        EVPN Layer 2 Attributes: flags: 0x0002, L2 MTU: 0
        Route Target: 65000:300
```

- We captured on the core link of PE1 during the commit. In less than two seconds, PE1 sent this update.
- The family is Layer 2 VPN EVPN. The next hop is the loopback of PE1.
- The route is type 1, Ethernet auto-discovery. PE1 made the route distinguisher from its router ID and the instance.
- The Ethernet segment identifier is zero, because the CE connects to one PE only. The Ethernet tag is 300, the service number. The remote PE uses it to match the two ends.
- There is a label field again: 917680.
- There are two extended communities. The first is Layer 2 attributes, with MTU zero. Zero means: do not check. The second is the route target.

Ep 14.3 · Route type 1 on the wire, part 2

Router / tool: capture · Source: lab-evidence/14-vpws/bgp-update-decode.txt

```
Path Attribute - BGP Prefix-SID
    SRv6 L2 Service
        Type: SRv6 L2 Service (6)
        SRv6 Service Sub-TLV - SRv6 SID Information
            SRv6 SID Value: fcbb:bb00:1::
            SRv6 Endpoint Behavior: End.DX2 with NEXT-CSID (0x0041)
            SRv6 SID Structure
                Locator Block Length: 32
                Locator Node Length: 16
                Function Length: 16
                Argument Length: 0
                Transposition Length: 16
                Transposition Offset: 48
```

- Part 2 has the same Prefix-SID attribute as in Episode 13, with one difference.
- The TLV is SRv6 L2 Service, type 6. In Episode 13, it was L3 Service, type 5.
- The SID value is the locator of PE1. The behavior is End.DX2 with NEXT-CSID, code 65. On IOS XR, this is uDX2: remove the outer header and send the frame out of one port.
- The transposition is the same: 16 bits at offset 48. Thus the function is in the label field again.

Ep 14.4 · The cross-connect is up

Source: lab-evidence/14-vpws/PE1-verify.log

```
RP/0/RP0/CPU0:PE1#show l2vpn xconnect detail
Group SRV6-VPWS, XC ELINE-300, state is up; Interworking none
  AC: GigabitEthernet0/0/0/4, state is up
    MTU 1500; XC ID 0x2; interworking none
  EVPN: PW ID: evi 300, ac-id 300, state is up ( established )
    Encapsulation SRv6
      SRv6              Local                      Remote
      uDX2              fcbb:bb00:1:e00b::         fcbb:bb00:6:e007::
      AC ID             300                        300
      MTU               1514                       0
      Locator Resolved  Yes                        N/A
      SRv6 Headend      H.Encaps.L2.Red            N/A
```

- Now the state on PE1.
- The cross-connect is up. The customer port is also up.
- The EVPN side is up. The local SID is e00b and the remote SID is e007. These are the SIDs that we built from the capture.
- The service is 300 on the two ends. PE6 advertised MTU zero, thus there is no MTU check. Remember this. It is important later.
- The headend behavior is H.Encaps.L2.Red: encapsulate a Layer 2 frame, reduced form.

Ep 14.5 · BGP and CEF on PE1

Source: lab-evidence/14-vpws/PE1-bgp.log

```
RP/0/RP0/CPU0:PE1#show bgp l2vpn evpn rd 10.0.0.6:300 [1][0000.0000.0000.0000.0000][300]/120 detail
    2001:db8:100::6 (metric 30) from 2001:db8:100::6 (10.0.0.6)
      Received Label 0xe00700
      Extended community: EVPN L2 ATTRS:0x02:0 RT:65000:300
       SID Locator: fcbb:bb00:6::/48, RIB metric: 31
        T:1(Sid information), Sid:fcbb:bb00:6::(Transposed), Behavior:65

RP/0/RP0/CPU0:PE1#show cef ipv6 fcbb:bb00:1:e00b::/64 detail
fcbb:bb00:1:e00b::/64, version 243, SRv6 Endpoint uDX2
   via ::/128, GigabitEthernet0/0/0/4, 2 dependencies, weight 0, class 0
    AC Interface: Gi0/0/0/4
    XConnect ID: 0x2
```

- This is the same chain as for Layer 3, but the last step is different.
- PE1 received the type 1 route of PE6. The label shows in its three-byte form.
- This is the locator check, metric 31, and behavior 65. If the locator of PE6 is not reachable, this service fails. It is the same as fault A in Episode 3.
- This is the SID of PE1 in CEF. For uDT4, the action was a lookup in a VRF. For uDX2, there is no lookup. The action is a port, Gi0/0/0/4.

Ep 14.6 · From CE-A

Router / tool: CE-A · Source: lab-evidence/14-vpws/CE-A-ping.log

```
CE-A:~$ sudo ip neigh flush dev eth2
CE-A:~$ sudo ping -c 3 172.16.30.6
64 bytes from 172.16.30.6: seq=0 ttl=64 time=2.686 ms
64 bytes from 172.16.30.6: seq=1 ttl=64 time=1.449 ms
64 bytes from 172.16.30.6: seq=2 ttl=64 time=1.362 ms
CE-A:~$ sudo ping6 -c 3 2001:db8:30::6
3 packets transmitted, 3 packets received, 0% packet loss
CE-A:~$ sudo ip neigh show dev eth2
172.16.30.6 lladdr 52:54:00:34:24:24 ref 1 used 0/0/0 probes 4 REACHABLE
2001:db8:30::6 lladdr 52:54:00:34:24:24 ref 1 used 0/0/0 probes 4 REACHABLE
```

- Now we send traffic. First, we clear the neighbor table on CE-A. Then ARP must also go through the service.
- The table is empty.
- The IPv4 ping to CE-B gets three replies, with TTL 64. No router decreased it. For the customer, there is no router between the two CEs.
- IPv6 also operates. The PEs have no IPv6 configuration for this service.
- This shows that the service is Layer 2. CE-A learned the MAC address of CE-B, which ends in 24:24. In a Layer 3 VPN, CE-A learns only the MAC address of PE1.

Ep 14.7 · The frames, captured at three points

Router / tool: tshark · Source: lab-evidence/14-vpws/dp-PE1-Gi0_0_0_4-CE-A.pcap, dp-PE1-P2.pcap, dp-P4-PE6.pcap

```
$ tshark -r <capture file> -Y "arp or icmp or ipv6.nxt == 143" -T fields \
    -e frame.len -e ipv6.dst -e ipv6.nxt -e ipv6.hlim
# CE-A to PE1
  42                                      ARP request
  60                                      ARP reply
  98                                      echo request
# PE1 to P2
 114   fcbb:bb00:6:e007::   143   255     ARP request
 114   fcbb:bb00:1:e00b::   143   253     ARP reply
 152   fcbb:bb00:6:e007::   143   255     echo request
# P4 to PE6
 114   fcbb:bb00:6:e007::   143   253     ARP request
 152   fcbb:bb00:6:e007::   143   253     echo request
```

- We captured at three locations during the ping from CE-A, and we read each file with tshark.
- On the access link, there are an ARP request of 42 bytes, an ARP reply of 60 bytes, and the 98-byte ping.
- In the core, the ARP broadcast is unicast, to the SID of PE6, with the outer next header 143. A point-to-point service has only one destination for a broadcast.
- The reply from CE-B comes back to the SID of PE1, e00b.
- The 98-byte ping is 152 bytes in the core. The next header 143 identifies Ethernet. RFC 8986 defines this value.
- On the link P4 to PE6, the frames are the same, and the outer hop limit is two lower, as for Layer 3.

Ep 14.8 · Testing the edge

Router / tool: CE-A · Source: lab-evidence/14-vpws/CE-A-mtu.log, CE-A-ping2 in lab-evidence/13-evpn-vpws

```
CE-A:~$ sudo ping -c 2 -s 1418 172.16.30.6
PING 172.16.30.6 (172.16.30.6): 1418 data bytes
1426 bytes from 172.16.30.6: seq=0 ttl=64 time=1.296 ms
1426 bytes from 172.16.30.6: seq=1 ttl=64 time=1.639 ms
2 packets transmitted, 2 packets received, 0% packet loss
CE-A:~$ sudo ping -c 2 -s 1419 172.16.30.6
PING 172.16.30.6 (172.16.30.6): 1419 data bytes
2 packets transmitted, 0 packets received, 100% packet loss
```

- A ping with 1418 bytes of data is a 1446-byte IP packet: 1418, plus 8 for ICMP, plus 20 for IPv4.
- The 1446-byte packet goes through.
- With one more byte, all pings are lost. CE-A gets no error message. It only waits.

Ep 14.9 · Where the 1447-byte packet died

Router / tool: CE-A, tshark and routers · Source: lab-evidence/14-vpws/CE-A-mtu.log, mtu-PE1-P2.pcap, mtu-P4-PE6.pcap, PE1-mtu.log, PE6-mtu.log

```
CE-A:~$ sudo ping -c 2 -s 1419 172.16.30.6
2 packets transmitted, 0 packets received, 100% packet loss

$ tshark -r <capture file> -Y "ipv6.fraghdr" -T fields -e frame.len -e ipv6.src -e ipv6.dst -e ipv6.fraghdr.offset
# PE1 to P2
1510   2001:db8:100::1   fcbb:bb00:6:e007::   0
  75   2001:db8:100::1   fcbb:bb00:6:e007::   181
# P4 to PE6: the same two fragments

RP/0/RP0/CPU0:PE1#show ipv6 traffic
         2 fragmented into 4 fragments, 0 failed
RP/0/RP0/CPU0:PE6#show ipv6 traffic
         0 fragments, 0 total reassembled
```

- Where did the packet go? We have four items of evidence. First, the ping with 1419 bytes of data.
- All pings are lost, and CE-A receives no error.
- PE1 did not drop the frame. It fragmented the outer IPv6 packet into two fragments: 1510 and 75 bytes on the wire.
- The counters of PE1 agree: two packets, fragmented into four fragments. These are our two pings.
- The two fragments went through the core and arrived at PE6.
- But the IPv6 counters of PE6 show zero fragments. The forwarding plane matched the SID and discarded the fragments before reassembly. CE-B received nothing, and no device sent a message.
- This is the worst type of failure: it is silent, and it occurs only for large packets.

Ep 15.1 · The policy on PE1

Source: configs/policy/PE1.cfg

```
segment-routing
 traffic-eng
  segment-lists
   segment-list VIA-P3-P5-UN
    srv6
     index 10 sid fcbb:bb00:3::
     index 20 sid fcbb:bb00:5::
     index 30 sid fcbb:bb00:6::
  policy BLUE-C100
   srv6
    locator MAIN binding-sid dynamic behavior ub6-insert-reduced
   color 100 end-point ipv6 2001:db8:100::6
   candidate-paths
    preference 100
     explicit segment-list VIA-P3-P5-UN
```

- This is the policy on PE1. Our goal is to send BLUE traffic to PE6 on the lower links, through P3 and P5.
- The segment list has three node SIDs: P3, P5 and PE6.
- The Binding SID comes from locator MAIN. It is dynamic, with the insert-reduced behavior. Episode 4 showed that this platform gives only this form.
- The color is 100. The endpoint is the loopback of PE6.
- There is one candidate path, with preference 100. It uses this list.

Ep 15.2 · The policy is up

Source: lab-evidence/15-policy/v3/PE1-steer.log

```
RP/0/RP0/CPU0:PE1#show segment-routing traffic-eng policy color 100 detail
Color: 100, End-point: 2001:db8:100::6
    Admin: up  Operational: up for 00:35:12 (since Oct  3 07:46:12.843)
    Preference: 100 (configuration) (active)
      Explicit: segment-list VIA-P3-P5-UN (valid)
          SID[0]: fcbb:bb00:3::/48
          SID[1]: fcbb:bb00:5::/48
          SID[2]: fcbb:bb00:6::/48
        Binding SID behavior: uB6 (PSP/USD Insert.Red)
        Install Final Node SID: No
    Binding SID: fcbb:bb00:1:e008::
    Steering BGP disabled: no
    Invalidation drop enabled: no
```

- This is the state on PE1.
- The policy is up. The explicit path is active, and its list is valid.
- Look at this line: install final node SID, no. The node SID of PE6 is in the list, but PE1 does not put it in the packet. The service SID already goes to PE6.
- The Binding SID is fcbb:bb00:1:e008::. It is a SID on the locator of PE1.
- BGP steering is permitted. Invalidation drop is off. If the policy fails, traffic goes back to the usual path. We test this at the end.

Ep 15.3 · Colour on the egress

Router / tool: PE6 · Source: configs/policy/PE6.cfg; commit in lab-evidence/15-policy/v3/PE6-commit.log

```
extcommunity-set opaque COLOR-100
  100
end-set
!
route-policy BLUE-EXPORT-C100
  set extcommunity color COLOR-100
end-policy
!
vrf BLUE
 address-family ipv4 unicast
  export route-policy BLUE-EXPORT-C100
```

- Now the egress side. PE6 selects which of its routes use color 100.
- A color is an opaque extended community. Here, its value is 100.
- A route policy sets it.
- The BLUE IPv4 export uses this policy. Only BLUE IPv4 gets the color. BLUE IPv6, RED and GREEN do not change.

Ep 15.4 · The coloured update on the wire

Router / tool: capture · Source: lab-evidence/15-policy/v3/colour-update-decode.txt, frame 157 of colour-P4-PE6.pcap (condensed)

```
Border Gateway Protocol - UPDATE Message
    Path Attribute - MP_REACH_NLRI
        Next hop:  RD=0:0 IPv6=2001:db8:100::6
            Label Stack: 917568 (bottom)  RD 65000:106  10.99.0.6
            Label Stack: 917568 (bottom)  RD 65000:106  10.1.6.0
            Label Stack: 917568 (bottom)  RD 65000:106  10.100.6.6
    Path Attribute - EXTENDED_COMMUNITIES
        Color: 0x0000 0x0000 0x0064 [Transitive Opaque]
        Route Target: 65000:100
    Path Attribute - BGP Prefix-SID
        SRv6 SID Value: fcbb:bb00:6::
        SRv6 Endpoint Behavior: End.DT4 with NEXT-CSID (0x003f)
```

- We captured on the link from PE6 to P4 during the commit. PE6 advertised BLUE again immediately.
- One update carries three BLUE prefixes, with the same label. Episode 13 told about this: the prefixes use one set of attributes together.
- This is the new attribute: color, value 0x64. This is 100 in decimal.
- The service SID did not change. The color does not change what PE6 does with the packet. It changes the path that PE1 uses to PE6.

Ep 15.5 · PE1 steers into the policy

Source: lab-evidence/15-policy/v3/PE1-steer.log

```
RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf BLUE 10.1.6.0/24 detail
    2001:db8:100::6 C:100 (bsid: fcbb:bb00:1:e008::) (metric 30) from 2001:db8:100::6
      Extended community: Color:100 RT:65000:100
      SR policy color 100, up, not-registered, bsid fcbb:bb00:1:e008::

RP/0/RP0/CPU0:PE1#show route vrf BLUE 10.1.6.0/24 detail
    2001:db8:100::6, from 2001:db8:100::6 via srv6-sid fcbb:bb00:1:e008:: (resolution len 64)
      SRv6 Headend: H.Encaps.Red [f3216], SID-list {fcbb:bb00:6:e004::}

RP/0/RP0/CPU0:PE1#show cef vrf BLUE 10.1.6.0/24 detail
   via local-srv6-sid fcbb:bb00:1:e008::, 3 dependencies, recursive
    SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e004::}
      SRv6 H.Insert.Red SID-list {fcbb:bb00:3:5::}
    0     Y   GigabitEthernet0/0/0/0    fe80::5454:ff:fe66:75e1
```

- On PE1, we look at the same three tables as in Episode 13. Each table shows something new.
- First, BGP. The next hop now shows C 100 and the Binding SID. Color 100 and next hop PE6 match our policy, and the policy is up.
- Next, the VRF route. The service SID is the same. But the route now resolves through the Binding SID, not through the locator of PE6.
- CEF shows two steps. First, encapsulate with the service SID. Then, insert the list of the policy before it: fcbb:bb00:3:5::. This is the insert form of the Binding SID.
- The first link is GigabitEthernet 0/0/0/0, to P2. It is not the link to P3. Remember this.

Ep 15.6 · On the wire: every core link

Router / tool: CE-A and tshark · Source: lab-evidence/15-policy/v3/CE-A-c100.log, c100-*.pcap

```
CE-A:~$ sudo ping -c 3 -I 10.1.1.2 10.1.6.2
3 packets transmitted, 3 packets received, 0% packet loss

$ tshark -r <capture file> -Y "icmp.type == 8" -T fields -e ipv6.dst -e ipv6.hlim | sort -u
# PE1 to P2       fcbb:bb00:3:5:6:e004::   255
# P2 to P3        fcbb:bb00:3:5:6:e004::   254
# P3 back to P2   fcbb:bb00:5:6:e004::     253
# P2 to P4        fcbb:bb00:5:6:e004::     252
# P4 to P5        fcbb:bb00:5:6:e004::     251
# P5 back to P4   fcbb:bb00:6:e004::       250
# P4 to PE6       fcbb:bb00:6:e004::       249
```

- We captured on all eight core links while CE-A sent a ping to CE-B in BLUE. All three pings got a reply. We read each file with tshark. These are all the links that carried a request.
- PE1 sends the packet to P2. It does not send it to P3.
- P2 sends it to P3, with no change.
- P3 removes its micro-SID, and the packet goes back to P2.
- Then the packet goes to P4 and P5. P5 removes its micro-SID.
- Then it goes back to P4, and then to PE6. This is seven hops, not three, with two U-turns. The packet did not use the lower links.
- The hop limit shows this: it decreases from 255 to 249, one for each router.

Ep 15.7 · A strict path, with adjacency SIDs

Source: lab-evidence/15-policy/v3/pe1-strict.cfg, PE1-strict.log

```
  segment-lists
   segment-list VIA-P3-P5-UA
    srv6
     index 10 sid fcbb:bb00:3:e005::
     index 20 sid fcbb:bb00:5:e003::
  policy BLUE-C100
   candidate-paths
    preference 200
     explicit segment-list VIA-P3-P5-UA

RP/0/RP0/CPU0:PE1#show cef vrf BLUE 10.1.6.0/24 detail
    SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e004::}
      SRv6 H.Insert.Red SID-list {fcbb:bb00:3:e005:5:e003::}
```

- To use the lower links, we need adjacency SIDs, from Episode 4. Read them on P3 and P5 immediately before the test. Dynamic uA values can change after a reboot, or after an outage of more than three minutes.
- The adjacency SID of P3 to P5 is e005.
- The adjacency SID of P5 to PE6 is e003.
- We add them as a second candidate path, with preference 200. It becomes active, and we do not delete the first path.
- CEF now inserts fcbb:bb00:3:e005:5:e003::. The Binding SID did not change. Thus BGP and the VRF route also did not change.

Ep 15.8 · Strict path on the wire

Router / tool: CE-A and tshark · Source: lab-evidence/15-policy/v3/CE-A-strict.log, strict-*.pcap

```
CE-A:~$ sudo ping -c 3 -I 10.1.1.2 10.1.6.2
3 packets transmitted, 3 packets received, 0% packet loss

$ tshark -r <capture file> -Y "icmp.type == 8 or icmp.type == 0" \
    -T fields -e ipv6.dst -e ipv6.hlim -e icmp.type | sort -u
# PE1 to P2    fcbb:bb00:3:e005:5:e003:6:e004   255   request
# P2 to P3     fcbb:bb00:3:e005:5:e003:6:e004   254   request
# P3 to P5     fcbb:bb00:5:e003:6:e004::        253   request
# P5 to PE6    fcbb:bb00:6:e004::               252   request
# P4 to PE6    fcbb:bb00:1:e004::               255   reply
# P2 to P4     fcbb:bb00:1:e004::               254   reply
# PE1 to P2    fcbb:bb00:1:e004::               253   reply
```

- These are the addresses on the wire, for the same ping.
- One address has six micro-SIDs: P3, its adjacency, P5, its adjacency, PE6 and the service. The address is full, but there is no routing header. The size is still 138 bytes.
- P2 forwards it with no change.
- P3 removes two micro-SIDs: its node SID and its adjacency SID. Then it sends the packet on the link from P3 to P5.
- P5 does the same, to PE6.
- What about the replies? They went PE6, P4, P2, PE1, on the IGP path, to the BLUE SID of PE1. Only the export of PE6 adds a color. A policy steers one direction only. To steer both directions, add a color on the other side too.

Ep 15.9 · When the policy goes down

Source: lab-evidence/15-policy/v3/PE1-shut.log

```
RP/0/RP0/CPU0:PE1#show segment-routing traffic-eng policy color 100
    Admin: down  Operational: down for 00:00:08 (since Oct  3 08:23:46.018)
    Preference: 200 (configuration) (inactive) (shutdown)
    Preference: 100 (configuration) (inactive) (shutdown)

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf BLUE 10.1.6.0/24 detail
    2001:db8:100::6 (metric 30) from 2001:db8:100::6 (10.0.0.6)
      Extended community: Color:100 RT:65000:100

RP/0/RP0/CPU0:PE1#show cef vrf BLUE 10.1.6.0/24 detail
   via fcbb:bb00:6::/128, 11 dependencies, recursive
    next hop fcbb:bb00:6::/128 via fcbb:bb00:6::/48
    SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e004::}
```

- This is the last test: what occurs when the policy fails? We shut it down.
- The policy is admin down. The two candidate paths are down. PE1 releases the Binding SID.
- The route still has color 100. But BGP does not point it to a policy now.
- CEF goes back to the locator of PE6 and a plain encapsulation, as in Episode 13. The capture showed PE1, P2, P4, PE6, and all pings got a reply.
- On this platform, this fallback is the default, because invalidation drop is off. If a service must always stay on its path, for example for separation, set invalidation drop on. Then test that it drops the traffic.
- After no shutdown, the policy came back with the same Binding SID value. But the value is dynamic. Do not make a design that needs this number.

Ep 16.1 · The template on PE1

Source: lab-evidence/16-odn/v3/A-fresh.cfg; configs/odn-srh/PE6.cfg

```
segment-routing
 traffic-eng
  on-demand color 200
   srv6
    locator FA128 binding-sid dynamic behavior ub6-insert-reduced
   !
   dynamic
    metric
     type latency

extcommunity-set opaque COLOR-200
  200
end-set
vrf RED
 address-family ipv4 unicast
  export route-policy RED-EXPORT-C200
```

- This is the template. First, there is no PCE.
- The on-demand color is 200.
- Its Binding SID comes from the Flex-Algo locator, FA128.
- The intent is a dynamic path with the lowest latency.
- On PE6, the RED IPv4 routes have color 200 in the export. Thus each RED IPv4 route now asks PE1 for a low-latency path.

Ep 16.2 · Test A: the headend computes

Source: lab-evidence/16-odn/v3/A4-PE1.log

```
RP/0/RP0/CPU0:PE1#show segment-routing traffic-eng policy color 200 detail
Color: 200, End-point: 2001:db8:100::6
    Admin: up  Operational: down for 00:00:48 (since Oct  3 08:34:41.702)
    Preference: 200 (BGP ODN) (inactive)
      Last error: SRv6 SIDs failed verification
      Dynamic (inactive)
      Last error: HE computation not supported for SRV6 policy
        Metric Type: LATENCY (IANA PCEP/IGP: 12/1), Path Accumulated Metric: 0
    Preference: 100 (BGP ODN) (inactive)
      Last error: SRv6 SIDs failed verification
```

- Test A. There is no PCE. PE1 must calculate the path.
- PE1 made the policy for PE6, but the policy is down.
- The output gives the reason: Headend computation not supported for SRv6 policy.
- A second candidate path, with preference 100, also fails. We explain its function later.

Ep 16.3 · Test A: what the traffic does

Source: lab-evidence/16-odn/v3/A4-PE1.log, A-CE-A.log

```
RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf RED 10.2.6.0/24 detail
    2001:db8:100::6 (metric 30) from 2001:db8:100::6 (10.0.0.6)
      Extended community: Color:200 RT:65000:200

RP/0/RP0/CPU0:PE1#show cef vrf RED 10.2.6.0/24 detail
   via fcbb:bb00:6::/128, 7 dependencies, recursive
    next hop fcbb:bb00:6::/128 via fcbb:bb00:6::/48
    SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e006::}
    0     Y   GigabitEthernet0/0/0/0    fe80::5454:ff:fe66:75e1

CE-A:~$ sudo ping -c 3 -I 10.2.1.2 10.2.6.2
3 packets transmitted, 3 packets received, 0% packet loss
```

- What occurs to the RED traffic?
- The route still has color 200, but it has no policy.
- CEF goes back to the locator of PE6 and the plain encapsulation, out to P2. The capture showed PE1, P2, P4, PE6. This is the lowest-cost path. In this lab, it is also the highest-latency path.
- All pings get a reply. Nothing is broken, thus no person sees the problem. The customer pays for low latency, but gets high latency, and no device tells anyone.

Ep 16.4 · A minimal PCE

Router / tool: P4 · Source: lab-evidence/16-odn/v3/P4-state.log, PE1-state.log; configs/pce/P4.cfg

```
RP/0/RP0/CPU0:P4#show running-config pce
pce
 address ipv6 2001:db8:100::4

router isis CORE
 distribute link-state

RP/0/RP0/CPU0:PE1#show running-config segment-routing traffic-eng pcc
  pcc
   source-address ipv6 2001:db8:100::1
   pce address ipv6 2001:db8:100::4

RP/0/RP0/CPU0:P4#show pce ipv6 peer
Peer address: 2001:db8:100::1
  State: Up
  Capabilities: Stateful, Segment-Routing, Update, Instantiation, SRv6, SR-P2MP
```

- Thus we add a PCE. In a production network, it is a special server or router. Here, P4 is the PCE, with three lines of configuration.
- P4 becomes a PCE on its loopback.
- IS-IS gives its topology database to the PCE, with the delays and the SIDs.
- On PE1, the client side tells PE1 the address of the PCE.
- The session is up, and the two sides support SRv6.

Ep 16.5 · Test B: the PCE computes

Source: lab-evidence/16-odn/v3/B-PE1.log

```
RP/0/RP0/CPU0:PE1#show segment-routing traffic-eng policy color 200 detail
    Admin: up  Operational: up for 00:00:48 (since Oct  3 08:30:28.609)
    Preference: 200 (BGP ODN) (inactive) (shutdown)
    Preference: 100 (BGP ODN) (active)
      Dynamic (pce 2001:db8:100::4) (valid)
        Metric Type: LATENCY (IANA PCEP/IGP: 12/1), Path Accumulated Metric: 3000
          SID[0]: fcbb:bb00:1:e002::/64 Behavior: uA (PSP/USD) (57)
          SID[1]: fcbb:bb00:3:e004::/64 Behavior: uA (PSP/USD) (57)
          SID[2]: fcbb:bb00:5:e002::/64 Behavior: uA (PSP/USD) (57)
    Binding SID: fcbb:bb01:1:e004::
```

- Test B. This is the same template, with pcep under dynamic.
- The policy is up.
- Now we see the function of the two candidate paths. Preference 200 is the calculation on the headend, and it is shut. Preference 100 is delegated to the PCE, and it is active.
- The total latency is 3000 microseconds.
- The path has three adjacency SIDs: PE1 to P3, P3 to P5 and P5 to PE6. All three are the protected type.
- The Binding SID comes from the FA128 locator, as the template specifies.

Ep 16.6 · Test B in CEF

Source: lab-evidence/16-odn/v3/B-PE1.log

```
RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf RED 10.2.6.0/24 detail
    2001:db8:100::6 C:200 (bsid: fcbb:bb01:1:e004::) (metric 30) from 2001:db8:100::6
      SR policy color 200, up, registered, bsid fcbb:bb01:1:e004::

RP/0/RP0/CPU0:PE1#show cef vrf RED 10.2.6.0/24 detail
   via local-srv6-sid fcbb:bb01:1:e004::, 3 dependencies, recursive
    SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e006::}
      SRv6 H.Insert.Red SID-list {fcbb:bb00:e004:e002::}
    0     Y   GigabitEthernet0/0/0/1    fe80::5454:ff:fec1:58eb
```

- The steering is the same as in Episode 15.
- The RED route now points to the Binding SID of the ODN policy.
- This is the RED service SID, e006, and the inserted list: fcbb:bb00:e004:e002::. There are only two micro-SIDs for three adjacencies, and there are no node numbers. We decode this now.
- The first hop is GigabitEthernet 0/0/0/1, directly to P3.

Ep 16.7 · Test B on the wire

Router / tool: CE-A and tshark · Source: lab-evidence/16-odn/v3/B-CE-A.log, B-*.pcap

```
CE-A:~$ sudo ping -c 3 -I 10.2.1.2 10.2.6.2
3 packets transmitted, 3 packets received, 0% packet loss

$ tshark -r <capture file> -Y "icmp.type == 8 or icmp.type == 0" -T fields -e ipv6.dst -e ipv6.hlim -e icmp.type
# PE1 to P3     fcbb:bb00:e004:e002:6:e006::   255   request
# P3 to P5      fcbb:bb00:e002:6:e006::        254   request
# P5 to PE6     fcbb:bb00:6:e006::             253   request
# PE1 to P2     fcbb:bb00:1:e007::             253   reply
# P2 to P4      fcbb:bb00:1:e007::             254   reply
# P4 to PE6     fcbb:bb00:1:e007::             255   reply
```

- Now the wire, while CE-A sends a ping to CE-B in RED. We read each capture with tshark.
- All three pings are answered. From PE1 to P3, the address is full.
- P3 removes e004 and sends the packet on its link to P5.
- P5 removes e002 and sends the packet to PE6. This is three hops, on the low-latency path.
- The upper links carried only the replies, on the IGP path. As in Episode 15, the color steers one direction only.

Ep 16.8 · The PCE's view

Router / tool: P4 · Source: lab-evidence/16-odn/v3/B-P4.log

```
RP/0/RP0/CPU0:P4#show pce lsp detail
PCC 2001:db8:100::1:
Tunnel Name: bgp_c_200_ep_2001:db8:100::6_discr_100
   State: Admin up, Operation active
   Setup type: SRv6
   Binding SID: fcbb:bb01:1:e004:: Behavior: uB6 (PSP/USD Insert.Red) (79)
   Prefix-SID algorithm: 0 (set: 0)
   PCEP information:
     PLSP-ID 0x2, flags: D:1 S:0 R:0 A:1 O:2 C:0
   Computed path: (Local PCE)
     Metric type: LATENCY (12), Accumulated Metric 3000
```

- This is the same policy, as the PCE shows it.
- P4 knows the name of the ODN policy of PE1, and the policy is active.
- Look at this line: prefix-SID algorithm zero. The path uses usual algorithm 0 adjacency SIDs, selected by latency. It is not a Flex-Algo path.
- The D flag shows that PE1 delegated the control of this path to the PCE.
- The PCE calculated the path locally, with latency 3000.

Ep 16.9 · Test C: add sid-algorithm 128

Source: lab-evidence/16-odn/v3/C-algo.cfg, C-PE1.log, C-P4.log

```
  on-demand color 200
   dynamic
    pcep
    metric
     type latency
   constraints
    segments
     sid-algorithm 128

RP/0/RP0/CPU0:PE1#show segment-routing traffic-eng policy color 200 detail
    Admin: up  Operational: down for 00:01:17 (since Oct  3 08:37:24.784)
    Preference: 100 (BGP ODN) (inactive)
      Last error: SRv6 SIDs failed verification
        Prefix-SID Algorithm: 128

RP/0/RP0/CPU0:P4#show pce lsp detail
   State: Admin up, Operation down
   Prefix-SID algorithm: 128 (set: 1)
   Computed path: (Local PCE)
```

- Test C. Can the ODN policy use the SIDs of Flex-Algo 128? Then the path follows the algorithm, and the repair also follows it.
- We add one more block to the template: segments must be from algorithm 128.
- The policy goes down, with the text: SRv6 SIDs failed verification.
- On P4, the request arrives with algorithm 128, and the calculated path is empty. The RED traffic goes back to the shortest path again, with no alarm, as in test A.

Ep 16.10 · The PCE has the data

Router / tool: P4 · Source: lab-evidence/16-odn/v3/C-P4-topo.log

```
RP/0/RP0/CPU0:P4#show pce ipv6 topology
Node 7
  Host name: P5
      SR Algo Participation:
         0, 1, 128
      SID: fcbb:bb01:5:: Behavior: uN (PSP/USD) (48)
      LBL:32 LNL:16 FL:0 AL:80 Algo:128
  Link[0]: local address 2001:db8:200:35::1, remote address 2001:db8:200:35::
    Metric: IGP 40, TE 40, Latency 1000 microseconds
      SID[2]: fcbb:bb01:5:e000:: (protected) Behavior: uA (PSP/USD) (57)
      LBL:32 LNL:16 FL:16 AL:64 Algo:128
    Flex-algo Metric: Min Latency 1000
```

- Does the PCE have all the data? This is its topology database, during test C.
- P5 is in algorithm 128.
- Its algorithm 128 node SID is in the database.
- Its algorithm 128 adjacency SIDs and the link delays are also in it. This is true for each node. The data is complete, but the platform does not support the calculation.
- We also tried a different lab with XRv9000 routers. The result was the same failure.

Ep 16.11 · Way one, re-checked today

Source: lab-evidence/16-odn/v3/alt1-PE1.log, alt1-*.pcap (summarised)

```
RP/0/RP0/CPU0:PE1#show route vrf RED ipv6 2001:db8:c:206::/64 detail
      SRv6 Headend: H.Encaps.Red [f3216], SID-list {fcbb:bb01:6:e004::}

RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb01:6::/48
  Known via "isis CORE", distance 115, metric 3001, SRv6-locator (algo 128), type level-2
    fe80::5454:ff:fec1:58eb, from 2001:db8:100::6, via GigabitEthernet0/0/0/1, Protected

Captured, ping6 from CE-A to CE-B in RED:
  PE1 to P3   fcbb:bb01:6:e004::   hop limit 255
  P3 to P5    fcbb:bb01:6:e004::   hop limit 254
  P5 to PE6   fcbb:bb01:6:e004::   hop limit 253
```

- We examined method 1 again today, on the same lab.
- RED IPv6 uses the SID of PE6 from the FA128 block, fcbb:bb01.
- IS-IS routes that locator in algorithm 128, with metric 3001, through P3. It has TI-LFA protection in the same algorithm.
- On the wire, the destination does not change. No router removes a micro-SID. Each router only routes fcbb:bb01:6 in algorithm 128. This is the simplest method.

Ep 16.12 · A trap: up, but not on Algo 128

Source: configs/color-flexalgo/PE1.cfg; lab-evidence/16-odn-srh/color-flexalgo/PE1-verify.log, c400-*.pcap

```
segment-list PE6-ALGO128
 srv6
  index 10 sid fcbb:bb01:6::
policy GREEN-C400
 color 400 end-point ipv6 2001:db8:100::6

RP/0/RP0/CPU0:PE1#show cef vrf GREEN 10.30.6.6/32 detail
    SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e008::}
    0     Y   GigabitEthernet0/0/0/1    fe80::5454:ff:fec1:58eb

Captured: PE1 to P3, then P4 to PE6. Nothing on P3 to P5 or P5 to PE6.
```

- One more case. Can we write an explicit policy with one segment, the algorithm 128 node SID of PE6?
- The color is 400, with one segment: fcbb:bb01:6::. The policy came up.
- But look at CEF. It shows only the service SID, from the usual block. PE1 does not install the final node SID, as Episode 15 showed. Thus the packet has no information about algorithm 128.
- The packet went to P3. Then it followed algorithm 0 and arrived at PE6 from P4. The policy was up, but the path was not the path in its name. We removed it.

Ep 17.1 · Layer 1: local, on the egress

Router / tool: PE6 · Source: lab-evidence/17-oam/v3/L-PE6.log

```
RP/0/RP0/CPU0:PE6#show segment-routing srv6 locator
Name                  ID       Algo  Prefix                  Status   Flags
FA128                 1        128   fcbb:bb01:6::/48        Up       U
MAIN                  2        0     fcbb:bb00:6::/48        Up       U

RP/0/RP0/CPU0:PE6#show segment-routing srv6 manager
    Block fcbb:bb01::/32:
        Number of SIDs free: 7675
        Max SIDs: 7680
        Status: Resource Available
```

- Layer 1, on PE6. PE6 owns the SIDs.
- The two locators are up: MAIN in algorithm 0 and FA128 in algorithm 128. The healthy sign is the status Up.
- The manager shows the SID resources for each block, and many are free. Episode 19 uses these numbers again. If a locator is down or a block is full, stop here. The higher layers cannot operate.

Ep 17.2 · Layer 2: the IGP

Source: lab-evidence/17-oam/v3/L-PE6.log, L-PE1.log

```
RP/0/RP0/CPU0:PE6#show isis segment-routing srv6 locators
Name                  ID       Algo  Prefix                    Status
FA128                 1        128   fcbb:bb01:6::/48          Active
MAIN                  2        0     fcbb:bb00:6::/48          Active

RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
  Known via "isis CORE", distance 115, metric 31, SRv6-locator, type level-2
    fe80::5454:ff:fe66:75e1, from 2001:db8:100::6, via GigabitEthernet0/0/0/0, Protected
    fe80::5454:ff:fec1:58eb, from 2001:db8:100::6, via GigabitEthernet0/0/0/1, Backup (TI-LFA)
      Repair Node(s): 2001:db8:100::5
```

- Layer 2 is the IGP. We examine it on the two ends.
- On PE6, IS-IS shows the two locators as active. Fault A in Episode 3 failed at this point. The locator was up locally, but IS-IS did not advertise it.
- On PE1, the locator is a route from IS-IS, with metric 31.
- The primary path shows protected, and there is a TI-LFA backup through repair node P5. The healthy sign is the route on the ingress.
- A loopback ping does not replace this check. In Episode 3, the loopback ping was correct, but the service did not operate.

Ep 17.3 · Layer 3: forwarding

Source: lab-evidence/17-oam/v3/L-PE1.log, L-PE6.log

```
RP/0/RP0/CPU0:PE1#show cef ipv6 fcbb:bb00:6::/48
fcbb:bb00:6::/48, version 130, SRv6 Headend
   via fe80::5454:ff:fe66:75e1/128, GigabitEthernet0/0/0/0, 8 dependencies, protected
   via fe80::5454:ff:fec1:58eb/128, GigabitEthernet0/0/0/1, 13 dependencies, backup (TI-LFA)
    next hop fe80::5454:ff:fec1:58eb/128, Repair Node(s): 2001:db8:100::5
    SRv6 H.Insert.Red SID-list {fcbb:bb00:3:e005::}

RP/0/RP0/CPU0:PE6#show cef ipv6 fcbb:bb00:6:e004::/64
fcbb:bb00:6:e004::/64, version 136, SRv6 Endpoint uDT4
    next hop VRF - 'BLUE', table - 0xe0000002
    0     Y   recursive                Lookup in table
```

- Layer 3 is forwarding. The RIB shows what the router wants to do. CEF shows what it does.
- On PE1, the primary path goes through P2. The dependencies are the service routes that resolve through it.
- The backup is also installed, with its repair segment, as in Episode 9.
- On PE6, the entry for the service SID is a lookup in BLUE. The healthy signs are an interface on the ingress and the correct VRF on the egress. In Episode 3, the forwarding table showed unresolved and drop.

Ep 17.4 · Layer 4: the service

Source: lab-evidence/15-policy/v3/PE1-steer.log

```
RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf BLUE 10.1.6.0/24 detail
    2001:db8:100::6 C:100 (bsid: fcbb:bb00:1:e008::) (metric 30) from 2001:db8:100::6
      Received Label 0xe0040
      Extended community: Color:100 RT:65000:100
      SR policy color 100, up, not-registered, bsid fcbb:bb00:1:e008::
       SID Locator: fcbb:bb00:6::/48, RIB metric: 31
        T:1(Sid information), Sid:fcbb:bb00:6::(Transposed), Behavior:63
```

- Layer 4 is the service. One command shows most of it. This is BLUE from Episode 15.
- This is the SID, with its label and behavior. If it is incorrect or not there, the egress did not allocate it. This was fault B in Episode 3.
- These are the color and the route target. With an incorrect route target, the VRF imports nothing, as in Episode 12.
- If the route has a color, look at the policy. Remember Episodes 15 and 16: a policy can be up, but it can do something that you did not want.
- This is the locator check, metric 31. If it shows inaccessible, go back to layer 2.

Ep 17.5 · Layer 5: end to end

Source: lab-evidence/17-oam/v3/PE1-oam.log

```
RP/0/RP0/CPU0:PE1#ping vrf BLUE 10.1.6.2 source 10.1.1.1 count 3
!!!
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/2/3 ms

RP/0/RP0/CPU0:PE1#traceroute vrf BLUE 10.1.6.2 source 10.1.1.1 probe 1 timeout 1 maxttl 5
 1  10.46.0.1 3 msec
 2  10.1.6.2 2 msec
```

- Layer 5 is end to end.
- We send a ping in the VRF, from the BLUE address of PE1 to CE-B. It gets a reply.
- Then a VRF traceroute shows PE6, then CE-B. As Episode 13 showed, the core is not visible in the VRF.
- A small ping that goes through shows only that small packets go through in one direction. In Episodes 7 and 14, only large packets showed the faults. Test with different sizes. If you are not sure, capture.

Ep 17.6 · OAM: pinging SIDs

Source: lab-evidence/17-oam/v3/PE1-oam.log

```
RP/0/RP0/CPU0:PE1#ping fcbb:bb00:6:: source 2001:db8:100::1 count 3
!!!
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/2/3 ms
RP/0/RP0/CPU0:PE1#ping fcbb:bb00:6:e000:: source 2001:db8:100::1 count 3
!!!
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/2/3 ms
RP/0/RP0/CPU0:PE1#ping fcbb:bb00:6:e004:: source 2001:db8:100::1 count 3 timeout 1
!!!
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/4/5 ms
```

- Now the operations tools. First, can you send a ping to a SID?
- The node SID of PE6 gets a reply.
- One of the adjacency SIDs of PE6 gets a reply.
- The BLUE service SID, a uDT4, also gets a reply.

Ep 17.7 · OAM: traceroute

Source: lab-evidence/17-oam/v3/PE1-oam.log, PE1-trace-srv6.log

```
RP/0/RP0/CPU0:PE1#traceroute ipv6 fcbb:bb00:6:: source 2001:db8:100::1 probe 1 timeout 1 maxttl 6
Tracing the route to fcbb:bb00:6::
 1  2001:db8:200:12::1 3 msec
 2  2001:db8:200:24::1 2 msec
 3  2001:db8:100::6 2 msec

RP/0/RP0/CPU0:PE1#traceroute ipv6 fcbb:bb00:6:: source 2001:db8:100::1 probe 1 timeout 1 maxttl 6 srv6
% Invalid input detected at '^' marker.
RP/0/RP0/CPU0:PE1#traceroute ipv6 fcbb:bb00:6:: srv6
% Invalid input detected at '^' marker.

! In the help tree of this release (not run):
traceroute segment-routing srv6 policy name <name>
```

- Now traceroute. We make four points.
- Point 1: a plain traceroute works. To the node SID of PE6, it shows P2, P4, then PE6, on the IGP path.
- Point 2: Cisco documents two SRv6 forms of traceroute. Neither is in the ipv6 help tree of XRd 26.2.2. Our statement is only about the ipv6 syntax.
- Point 3: this release has traceroute segment-routing srv6 policy name, followed by the name. It traces a policy. We did not run it.
- Point 4: a document does not always match your release. Use the question mark on your router, and test the syntax.

Ep 17.8 · Traceroute into a container

Source: lab-evidence/17-oam/PE1-oam.log

```
RP/0/RP0/CPU0:PE1#traceroute fcbb:bb00:3:5:6:: source 2001:db8:100::1
 1  2001:db8:200:12::1 3 msec 1 msec 1 msec
 2  2001:db8:100::3 1 msec 1 msec 1 msec
 3  2001:db8:200:23:: 1 msec 1 msec 1 msec
 4  2001:db8:200:24::1 2 msec 1 msec 1 msec
 5  2001:db8:100::5 2 msec 4 msec 1 msec
 6  2001:db8:200:45:: 1 msec 1 msec 1 msec
 7   *  *  *
 8   *  *  *
```

- Now a traceroute to an address that we made manually: fcbb:bb00:3:5:6::. This is the loose path of Episode 15.
- Hop 1 is P2.
- Hop 2 is P3. It replies from its loopback, because the address was its node SID. Then hop 3 is P2 again. This is the first U-turn of Episode 15, and we see it without a capture.
- Then P4, then P5 from its loopback, then P4 again. This is the second U-turn.
- PE6 does not reply. This agrees with Episodes 4 and 5. The checksum of the probe uses the first destination, thus the last node discards it. We see the path, but not the last hop.

Ep 18.1 · Fault 1: capture the path

Router / tool: CE-A and tshark · Source: lab-evidence/18-faults/F1-CE-A.log, F1-PE1-P2.pcap, F1-P2-P4.pcap, F1-P4-PE6.pcap

```
CE-A:~$ sudo ping -c 2 -s 1300 172.16.30.6
2 packets transmitted, 2 packets received, 0% packet loss
CE-A:~$ sudo ping -c 2 -s 1400 172.16.30.6          (no reply)

$ tshark -r <capture file> -Y "ipv6.nxt == 143 or icmpv6.type == 2" -T fields -e frame.len -e ipv6.src -e ipv6.dst
# PE1 to P2
1396   2001:db8:100::1   fcbb:bb00:6:e007::    1300-byte ping, request
1396   2001:db8:100::6   fcbb:bb00:1:e00b::    1300-byte ping, reply
1496   2001:db8:100::1   fcbb:bb00:6:e007::    1400-byte ping, request
1294   2001:db8:100::1   (to PE1's loopback)   Packet Too Big, MTU 1336
# P2 to P4, and P4 to PE6
1396   the 1300-byte request and reply only
```

- Layer 5 tells us where to look: the wire. We captured on three core links while CE-A sent pings of three sizes. We read each file with tshark.
- The 1300-byte pings are answered. The 1400-byte pings get no reply.
- The 1300-byte pings are 1396-byte frames, and they cross the core in both directions.
- The 1400-byte request leaves PE1 as a 1496-byte frame, and it arrives at P2.
- P2 sent an ICMPv6 Packet Too Big to the encapsulation source, the loopback of PE1. For a Layer 2 frame, PE1 cannot use this message. Thus the customer gets no message.
- On the links from P2 to P4 and from P4 to PE6, there are only the small frames. The large frames stop at P2. Thus the fault is on P2, on its link to P4.

Ep 18.2 · Fault 1: the cause

Router / tool: P2 · Source: lab-evidence/18-faults/F1-P2.log, F1-PE1-P2.pcap, F1-fix.cfg

```
RP/0/RP0/CPU0:P2#show ipv6 interface GigabitEthernet0/0/0/1
  MTU is 1514 (1400 is available to IPv6)

Packet Too Big, captured on PE1 to P2:
    Type: Packet Too Big (2)
    MTU: 1336
        Source Address: 2001:db8:100::1
        Destination Address: fcbb:bb00:6:e007::

Fix on P2:
interface GigabitEthernet0/0/0/1
 no ipv6 mtu
```

- Layer 3 on P2 shows the cause.
- The interface MTU is still 1514, but IPv6 can use only 1400. Somebody set an IPv6 MTU on this one link. This is the fault. In a real network, it is often a change on one link that nobody relates to a customer.
- The Packet Too Big contains the outer packet from PE1, to the Layer 2 SID of PE6. Its MTU field shows 1336, not 1400.
- We tested this later on XRd and on XRv9000. Both platforms show the MTU minus 64 when the destination route has a TI-LFA repair path. With no repair path, the value is correct.
- Forwarding always uses the real MTU. We think that the 64 bytes are space for a repair header, but Cisco does not document this. Trust the capture, not only the reported value.
- We removed one line, and the 1418-byte ping went through again. The lesson: test with different sizes. When a size fails, capture on each link until the packet disappears.

Ep 18.3 · Fault 1: the configuration change

Router / tool: P2 · Source: lab-evidence/18-faults/F1-break.cfg

```
! On P2, toward P4
interface GigabitEthernet0/0/0/1
 ipv6 mtu 1400

# Largest customer IP packet over E-Line with this limit:
# 1400 - 40 (outer IPv6 header) - 14 (inner Ethernet header) = 1346 bytes
```

- This is the change on P2.
- One command sets the IPv6 MTU to 1400 on the link toward P4. The interface MTU stays at 1514.
- The arithmetic: 1400, minus 40 for the outer IPv6 header, minus 14 for the inner Ethernet header, is 1346. A customer packet larger than 1346 bytes cannot cross this link. This agrees with the pings: 1300 passed and 1400 failed.

Ep 18.4 · Fault 2: one direction

Router / tool: CE-A and tshark · Source: lab-evidence/18-faults/F2-CE-A.log, F2-*.pcap

```
CE-A:~$ sudo ping -c 3 -W 2 -I 10.1.1.2 10.1.6.2
3 packets transmitted, 0 packets received, 100% packet loss

$ tshark -r <capture file> -Y "icmp.type == 8 or icmp.type == 0" -T fields -e ipv6.dst -e icmp.type | sort | uniq -c
# CE-A to PE1       3 requests
# PE1 to P2         3 requests   fcbb:bb00:3:5:6:e004::
# P4 to PE6         3 requests   fcbb:bb00:6:e004::
# PE6 to CE-B       3 requests, and 3 replies from CE-B
```

- Fault 2. BLUE does not operate: CE-A cannot ping CE-B. On PE1, all the layers look correct for the prefix of CE-B. Thus we go directly to the wire.
- Three requests leave CE-A, and no replies come back.
- The requests go through the core to the BLUE service SID on PE6.
- CE-B receives them and replies three times. The replies arrive at PE6 and stop there. The forward direction is correct. The return direction is broken. A ping cannot show which direction is broken. A capture can.

Ep 18.5 · Fault 2: layer 4, on the other PE

Source: lab-evidence/18-faults/F2-PE6.log, F2-PE1.log

```
RP/0/RP0/CPU0:PE6#show route vrf BLUE 10.1.1.0/24
% Network not in table
RP/0/RP0/CPU0:PE6#show bgp vpnv4 unicast rd 65000:101 10.1.1.0/24
Paths: (0 available, no best path)

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf BLUE 10.1.1.0/24 detail
  Advertised to peers (in unique update groups):
    2001:db8:100::6
      Extended community: RT:65000:110
```

- PE6 sends the return traffic. Thus we examine the layers on PE6, for the prefix of CE-A.
- PE6 has no route to the subnet of CE-A in BLUE.
- BGP has no path for it.
- But PE1 shows that it advertised the route to PE6, with route target 65000:110. PE6 imports 65000:100. One digit is different.
- PE6 received the route, found no VRF that matches, and discarded it. We set the export on PE1 back to 65000:100, and the fault was repaired.
- The lesson: when a ping fails, examine the two directions. Examine the PE where the reply stops, also when the customer who reports the problem is on the other side.

Ep 18.6 · Fault 3: one VRF dead

Source: lab-evidence/18-faults/F3b-PE1.log, F3c-PE1.log

```
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 0 percent (0/3)
RP/0/RP0/CPU0:PE1#ping vrf BLUE 10.1.6.2 source 10.1.1.1 count 2
Success rate is 100 percent (2/2), round-trip min/avg/max = 2/2/2 ms
RP/0/RP0/CPU0:PE1#ping vrf RED 10.2.6.2 source 10.2.1.1 count 2
Success rate is 100 percent (2/2), round-trip min/avg/max = 3/5/7 ms

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
       SID Locator: fcbb:bb00:66::/48, RIB metric: inaccessible
RP/0/RP0/CPU0:PE1#show cef vrf GREEN 10.30.6.6/32
    unresolved
    SRv6 H.Encaps.Red SID-list {fcbb:bb00:66:e008::}
    0     Y   recursive                drop
```

- Fault 3. Only GREEN does not operate.
- GREEN fails.
- BLUE and RED to the same PE go through. Thus PE6 is reachable, and its main locator operates.
- Layer 4 for GREEN: the locator of the SID is fcbb:bb00:66::/48. It is not the MAIN locator of PE6, and it is inaccessible.
- CEF shows unresolved and drop. This is the same as fault A in Episode 3, but for one VRF only.

Ep 18.7 · Fault 3: down to layers 1 and 2

Router / tool: PE6 · Source: lab-evidence/18-faults/F3-PE6.log, F3c-PE6.log

```
RP/0/RP0/CPU0:PE6#show segment-routing srv6 locator
FA128                1        128   fcbb:bb01:6::/48          Up       U
MAIN                 2        0     fcbb:bb00:6::/48          Up       U
TEST                 3        0     fcbb:bb00:66::/48         Up       U

RP/0/RP0/CPU0:PE6#show isis segment-routing srv6 locators
FA128                1        128   fcbb:bb01:6::/48          Active
MAIN                 2        0     fcbb:bb00:6::/48          Active
TEST                 3        0     fcbb:bb00:66::/48         Inactive

RP/0/RP0/CPU0:PE6#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
    SRv6-VPN SID: fcbb:bb00:66:e008::/64
    Alloc Mode/Locator ID: per-vrf/3
```

- Inaccessible points to layer 2. Thus we go to the owner, PE6.
- Layer 1 shows a third locator, TEST. It is up locally.
- Layer 2: IS-IS knows the locator, but shows it as inactive. Nobody added it under IS-IS. Thus the other routers cannot reach it.
- The BGP configuration of GREEN allocates from it, locator ID 3. This is an incomplete change. Somebody made a new locator and moved a VRF to it, but did not do the IGP step.
- We moved GREEN back to MAIN, and the fault was repaired. During the change, for approximately forty seconds, PE1 used the old SID, and PE6 had already removed it.
- A change of SID is also an outage. Plan it.

Ep 18.8 · Fault 3: the configuration changes

Router / tool: PE6 · Source: lab-evidence/18-faults/F3-break.cfg, F3b-break.cfg (the nesting is shortened)

```
! Step 1, on PE6: a new locator, and RED allocates from it
segment-routing srv6 locators locator TEST
 micro-segment behavior unode psp-usd
 prefix fcbb:bb00:66::/48
router bgp 65000 vrf RED address-family ipv4 unicast
 segment-routing srv6 locator TEST
! (the locator is NOT added under router isis: the missing step)

! Step 2, on PE6: RED goes back to MAIN, and GREEN moves to TEST
router bgp 65000 vrf RED address-family ipv4 unicast
 segment-routing srv6 locator MAIN
router bgp 65000 vrf GREEN address-family ipv4 unicast
 segment-routing srv6 locator TEST
```

- These are the configuration changes of fault 3, all on PE6. We made them in two steps.
- Step 1 creates a locator TEST, and RED IPv4 allocates its service SID from it.
- Look at the last line. The locator is not added under IS-IS. This is the missing step. Thus no other router has a route to it.
- Step 2 puts RED back on MAIN, and moves GREEN to TEST. Now only GREEN depends on the broken locator. The twist after this page explains why RED survived in step 1.

Ep 18.9 · Fault 3: the repair

Router / tool: PE6 · Source: lab-evidence/18-faults/F3-fix1.cfg, F3-fix2.cfg (the nesting is shortened)

```
! Repair 1, on PE6: GREEN back to MAIN
router bgp 65000 vrf GREEN address-family ipv4 unicast
 segment-routing srv6 locator MAIN

! Repair 2, on PE6: remove the unused locator
segment-routing srv6 locators
 no locator TEST
```

- Now the repair, also on PE6, in two steps.
- Repair 1 moves GREEN back to the MAIN locator. The traffic recovers after the new SID is advertised.
- Repair 2 removes the locator TEST, which nobody uses. Do this last. Remove a locator only after no VRF allocates from it.

Ep 18.10 · Fault 4: hidden until a link fails

Source: lab-evidence/18-faults/F4-before.log, F4-latent.log

```
Before the change:
RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb01:6::/48
    fe80::5454:ff:fe66:75e1, from 2001:db8:100::6, via GigabitEthernet0/0/0/0, Backup (TI-LFA)
    fe80::5454:ff:fec1:58eb, from 2001:db8:100::6, via GigabitEthernet0/0/0/1, Protected

After the change on P2 and P4, everything still passes:
RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb01:6::/48
    fe80::5454:ff:fec1:58eb, from 2001:db8:100::6, via GigabitEthernet0/0/0/1
```

- Fault 4 is the most difficult type. When we make the change, nothing fails. We removed Flex-Algo 128 from P2 and P4. An engineer can make this change to make those routers simpler. RED IPv6 uses the algorithm 128 locator, and it continued to operate.
- Before the change, the route on PE1 to the algorithm 128 locator of PE6 was protected, with a TI-LFA backup through P2.
- After the change, there is one path and no protection. The algorithm 128 topology became one line: PE1, P3, P5, PE6. All pings still went through.
- Only layer 2 showed the problem, and only when we compared it with the reference.

Ep 18.11 · Fault 4: the link fails

Source: lab-evidence/18-faults/F4-event-*.log

```
P3: interface GigabitEthernet0/0/0/1 shutdown   (the link to P5)

CE-A:~$ sudo ping6 -c 3 -W 2 -I 2001:db8:c:201::2 2001:db8:c:206::2
3 packets transmitted, 0 packets received, 100% packet loss

RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb01:6::/48
% Network not in table
RP/0/RP0/CPU0:PE1#show cef vrf RED ipv6 2001:db8:c:206::/64
    unresolved
    0     Y   recursive                drop

BLUE: 2 of 2.  RED IPv4: 2 of 2 (the PCE recomputed: latency 15000, via PE6's node SID).
```

- Now the topology event: the link from P3 to P5 fails.
- We shut it on P3.
- RED IPv6 does not operate.
- Layer 2: the algorithm 128 locator is not there. P2 and P4 are not in the algorithm, thus the algorithm has no other path. Layer 3 shows unresolved and drop.
- All the other services continue. BLUE uses algorithm 0. RED IPv4 uses the policy from the PCE, and the PCE calculated a new path on the remaining links.
- The customer that paid for the special path is the customer that lost service. We set the link and the algorithm back, and the route was protected again.

Ep 18.12 · The four configuration changes

```
! fault 1, on P2: a smaller IPv6 MTU toward P4
interface GigabitEthernet0/0/0/1
 ipv6 mtu 1400
! fault 2, on PE1: a different export route target
vrf BLUE
 address-family ipv4 unicast
  export route-target
   no 65000:100
   65000:110
! fault 3, on PE6: a locator that IS-IS does not advertise
! (full configuration on the Fault 3 pages)
locator TEST  prefix fcbb:bb00:66::/48
router bgp 65000 vrf RED address-family ipv4 unicast
 segment-routing srv6 locator TEST
! fault 4, on P2 and P4
router isis CORE
 no flex-algo 128
```

- Here are the four changes, as the answer key. You can repeat each one in your lab, and then find it with the five layers, before you look at this page.
- Fault 1: on P2, the IPv6 MTU of the link toward P4 is 1400.
- Fault 2: on PE1, the export route target of BLUE IPv4 changes from 65000:100 to 65000:110.
- Fault 3: on PE6, RED allocates its service SID from a new locator. IS-IS does not advertise this locator.
- Fault 4: on P2 and P4, we remove Flex-Algo 128. The service is fine until a link fails.

Ep 19.1 · The untrusted port

Source: configs/boundary/PE1-outside.cfg; lab-evidence/19-boundary/v3/CE-A-setup.log

```
interface GigabitEthernet0/0/0/4
 description UNTRUSTED outside
 ipv6 address 2001:db8:ffff:1::1/64

Outside host, on CE-A's third port:
CE-A:~$ sudo ip -6 addr add 2001:db8:ffff:1::2/64 dev eth2
CE-A:~$ sudo ip -6 route add fcbb:bb00::/31 via 2001:db8:ffff:1::1
CE-A:~$ sudo ip -6 route add 2001:db8:100::/64 via 2001:db8:ffff:1::1
```

- Now the test. The port for the Layer 2 service in Episode 14 becomes a port that is not trusted, in the global table of PE1. In a real network, it can be an internet peering, a management network or a customer that makes mistakes.
- It has a plain IPv6 address, with no VRF.
- The outside host gets an address on that link.
- The host also gets routes to the two SID blocks and to the loopbacks, through PE1. A real attacker does not need these routes. A default route does the same.

Ep 19.2 · Before: the outside reaches the core

Router / tool: CE-A · Source: lab-evidence/19-boundary/v3/before-CE-A.log

```
CE-A:~$ sudo ping6 -c 2 -W 2 fcbb:bb00:6::
64 bytes from 2001:db8:100::6: seq=0 ttl=61 time=3.693 ms
CE-A:~$ sudo ping6 -c 2 -W 2 fcbb:bb00:4::
64 bytes from 2001:db8:100::4: seq=0 ttl=62 time=1.757 ms
CE-A:~$ sudo ping6 -c 2 -W 2 fcbb:bb00:6:e000::
64 bytes from 2001:db8:100::6: seq=0 ttl=61 time=1.801 ms
CE-A:~$ sudo ping6 -c 2 -W 2 fcbb:bb00:6:e004::
64 bytes from 2001:db8:100::6: seq=0 ttl=61 time=1.656 ms
```

- There is no filter yet. The outside host tries the SIDs of the core.
- The node SID of PE6 gets a reply, from the loopback of PE6.
- The node SID of P4, a transit router, gets a reply.
- One of the adjacency SIDs of PE6 gets a reply.
- The BLUE service SID also gets a reply. From outside the network, an unknown person touched the door to the VRF of a customer.
- As Episode 17 showed, a ping does not cause a VRF lookup. But it shows that packets from outside arrive at the SID. A host that also makes the inner packet needs nothing more. We did not do that attack. It was not necessary.

Ep 19.3 · Before, on the wire

Router / tool: tshark · Source: lab-evidence/19-boundary/v3/before-PE1-P2.pcap, before-P4-PE6.pcap

```
$ tshark -r <capture file> -Y "icmpv6.type == 128 or icmpv6.type == 129" \
    -T fields -e ipv6.src -e ipv6.dst -e icmpv6.type | sort | uniq -c
# PE1 to P2: requests and replies
  2  2001:db8:ffff:1::2   fcbb:bb00:6::         128
  2  2001:db8:ffff:1::2   fcbb:bb00:4::         128
  2  2001:db8:100::6      2001:db8:ffff:1::2    129
  2  2001:db8:100::4      2001:db8:ffff:1::2    129
# P4 to PE6
  2  2001:db8:ffff:1::2   fcbb:bb00:6::         128
  2  2001:db8:100::6      2001:db8:ffff:1::2    129
```

- This is the capture from that test. We read each file with tshark. Type 128 is a request, and type 129 is a reply.
- On the link PE1 to P2, the requests of the outside host went into the core with no change, to the SIDs. The loopbacks of the routers replied.
- The same request arrived at PE6. Nothing at the edge stopped a packet from outside the domain to an instruction in the domain.

Ep 19.4 · What the platform lists

Router / tool: PE6 · Source: lab-evidence/17-oam/v3/L-PE6.log

```
RP/0/RP0/CPU0:PE6#show segment-routing srv6 manager
  Security rules:
    SEC-1
    SEC-2
    SEC-3
```

- Does the router protect itself? The SRv6 manager shows security rules.
- These are SEC-1 to SEC-3. The names come from RFC 8754, section 5.1. It describes filters at the domain edge and on SR nodes.
- The list shows what the software supports. Our test shows the result without our filter: each SID replied to the outside host. A rule in the list is not a configured rule. Episode 14 gave the same lesson about capabilities.

Ep 19.5 · The boundary ACL

Source: configs/boundary/PE1-acl.cfg

```
ipv6 access-list SRV6-BOUNDARY-IN
 10 deny ipv6 any fcbb:bb00::/31 log
 20 deny ipv6 any any routing log
 30 permit ipv6 any any
!
interface GigabitEthernet0/0/0/4
 ipv6 access-group SRV6-BOUNDARY-IN ingress
```

- The repair is an access list on each interface that faces outside the SR domain.
- Line 10: no packet from outside can go to the SID space. One slash 31 prefix includes the two blocks: fcbb:bb00 for algorithm 0 and fcbb:bb01 for algorithm 128. This is an advantage of blocks that are next to each other.
- Line 20: no packet with a routing header can go in. On this release, the ACL did not accept routing-type 4. It accepted the general routing keyword.
- All other packets are permitted.
- We apply the ACL inbound on the port that is not trusted.

Ep 19.6 · After: the door is shut

Router / tool: CE-A · Source: lab-evidence/19-boundary/v3/after-CE-A.log

```
CE-A:~$ sudo ping6 -c 2 -W 2 -I 2001:db8:ffff:1::2 fcbb:bb00:6::
2 packets transmitted, 0 packets received, 100% packet loss
CE-A:~$ sudo ping6 -c 2 -W 2 -I 2001:db8:ffff:1::2 fcbb:bb00:4::
2 packets transmitted, 0 packets received, 100% packet loss
CE-A:~$ sudo ping6 -c 2 -W 2 -I 2001:db8:ffff:1::2 fcbb:bb00:6:e000::
2 packets transmitted, 0 packets received, 100% packet loss
CE-A:~$ sudo ping6 -c 2 -W 2 -I 2001:db8:ffff:1::2 fcbb:bb00:6:e004::
2 packets transmitted, 0 packets received, 100% packet loss
CE-A:~$ sudo ping6 -c 2 -W 2 -I 2001:db8:ffff:1::2 2001:db8:100::6
2 packets transmitted, 2 packets received, 0% packet loss
```

- We do the same tests with the ACL.
- The four SIDs get no reply.
- The loopback of PE6 still replies. The boundary stops only the SID space. Usual reachability does not change.
- Look at the -I option. Without it, the outside host used a different address as the source. PE6 had no route back to that address, thus the loopback ping also failed. When a test fails, examine the source address before you blame the filter.

Ep 19.7 · After, on the wire

Router / tool: tshark · Source: lab-evidence/19-boundary/v3/after-PE1-Gi0_0_0_4-CE-A.pcap, after-PE1-P2.pcap

```
$ tshark -r <capture file> -Y "icmpv6.type == 128" -T fields -e ipv6.src -e ipv6.dst | sort | uniq -c
# outside host to PE1, on the port that is not trusted
  2  2001:db8:ffff:1::2   fcbb:bb00:4::
  2  2001:db8:ffff:1::2   fcbb:bb00:6::
  1  2001:db8:ffff:1::2   fcbb:bb00:6:e000::
# PE1 to P2
(no echo request)
```

- Did the ACL drop the packets? We use captures to see this, with the same command.
- The requests arrived at the port of PE1 that is not trusted.
- None of them appeared on the core link of PE1. Thus PE1 dropped them at the edge.
- On this platform, the deny counters of the ACL showed no matches. Verify the filter with a packet from outside and a capture inside. Do not use a counter as your only evidence.

Ep 19.8 · Planning the SID space for the boundary

Router / tool: PE6 · Source: lab-evidence/17-oam/v3/L-PE6.log; configs/boundary/PE1-acl.cfg

```
RP/0/RP0/CPU0:PE6#show segment-routing srv6 manager
  SRv6 Operational Mode:
    Micro-segment:
      SID Base Block: fcbb:bb00::/24

Blocks in use:     fcbb:bb00::/32 (algorithm 0)   fcbb:bb01::/32 (algorithm 128)
Our ACL line:      deny ipv6 any fcbb:bb00::/31
Whole base block:  deny ipv6 any fcbb:bb00::/24
```

- One more design point: how large must the deny prefix be?
- The manager shows the micro-SID base block: fcbb:bb00::/24. All the blocks that this router can use are in it.
- We use two blocks today, and our ACL line includes only those two.
- If you deny all the base block, you also include each block that you add later. This can be another algorithm, or a migration block in Module 6.
- The small prefix is precise. With the large prefix, you do not forget a block when the network grows. Select one, and record the reason.

Ep 19.9 · Scale: the numbers on PE6

Router / tool: PE6 · Source: lab-evidence/17-oam/v3/L-PE6.log

```
RP/0/RP0/CPU0:PE6#show segment-routing srv6 manager
      uSID LIB Range:
        LIB Start   : 0xe000
        ELIB Start  : 0xfe00
Summary:
  Number of Locators: 2 (2 operational)
  Number of SIDs: 14 (0 stale)
  Max SID resources: 64000
  Number of free SID resources: 63986
    Block fcbb:bb00::/32:
        Number of SIDs free: 7671
        Max SIDs: 7680
  Max Locators: 16
```

- Now scale. The layer 1 command from Episode 17 shows the numbers.
- The local SID range starts at e000 and stops before fe00. All the functions in this series came from this range: e004, e007, e00b. They are local, and each router uses the same values again, as Episode 16 showed.
- PE6 uses 14 SIDs from 64000 resources.
- Each block has 7680 SIDs. This is the size of the range from e000 to fdff. Thus the real limit for each locator is the local range, not the total.
- The maximum is 16 locators.

Ep 20.1 · The legacy configuration on PE1

Source: lab-evidence/20-ldp-migration/PE1-commit.log

```
interface GigabitEthernet0/0/0/0
 ipv4 address 10.12.0.0 255.255.255.254
router isis CORE
 address-family ipv4 unicast
  metric-style wide
 interface GigabitEthernet0/0/0/0
  address-family ipv4 unicast
   metric 10
router bgp 65000
 neighbor 10.0.0.6
  remote-as 65000
  update-source Loopback0
  address-family vpnv4 unicast
mpls ldp
 router-id 10.0.0.1
 interface GigabitEthernet0/0/0/0
```

- This is the MPLS configuration on PE1, with some lines removed. The other routers have the same pattern.
- There are IPv4 addresses on the core links.
- IPv4 is in the same IS-IS instance, with the same metrics as IPv6.
- There is a VPNv4 session to the IPv4 loopback of PE6.
- LDP is on the core interfaces. GREEN has no SRv6 allocation on any router.

Ep 20.2 · GREEN on MPLS

Source: lab-evidence/20-ldp-migration/PE1-mpls.log

```
RP/0/RP0/CPU0:PE1#show mpls ldp neighbor brief
10.0.0.2:0         N   N    00:01:26    1     0     4     0     14     0
10.0.0.3:0         N   N    00:01:24    1     0     4     0     14     0

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
    10.0.0.6 (metric 30) from 10.0.0.6 (10.0.0.6), if-handle 0x00000000
      Received Label 24000

RP/0/RP0/CPU0:PE1#show cef vrf GREEN 10.30.6.6/32 detail
    next hop 10.0.0.6/32 via 24011/0/21
     next hop 10.12.0.1/32 GigabitEthernet0/0/0/0 labels imposed {24009 24000}
```

- This is GREEN on MPLS, as an older network shows it.
- There are LDP sessions to P2 and P3.
- The GREEN route from PE6 has next hop 10.0.0.6 and a real MPLS label, 24000.
- CEF pushes two labels. 24009 is the LDP label to PE6, and 24000 is the VPN label. Compare this with Episode 13: one IPv6 header with one SID.

Ep 20.3 · Both worlds on one wire

Source: lab-evidence/20-ldp-migration/PE1-mpls.log, mpls-PE1-P2.pcap, mpls-P2-P4.pcap, mpls-P4-PE6.pcap

```
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/2/3 ms
RP/0/RP0/CPU0:PE1#ping vrf BLUE 10.100.6.6 source 10.100.1.1 count 2
Success rate is 100 percent (2/2), round-trip min/avg/max = 2/2/3 ms

$ tshark -r <capture file> -Y "mpls or ipv6.nxt == 4" -T fields -e frame.len -e mpls.label -e ipv6.dst -e icmp.type
# PE1 to P2        122   24009,24000          GREEN request, MPLS
#                  154   fcbb:bb00:6:e004::   BLUE request, SRv6
# P2 to P4         122   24009,24000          GREEN request
#                  154   fcbb:bb00:6:e004::   BLUE request
# P4 to PE6        118   24000                GREEN request, top label popped
#                  154   fcbb:bb00:6:e004::   BLUE request
```

- We captured on three core links while PE1 sent 100-byte pings to GREEN and to BLUE. We read each file with tshark.
- Both pings work. When they leave PE1, GREEN has two labels, 122 bytes. BLUE has an IPv6 header, 154 bytes.
- It is the same on the link from P2 to P4.
- On the last link, GREEN has no top label. P4 removed it: this is penultimate-hop popping. BLUE does not change from end to end. The two technologies use each router and each link, and they do not affect each other.
- Look at the sizes. The SRv6 packet is 32 bytes larger than the MPLS packet with two labels. When GREEN moves, it loses 32 bytes of MTU. Examine this before you migrate, with the method of Episode 7.

Ep 20.4 · Reading the labels hop by hop

Router / tool: tshark · Source: lab-evidence/20-ldp-migration/mpls-PE1-P2.pcap, mpls-P2-P4.pcap, mpls-P4-PE6.pcap

```
$ tshark -r <capture file> -Y "mpls" -T fields -e frame.len -e mpls.label -e icmp.type
# GREEN request, from PE1 to PE6
PE1 to P2     122   24009,24000
P2 to P4      122   24009,24000
P4 to PE6     118   24000          (the top label was popped by P4)
# GREEN reply, from PE6 to PE1
P4 to PE6     122   24001,24000
P2 to P4      122   24000,24000
PE1 to P2     118   24000          (the top label was already popped)
```

- We look at MPLS again, because it shows what SRv6 removes.
- PE1 sends 24009 on top. This is the label of P2 for PE6. Below it is 24000, the VPN label of PE6.
- P2 replaces the top label with the label of P4 for PE6. This label is also 24009. Each router has its own labels. The equal numbers are a coincidence.
- P4 removes the top label before PE6.
- The replies show the same pattern in the opposite direction, and PE6 starts with 24001. Each router keeps a label table for each PE. With SRv6, the transit routers forwarded one IPv6 destination that did not change. They used their usual routing table.

Ep 20.5 · Step 1: PE6 moves first

Source: lab-evidence/20-ldp-migration/step1-PE6-commit.log, step1-PE1.log

```
router bgp 65000
 vrf GREEN
  address-family ipv4 unicast
   segment-routing srv6
    locator MAIN
    alloc mode per-vrf

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
    10.0.0.6 (metric 30) from 10.0.0.6 (10.0.0.6), if-handle 0x00000000
      Received Label 0xe0080
       SID Locator: fcbb:bb00:6::/48, RIB metric: 31
RP/0/RP0/CPU0:PE1#show cef vrf GREEN 10.30.6.6/32 detail
    SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e008::}
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 100 percent (3/3), round-trip min/avg/max = 1/3/6 ms
```

- Step 1. Only PE6 changes.
- PE6 allocates an SRv6 SID for GREEN. These are the same five lines as for BLUE in Episode 12.
- On PE1, the GREEN route now arrives with a SID. Look at the next hop: it is still 10.0.0.6, the IPv4 session. PE6 advertised the SID on the two sessions, and PE1 selected the path from the IPv4 session.
- This is not important. The SID selects the encapsulation. The session does not.
- CEF now encapsulates with the GREEN SID of PE6.
- GREEN operates.

Ep 20.6 · Step 1: one route, two paths

Source: lab-evidence/20-ldp-migration/step1-PE1.log

```
RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast rd 65000:306 10.30.6.6/32
Paths: (2 available, best #1)
  Path #1: Received by speaker 0
    10.0.0.6 (metric 30) from 10.0.0.6 (10.0.0.6)
      Received Label 0xe0080
      Origin incomplete, metric 0, localpref 100, valid, internal, best, group-best
  Path #2: Received by speaker 0
    2001:db8:100::6 (metric 30) from 2001:db8:100::6 (10.0.0.6)
      Received Label 0xe0080
      Origin incomplete, metric 0, localpref 100, valid, internal
```

- Why did PE1 use the path from the IPv4 session? This is the route from PE6 on PE1, before the import into GREEN.
- The same route has two paths, one from each session.
- Path 1 is over IPv4, with the SID. It is the best path.
- Path 2 is over IPv6, with the same SID. All the attributes in the output are equal. Thus the last BGP tie-break rules selected the path.
- The two paths go to the same SID and make the same packet. During a migration, two sessions are usual. Know which path is best, and why.

Ep 20.7 · Step 1 on the wire: half and half

Source: lab-evidence/20-ldp-migration/step1-PE1.log, step1-PE1-P2.pcap

```
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 100 percent (3/3), round-trip min/avg/max = 1/3/6 ms

$ tshark -r step1-PE1-P2.pcap -Y "mpls or ipv6.nxt == 4" \
    -T fields -e frame.len -e mpls.label -e ipv6.dst -e ip.src | sort | uniq -c
  3   154                     fcbb:bb00:6:e008::   10.30.1.1    PE1 to PE6, requests: SRv6
  3   118   24000                                  10.30.6.6    PE6 to PE1, replies: MPLS
```

- The wire shows the result of step 1, on the link PE1 to P2. The ping works.
- The requests from PE1 to PE6 are SRv6, because their egress, PE6, moved.
- The replies from PE6 to PE1 are still MPLS, because their egress, PE1, did not move. One customer and one ping use two technologies, and the ping operates. One end can move first.

Ep 20.8 · Step 2: PE1 follows

Source: lab-evidence/20-ldp-migration/step2-PE1.log, step2-PE1-P2.pcap

```
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 100 percent (3/3), round-trip min/avg/max = 1/3/6 ms
RP/0/RP0/CPU0:PE1#ping vrf BLUE 10.100.6.6 source 10.100.1.1 count 2
Success rate is 100 percent (2/2), round-trip min/avg/max = 2/2/3 ms

$ tshark -r step2-PE1-P2.pcap -Y "ipv6.nxt == 4" -T fields -e frame.len -e ipv6.dst -e ip.src | sort | uniq -c
  3   154   fcbb:bb00:6:e008::   10.30.1.1     GREEN request
  3   154   fcbb:bb00:1:e00a::   10.30.6.6     GREEN reply
  2   154   fcbb:bb00:6:e004::   10.100.1.1    BLUE request
  2   154   fcbb:bb00:1:e004::   10.100.6.6    BLUE reply
```

- Step 2: we configure the same five lines on PE1. Then we run the same two pings.
- The GREEN requests go to the GREEN SID of PE6, as before.
- Now the replies go to the new GREEN SID of PE1, e00a. The two directions are SRv6.
- BLUE did not change. After this step, the GREEN and BLUE pings went through.

Ep 20.9 · Rollback of step 1

Source: lab-evidence/20-ldp-migration/rollback-PE6-commit.log, rollback-PE6-PE1.log

```
router bgp 65000
 vrf GREEN
  address-family ipv4 unicast
   no segment-routing srv6

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
    10.0.0.6 (metric 30) from 10.0.0.6 (10.0.0.6), if-handle 0x00000000
      Received Label 24000
RP/0/RP0/CPU0:PE1#show cef vrf GREEN 10.30.6.6/32 detail
     next hop 10.12.0.1/32 GigabitEthernet0/0/0/0 labels imposed {24009 24000}
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/4/6 ms
```

- Now the test that each change plan needs: can we go back? PE1 had already moved, and we rolled back step 1 on PE6.
- One line on PE6 removes the SRv6 allocation of GREEN.
- PE1 receives label 24000 again.
- CEF uses the two labels again. LDP was still in operation, thus the transport label was ready.
- GREEN goes through. On the wire, the requests were MPLS again. The replies stayed SRv6, because we did not roll back PE1. The two directions use different technologies again, but in the opposite way.
- We applied step 1 again, and the two directions were SRv6 again.

Ep 20.10 · Rollback on the wire

Source: lab-evidence/20-ldp-migration/rollback-PE6-PE1.log, rollback-PE6-PE1-P2.pcap

```
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/4/6 ms

$ tshark -r rollback-PE6-PE1-P2.pcap -Y "mpls or ipv6.nxt == 4" \
    -T fields -e frame.len -e mpls.label -e ipv6.dst -e ip.src | sort | uniq -c
  3   122   24009,24000                              10.30.1.1    GREEN request: MPLS again
  3   154                     fcbb:bb00:1:e00a::   10.30.6.6    GREEN reply: still SRv6
```

- This is the capture during the rollback. The ping works.
- The requests to PE6 are MPLS again, with two labels, because PE6 withdrew its SID.
- The replies to PE1 stay SRv6, to the GREEN SID of PE1. The rule is also true in the opposite direction: each direction follows its egress, when the egress moves forward or back.

Ep 20.11 · Step 3: remove the legacy

Source: lab-evidence/20-ldp-migration/step3-*-commit.log, step3-PE1.log, step3-PE1-mpls.log

```
router bgp 65000
 no neighbor 10.0.0.6
no mpls ldp                                (on all six routers)

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
    2001:db8:100::6 (metric 30) from 2001:db8:100::6 (10.0.0.6), if-handle 0x00000000
      Received Label 0xe0080
RP/0/RP0/CPU0:PE1#show mpls forwarding
RP/0/RP0/CPU0:PE1#show mpls ldp neighbor brief
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/2/3 ms
```

- Step 3: remove the MPLS configuration, in sequence.
- First, remove the IPv4 BGP session on the two PEs.
- Then remove LDP on all six routers.
- The GREEN route now comes from the IPv6 loopback of PE6, with its SID.
- The MPLS forwarding table is empty, and there are no LDP neighbors.
- GREEN and BLUE go through. On the wire, GREEN is SRv6 in the two directions. The migration is complete.

Ep 21.1 · Step 1: SR-MPLS configuration

Source: lab-evidence/21-srmpls-migration/P2-commit.log, start-PE1-commit.log

```
On every router, here P2:
router isis CORE
 address-family ipv4 unicast
  segment-routing mpls
 interface Loopback0
  address-family ipv4 unicast
   prefix-sid index 2

On PE1, mirrored on PE6:
router bgp 65000
 neighbor 10.0.0.6
  remote-as 65000
  update-source Loopback0
  address-family vpnv4 unicast
 vrf GREEN
  address-family ipv4 unicast
   no segment-routing srv6
```

- The IPv4 core from Episode 20 remains. This time, it has no LDP.
- We enable segment routing for MPLS under the IS-IS IPv4 address family.
- Each loopback gets a prefix SID index. P2 uses 2. PE6 uses 6.
- The PEs use an IPv4 BGP session for VPNv4 routes with labels.
- GREEN has no SRv6 allocation. Thus it uses MPLS labels.

Ep 21.2 · Step 1: the result

Source: lab-evidence/21-srmpls-migration/srmpls-start-PE1.log

```
RP/0/RP0/CPU0:PE1#show isis segment-routing label table
Label         Prefix                  Algorithm    Interface
16006         10.0.0.6/32             SPF

RP/0/RP0/CPU0:PE1#show mpls forwarding prefix 10.0.0.6/32
Local  Outgoing    Prefix             Outgoing     Next Hop        Bytes
16006  16006       SR Pfx (idx 6)     Gi0/0/0/0    10.12.0.1       872

RP/0/RP0/CPU0:PE1#show cef vrf GREEN 10.30.6.6/32 detail
    next hop 10.0.0.6/32 via 16006/0/21
     next hop 10.12.0.1/32 GigabitEthernet0/0/0/0 labels imposed {16006 24000}

RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/2/2 ms
```

- This is the result on PE1.
- The loopback of PE6 has label 16006. The default label block starts at 16000. Add the index of PE6, which is 6.
- PE1 replaces label 16006 with label 16006. The values match because these routers use the same SRGB. The advertised index has domain-wide meaning.
- GREEN packets get two labels. Label 16006 reaches PE6. Label 24000 identifies the VPN service on PE6.
- The ping receives replies.

Ep 21.3 · Step 1: on the wire

Source: lab-evidence/21-srmpls-migration/srmpls-start-PE1.log, srmpls-start-PE1-P2.pcap, srmpls-start-P2-P4.pcap

```
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/2/2 ms

$ tshark -r <capture file> -Y "mpls" -T fields -e frame.len -e mpls.label -e ip.src
# PE1 to P2
122   16006,24000   10.30.1.1    GREEN request
118   24000         10.30.6.6    GREEN reply
# P2 to P4
122   16006,24000   10.30.1.1    GREEN request
122   16001,24000   10.30.6.6    GREEN reply
```

- We examine two core links, with a GREEN ping from PE1, and we read each file with tshark.
- At PE1, label 16006 is above label 24000. The replies carry the label 24000 only, because the router before PE1 removed the top label.
- On the next link, the top label is still 16006. These routers use the same SRGB and associate index 6 with PE6. LDP allocated labels independently in Episode 20. The replies carry label 16001 for PE1.

Ep 21.4 · Step 2: move GREEN to SRv6

Source: lab-evidence/21-srmpls-migration/migrate-PE6-commit.log, migrate-PE1-commit.log, srmpls-migrated-PE1.log

```
On PE6 and PE1:
router bgp 65000
 vrf GREEN
  address-family ipv4 unicast
   segment-routing srv6
    locator MAIN
    alloc mode per-vrf

RP/0/RP0/CPU0:PE1#show cef vrf GREEN 10.30.6.6/32 detail
10.30.6.6/32, version 19, SRv6 Headend
    SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e008::}
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/2/3 ms
```

- The migration uses the same five lines as Episode 20.
- We configure SRv6 allocation for GREEN on each PE. Episode 20 demonstrated separate PE migrations. The same rule applies here: the egress decides the encapsulation.
- CEF now encapsulates GREEN with the SID of PE6.
- GREEN receives ping replies.

Ep 21.5 · Step 2: on the wire

Router / tool: tshark · Source: lab-evidence/21-srmpls-migration/srmpls-migrated-PE1-P2.pcap, srmpls-migrated-P2-P4.pcap

```
$ tshark -r <capture file> -Y "ipv6.nxt == 4" -T fields -e frame.len -e ipv6.dst -e ip.src
# PE1 to P2
154   fcbb:bb00:6:e008::   10.30.1.1    GREEN request
154   fcbb:bb00:1:e00a::   10.30.6.6    GREEN reply
# P2 to P4
154   fcbb:bb00:6:e008::   10.30.1.1    GREEN request
154   fcbb:bb00:1:e00a::   10.30.6.6    GREEN reply
```

- These are the same two links after migration, read with the same command.
- Both directions use SRv6. Each destination is the GREEN SID of the egress PE. These customer packets have no MPLS labels.
- P2 forwards the packet without changing its destination, as with BLUE in Episode 13.

Ep 21.6 · Step 3: who still uses label 16006?

Source: lab-evidence/21-srmpls-migration/srmpls-start-PE1.log, srmpls-migrated-PE1.log

```
Before the move:
RP/0/RP0/CPU0:PE1#show mpls forwarding prefix 10.0.0.6/32
Local  Outgoing    Prefix            Outgoing     Next Hop        Bytes
16006  16006       SR Pfx (idx 6)    Gi0/0/0/0    10.12.0.1       872

After the move, GREEN on SRv6:
RP/0/RP0/CPU0:PE1#show mpls forwarding prefix 10.0.0.6/32
Local  Outgoing    Prefix            Outgoing     Next Hop        Bytes
16006  16006       SR Pfx (idx 6)    Gi0/0/0/0    10.12.0.1       1415
```

- GREEN uses SRv6 now. Does SR-MPLS still carry traffic?
- Before migration, the label entry for PE6 showed 872 switched bytes.
- After migration, the counter showed 1415 bytes. Some traffic still used label 16006. We tested the cause later. During sixty seconds with no customer traffic, the counter increased by 99 bytes.
- The capture showed two labeled packets for the IPv4 BGP session to 10.0.0.6. A BGP keepalive was 59 bytes. A TCP acknowledgement was 40 bytes. Together, they account for the 99-byte increase.
- After we removed that session, the counter stopped increasing. The old transport still carried control traffic. Identify all traffic that uses the old transport before you remove it.

Ep 21.7 · Step 3: remove the IPv4 VPN session

Router / tool: PE1 · Source: lab-evidence/label-16006/S2-PE1-nobgp.log, S2-before.log, S2-after.log

```
! On PE1. PE6 does the same with neighbor 10.0.0.1
router bgp 65000
 no neighbor 10.0.0.6

RP/0/RP0/CPU0:PE1#show mpls forwarding prefix 10.0.0.6/32     (00:28:51)
16006  16006       SR Pfx (idx 6)     Gi0/0/0/0    10.12.0.1       1251

RP/0/RP0/CPU0:PE1#show mpls forwarding prefix 10.0.0.6/32     (00:29:53)
16006  16006       SR Pfx (idx 6)     Gi0/0/0/0    10.12.0.1       1251

RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 2
Success rate is 100 percent (2/2), round-trip min/avg/max = 2/2/2 ms
```

- Step 3 removes the last user of label 16006: the IPv4 VPN session.
- On PE1, we remove the neighbor 10.0.0.6. PE6 removes the neighbor 10.0.0.1.
- Now we read the counter of label 16006 twice, about one minute apart. It is 1251 both times. The counter stopped.
- GREEN still works, over SRv6.

Ep 21.8 · Step 4: remove SR-MPLS

Router / tool: PE1 · Source: lab-evidence/label-16006/PE1-cleanup.log, after-PE1.log

```
! On every router, here PE1
RP/0/RP0/CPU0:PE1(config)#no router isis CORE address-family ipv4 unicast
RP/0/RP0/CPU0:PE1(config)#no router isis CORE interface Loopback0 address-family ipv4 unicast
RP/0/RP0/CPU0:PE1(config)#no router isis CORE interface GigabitEthernet0/0/0/0 address-family ipv4 unicast
RP/0/RP0/CPU0:PE1(config)#no router isis CORE interface GigabitEthernet0/0/0/1 address-family ipv4 unicast
RP/0/RP0/CPU0:PE1(config)#commit

RP/0/RP0/CPU0:PE1#show running-config router isis CORE
 (only SRv6 lines remain: segment-routing srv6, microloop avoidance)
```

- Step 4 removes SR-MPLS. Do it last, and only after the counter stopped.
- On each router, we remove the IPv4 address family from IS-IS, on the instance and on each interface. This also removes the prefix SIDs and segment routing for MPLS.
- Then we commit.
- The running configuration of IS-IS now has only the SRv6 lines.

Ep 22.1 · The legacy IGP, added to every router

Router / tool: P3 · Source: lab-evidence/22-ospf-migration/P3-commit.log

```
router ospf LEGACY
 router-id 10.0.0.3
 area 0
  interface Loopback0
   passive enable
  interface GigabitEthernet0/0/0/0
   cost 40
   network point-to-point
  interface GigabitEthernet0/0/0/2
   cost 10
   network point-to-point
router ospfv3 LEGACY
 router-id 10.0.0.3
 area 0
  interface GigabitEthernet0/0/0/0
   cost 40
```

- All six routers run OSPF for IPv4 and OSPFv3 for IPv6. This is the configuration on P3.
- We use area 0 and make the loopback passive.
- The link costs match the IS-IS metrics. The lower links use 40. The other links use 10. We verify that both protocols select the same paths. Then the migration changes route ownership without changing those paths.
- OSPFv3 uses the same link costs. The IS-IS configuration remains unchanged.

Ep 22.2 · Two IGPs: who owns what

Source: lab-evidence/22-ospf-migration/both-igps-PE1.log

```
RP/0/RP0/CPU0:PE1#show ospf neighbor
10.0.0.2        1     FULL/  -        00:00:37    10.12.0.1       GigabitEthernet0/0/0/0
10.0.0.3        1     FULL/  -        00:00:37    10.13.0.1       GigabitEthernet0/0/0/1

RP/0/RP0/CPU0:PE1#show route ipv6 2001:db8:100::6/128
  Known via "ospf LEGACY", distance 110, metric 30, type intra area
    fe80::5454:ff:fe66:75e1, from ::, via GigabitEthernet0/0/0/0

RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
  Known via "isis CORE", distance 115, metric 31, SRv6-locator, type level-2
    fe80::5454:ff:fe66:75e1, from 2001:db8:100::6, via GigabitEthernet0/0/0/0, Protected
    fe80::5454:ff:fec1:58eb, from 2001:db8:100::6, via GigabitEthernet0/0/0/1, Backup (TI-LFA)

RP/0/RP0/CPU0:PE1#show ospfv3 database | include fcbb
```

- Which protocol supplies each route when both IGPs operate?
- OSPF adjacencies show Full. IS-IS adjacencies also remain up.
- OSPF supplies the selected loopback route for PE6. Both protocols know the prefix. OSPF has distance 110, which is lower than the IS-IS distance of 115. This selected route has one path and no backup.
- IS-IS still supplies the locator route for PE6, with a TI-LFA backup. In this configuration, OSPF does not advertise the locator.
- The OSPFv3 database has no SID block prefixes. A BLUE ping receives replies.

Ep 22.3 · Step 1: prefer IS-IS everywhere

Source: lab-evidence/22-ospf-migration/step1-PE1-commit.log, step1-isis-preferred-PE1.log

```
On every router:
router isis CORE
 address-family ipv4 unicast
  distance 100
 address-family ipv6 unicast
  distance 100

RP/0/RP0/CPU0:PE1#show route ipv6 2001:db8:100::6/128
  Known via "isis CORE", distance 100, metric 30, type level-2
    fe80::5454:ff:fe66:75e1, from 2001:db8:100::6, via GigabitEthernet0/0/0/0, Protected
    fe80::5454:ff:fec1:58eb, from 2001:db8:100::6, via GigabitEthernet0/0/0/1, Backup (TI-LFA)
RP/0/RP0/CPU0:PE1#show route ipv4 10.0.0.6/32
  Known via "isis CORE", distance 100, metric 30, labeled SR
```

- Step 1 makes IS-IS preferred. Both protocols remain configured.
- We set IS-IS distance 100 in both address families on all six routers. It is lower than the OSPF distance of 110. OSPF adjacencies remain Full, so its routes remain available.
- IS-IS now supplies the selected loopback route for PE6. The route has TI-LFA protection.
- The IPv4 loopback also uses IS-IS. It retains the SR-MPLS label from Episode 21. The service tests pass.
- The IPv4 metric is 30. Under OSPFv2, it was 31 because OSPFv2 added the loopback interface cost of 1. Equal link costs did not produce equal totals. Compare total costs and selected paths before you rely on a mixed network.

Ep 22.4 · Step 2: remove OSPF

Source: lab-evidence/22-ospf-migration/step2-PE1-commit.log, step2-ospf-removed-PE1.log

```
On every router:
no router ospf LEGACY
no router ospfv3 LEGACY

RP/0/RP0/CPU0:PE1#show ospf neighbor
RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
  Known via "isis CORE", distance 100, metric 31, SRv6-locator, type level-2
RP/0/RP0/CPU0:PE1#ping vrf BLUE 10.100.6.6 source 10.100.1.1 count 2
Success rate is 100 percent (2/2), round-trip min/avg/max = 2/3/4 ms
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 2
Success rate is 100 percent (2/2), round-trip min/avg/max = 3/3/4 ms
```

- Step 2 removes OSPF.
- We enter two lines on each router.
- No OSPF neighbors remain. The selected routing table does not change because IS-IS already supplies these routes.
- BLUE and GREEN receive replies. Step 1 changed route preference. Step 2 removes the unused protocol. This final removal should not change the selected paths.

Ep 23.1 · The same command, on both

Router / tool: PE6 · Source: lab-evidence/17-oam/v3/L-PE6.log

```
RP/0/RP0/CPU0:PE6#show segment-routing srv6 manager
  Signaled parameters:
    Max-SL          : 3
    Max-End-Pop-SRH : 3
    Max-H-Insert    : 3 sids
    Max-H-Encap     : 3 sids
    Max-End-D       : 4
  Configurable parameters (under srv6):
    Encapsulation:
      Hop-Limit     : value=Yes, propagate=No
      Traffic-class : value=Yes, propagate=Yes
  Max Locators: 16
  Max SIDs: 64000
```

- Run the same manager command on both platforms. This is the XRd output from Episode 17.
- The output lists segments-left, insertion, encapsulation and decapsulation limits. IS-IS advertises these capabilities. A headend or PCE can use them when it constructs a path.
- The output also describes header propagation. Hop limit propagation is disabled here. Traffic class propagation is available. Episode 13 measured the resulting DSCP behavior.
- The resource limits appear below. Compare this output with the hardware output before you complete the design. Record the platform and release with each output.



Three sentences to keep

The program has reached its last node.

- The DA is a program: find the active instruction and who owns it.
- Reachable, installed and delivered are three different facts.
- Predict before you look.